GDAŃSK UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRONICS, TELECOMMUNICATIONS AND INFORMATICS

Student's name and surname: Jakub Grzybowski
ID: 184670
Cycle of studies: master's degree studies
Mode of study: full-time studies
Field of study: Informatics
Specialization: Computer Networks

MASTER'S THESIS

Title of the thesis: Optimization of location of controllers in Software Defined Networks aimed at reducing the latency of serving the demands

Title of the thesis (in Polish): Optymalizacja lokalizacji kontrolerów w programowalnych sieciach komputerowych ukierunkowana na redukcję opóźnienia obsługi żądań

Supervisor: dr hab. inż. Jacek Rak

GDAŃSK UNIVERSITY OF TECHNOLOGY

FACULTY OF ELECTRONICS, TELECOMMUNICATIONS AND INFORMATICS

DECLARATION regarding the diploma thesis titled: Optimization of location of controllers in Software Defined Networks aimed at reducing the latency of serving the demands

First name and surname of student: Jakub Grzybowski
Date and place of birth: 26.01.2000, Bydgoszcz
ID: 184670

Faculty: Faculty of Electronics,Telecommunications and Informatics
Field of study: Informatics

Cycle of studies: master's degree studies
Mode of study: Full-time studies

Type of the diploma thesis: master's thesis

1. Statement of authorship

Aware of criminal liability for violations of the Act of 4th February 1994 on Copyright and Related Rights (Journal of Laws 2022 item 2509 with later amendments) and disciplinary actions set out in the Act of 20th July 2018 on the Law on Higher Education and Science (Journal of Laws 2024, item 1571 with later amendments),[1] as well as civil liability, I hereby declare that the submitted diploma thesis is my own work.

This diploma thesis has never before been the basis of an official procedure associated with the awarding of a professional title.

All the information contained in the above diploma thesis which is derived from written and electronic sources is documented in a list of relevant literature in accordance with art. 34 of the Copyright and Related Rights Act.

2. Statement on the Use of GenAI Tools

I used tools with a potentially low degree of interference

I hereby declare that I have critically analyzed and verified the content generated using GenAI.

24.06.2025, Jakub Grzybowski

Date and signature of the student or authentication on the university portal Moja PG

**) The document was drawn up in the IT system, on the basis of paragraph 15 clause 3b of the Decree of the Ministry of Science and Higher Education of 12 May 2020, amending the decree concerning university studies (Journal of Laws of 2020, item 853). No signature or stamp required.*

[1] The Act of 20th July 2018 on the Law on Higher Education and Science:

Art. 312, section 3. Should a student be suspected of committing an act referred to in Art. 287 section 2 items 1–5, the rector shall forthwith order an enquiry.

Art. 312, section 4. If the evidence collected during an enquiry confirms that the act referred to in section 5 has been committed, the rector shall suspend the procedure for the awarding of a professional title pending a judgement of the disciplinary committee and submit formal notification on suspicion of committing a criminal offence.

STRESZCZENIE

Programowalne sieci komputerowe (ang. Software-Defined Networks, SDN) stanowią koncepcję, której głównym założeniem jest oddzielenie płaszczyzny sterowania od płaszczyzny przesyłania danych. Takie podejście umożliwia stworzenie warstwy abstrakcji, polegającej na przekierowaniu wszystkich sygnałów sterujących do wyodrębnionego elementu w sieci, podczas gdy przełączniki odpowiadają jedynie za przesyłanie pakietów zgodnie z określonymi regułami. Centralnym elementem zarządzania siecią w tej architekturze jest kontroler.

W przypadku sieci rozległych (np. sieci WAN czy szkieletowych) zachodzi potrzeba zastosowania wielu kontrolerów w topologii sieciowej w celu efektywnego zarządzania oraz uniknięcia przeciążenia pojedynczego punktu, którego awaria mogłaby spowodować paraliż całej infrastruktury.

Celem niniejszej pracy jest zbadanie zależności pomiędzy rozmieszczeniem wielu kontrolerów w sieci a opóźnieniami związanymi z obsługą żądań, rozumianymi jako czas wymiany wiadomości pomiędzy kontrolerami a przełącznikami. Przeprowadzone eksperymenty mają na celu optymalizację wartości tego opóźnienia. Analizie poddano zarówno średnie wartości opóźnienia (average delay), jak i opóźnienia w najgorszym przypadku (maximal delay).

W ramach pracy dokonano przeglądu literatury przedmiotu oraz wyselekcjonowano istniejące rozwiązania, które stanowią solidną podstawę do dalszych analiz. Kluczowym algorytmem wykorzystanym w badaniach jest algorytm Advanced K-Means, oparty na grupowaniu węzłów sieciowych w klastry oraz wyznaczaniu centroidów – punktów uznanych za optymalnych kandydatów na kontrolery, ze względu na minimalizację opóźnień w obsłudze żądań.

Słowa kluczowe: SDN, kontroler, opóźnienie, optymalizacja, heurestyki

Dziedzina nauki i techniki, zgodnie z wymogami OECD: 1.2 Informatyka (Computer and Information Sciences)

ABSTRACT

Software Defined Networks (SDN) represent a concept based on the separation of the control plane from the data plane. This approach enables the creation of an abstraction layer that redirects all control signals to a controller, while switches are responsible solely for forwarding packets according to predefined rules. The controller constitutes the central element of network management in this architecture.

In the context of wide-area networks (e.g., WANs or backbone networks), it becomes necessary to deploy multiple controllers within the network topology to ensure efficient management and to avoid overloading a single point of control, whose failure could paralyze the entire network.

The primary objective of this thesis is to investigate the relationship between the placement of multiple controllers in a network and the latency associated with handling requests, understood as the time required for message exchange between controllers and switches. The conducted experiments aim to optimize this latency, analyzing both the average delay and the worst-case delay.

As part of the study, a comprehensive literature review was performed, leading to the selection of existing solutions that provide a solid foundation for further analysis. The key algorithm utilized in the research is the Advanced K-Means algorithm, which is based on clustering network nodes and selecting centroids—points identified as optimal controller candidates due to their potential to minimize request handling latency.

Keywords: SDN, controller, delay, optimization, heurestics

TABLE OF CONTENTS

1 Introduction 8

1.1 Background on Software-Defined Networking 8

1.2 Problem Statement 9

1.3 Objectives and Scope 10

1.4 Thesis Structure 11

2 Controller Placement Problem 14

2.1 Theoretical Background 14

3 Taxonomy of Controller Placement Strategies 16

3.1 Optimal Approaches 16

3.2 Heuristic and Metaheuristic Approaches 17

3.3 Clustering-Based Approaches 17

3.4 Game-Theoretic Approaches 18

3.5 Multi-Objective and Hybrid Approaches 19

3.6 Reinforcement Learning Approaches 20

3.7 Comparison of Controller Placement Methods 22

3.8 Alignment of HDIDS and Advanced K-Means with Research Objectives 23

4 Problem Formulation 24

4.1 System Model and Overview 24

4.1.1 Network Topology 24

4.1.2 Clustering the Topology 25

4.1.3 Decision Variables 25

4.1.4 Constraints 26

4.1.5 Objective Function 27

4.1.6 Goal of the Optimization 27

4.2 On Request Handling Latency in CPP 27

4.2.1 Minimizing Propagation Latency 28

4.2.2 Evaluation of the Number of Controllers 28

5 Selected Approaches and Their Contributions 29

5.1 Independent Dominating Set Approach 29

5.2 K-Means Approach 30

5.3 High Degree with Independent Dominating Set (HDIDS) Method 32

5.3.1 Algorithmic Reasoning and Heuristic Integration 32

5.3.2 Properties and Limitations 33

5.3.3 Illustrative Example of HDIDS Execution . . . 33
5.3.4 Experimental Evaluation and Performance Results . . . 35
5.3.5 Factors Contributing to HDIDS Performance . . . 39
5.3.6 Potential Bottlenecks and Limitations of HDIDS . . . 40
5.3.7 Critical Assessment and Value of Degree-Based Approach . . . 41
5.4 Advanced K-Means Method . . . 42
5.4.1 Algorithmic Details and Operational Principles . . . 42
5.4.2 Pseudocode of the Advanced K-Means Algorithm . . . 43
5.4.3 Experimental Evaluation and Performance Results . . . 46
5.5 Discussion and Key Observations . . . 48
6 Limitations and Enhancements Strategy . . . 51
6.1 Identified Gaps in Advanced K-Means . . . 51
6.2 Assessment of Possible Improvements . . . 52
6.2.1 Strategies for Initial Center Selection . . . 52
6.2.2 Strategies for Selecting Subsequent Centers . . . 53
6.3 Direction of Enhancements . . . 56
7 Own Contributions . . . 57
7.1 Realized Enhancements . . . 57
7.1.1 Centroid Selection Weights . . . 57
7.1.2 Probabilistic Subsequent Center Selection . . . 58
7.2 Research Environment . . . 60
7.2.1 Topology Test Data . . . 60
7.2.2 Simulator . . . 66
7.2.3 Project Structure . . . 67
7.3 Experiments . . . 67
7.3.1 Results . . . 68
7.3.2 Comparative Analysis . . . 88
8 Assessment and Recommendations . . . 89
8.1 Evaluation of the Results and Key Findings . . . 89
8.2 Recommendations . . . 90
9 Conclusions . . . 92
Bibliography . . . 94
List of Figures . . . 97
List of Tables . . . 100
A SOURCE CODE . . . 102
B PLOTS . . . 103

C RESULTS OF THE EXPERIMENTS 104
D LOADS OF THE CONTROLLERS 105

1. INTRODUCTION

1.1. Background on Software-Defined Networking

Software-Defined Networking (SDN) constitutes a paradigm shift in the design and operation of packet-switched networks by decoupling the control plane—responsible for higher-level forwarding decisions and policy enforcement—from the data plane, which performs the actual packet forwarding [1]. In an SDN architecture (see Figure 1.1), the forwarding elements (switches) expose a simplified, flow-based interface to an external controller over a well-defined "southbound" protocol (most notably OpenFlow) [1], which maintains a global view of network state and enforces forwarding policies across all devices in the network. This separation enables network administrators to program network behavior centrally, rather than configuring each device individually, thereby greatly simplifying network management, accelerating the deployment of new services, and supporting dynamic reconfiguration at runtime.

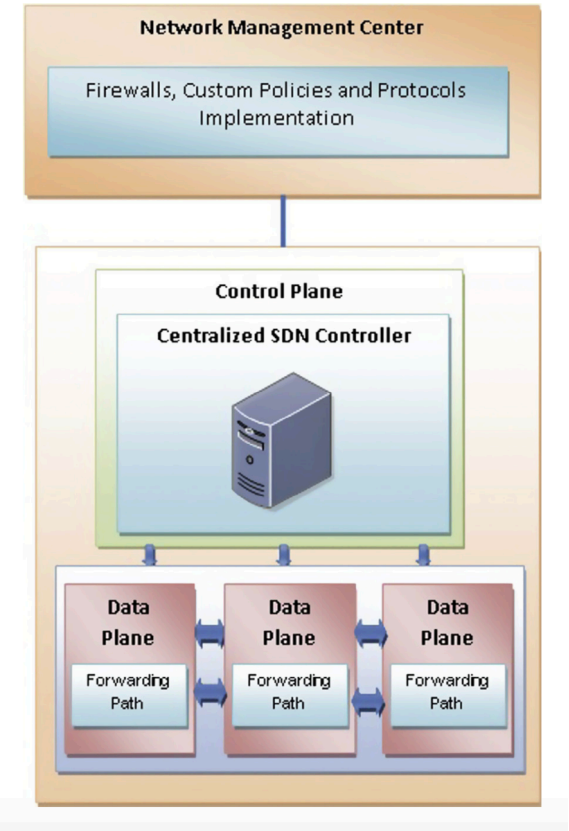


Fig. 1.1. SDN architecture

The principal performance advantage afforded by SDN arises from this global, logically centralized control layer. By abstracting the control logic into a single or clustered controller entity, SDN provides network-wide visibility into traffic flows, link utilization, and topological changes,

enabling fine-grained traffic engineering, efficient load balancing, and rapid failure recovery. In contrast to traditional distributed control architectures—where routing and switching decisions are made independently at each hop—SDN's centralized model can enforce end-to-end path selection that optimizes metrics such as latency, throughput, and fault tolerance. Moreover, SDN's programmability fosters the deployment of complex virtual network overlays and multi-tenant isolation without manual reconfiguration of the physical underlay, supporting features such as live virtual-machine migration and on-demand creation of virtual networks with tailored Quality-of-Service guarantees.

1.2. Problem Statement

The advantages of the SDN paradigm heavily depend on the performance of its centralized controller, which governs the efficiency and responsiveness of network operations. In small-scale deployments, a single controller suffices to manage the control plane, as its capacity—measured by the maximum load of flow requests it can process—remains underutilized. Furthermore, the limited geographical scope of such networks ensures that propagation latencies between switches and the controller remain negligible, avoiding performance degradation.

However, in large-scale or geographically distributed networks—such as wide-area networks (WANs) or hyper-scale data centers—the limitations of a single-controller architecture become pronounced. As the number of switches and traffic flows increases, the controller's capacity is rapidly exhausted, creating a critical bottleneck. For instance, studies indicate that a large data center with 2 million virtual machines may generate over 20 million flows per second, far exceeding the capacity of a single controller (e.g., NOX handles 30,000 requests/sec) [1]. This imbalance leads to severe flow-setup delays, packet loss, and degraded QoS. Additionally, the geographical dispersion of switches in WANs introduces significant propagation latencies. As highlighted in [1], long propagation delays in wide-area SDN deployments directly impact real-time controller responsiveness.

To address these challenges, the SDN paradigm evolves from a single-controller to a multi-controller architecture, in which multiple distributed controllers collaboratively manage different regions of the network. While this approach enhances scalability, resilience, and reduces the control-plane latency, it introduces a fundamental design question: how many controllers should be deployed, where should they be placed within the topology, and how should switches be assigned to them? This constitutes the essence of the Controller Placement Problem (CPP), a well-known combinatorial optimization problem that has become a key focus of research in SDN deployment strategies [2, 3].

The CPP aims to determine the optimal number and locations of controllers such that the overall performance of the SDN is improved according to defined metrics—primarily the latency of switch-to-controller communication, which directly influences flow-setup time and network responsiveness. Empirical studies show that improper controller placement may lead to up to 70% increase in propagation delays compared to optimal configurations [4]. However, latency is not the only factor that influences placement decisions. Other important criteria include controller load balancing,

inter-controller synchronization overhead, fault-tolerance requirements and network resilience against failures [2, 5, 6]. Therefore, the CPP is often treated as a multi-objective optimization problem, where trade-offs between these conflicting objectives must be carefully evaluated.

Thus, the critical research gap lies in developing a latency-optimized CPP framework that addresses scalability, reliability, and adaptability in large-scale SDN deployments. This thesis seeks to bridge this gap by analyzing existing placement methodologies, identifying their limitations in latency reduction, and proposing enhancements to improve demand-serving efficiency.

1.3. Objectives and Scope

The primary objective of this thesis is to investigate the CPP in SDNs with a specific focus on minimizing the request-processing latency, understood as the delay incurred in the communication between switches and their assigned controllers. This performance metric—commonly referred to as switch-to-controller latency—is a critical determinant of the overall responsiveness of an SDN and has profound implications for the Quality of Service (QoS) and scalability of the control plane.

This work pursues several complementary goals. First, a comprehensive literature review is conducted to identify and categorize existing CPP solution strategies, with particular emphasis on those tailored for latency minimization. This survey establishes the methodological foundation for selecting a suitable baseline algorithm that serves as the reference point for subsequent analyses and modifications.

Next, the selected baseline method is subject to an in-depth qualitative and quantitative evaluation. This includes the identification of potential limitations, structural inefficiencies, or heuristic shortcomings that may influence its effectiveness under varying network conditions. Based on this assessment, specific improvements or modifications are proposed and implemented, targeting the enhancement of latency performance without significantly compromising computational efficiency or scalability.

The improved variants are then subjected to a series of controlled simulation-based experiments. These experiments are designed not only to validate the correctness and performance gains of the proposed modifications, but also to analyze the sensitivity of the algorithm to key factors such as controller quantity, topology size and structure, and distribution of flow demand. Particular attention is given to identifying interdependencies between these factors and their joint impact on the latency metric.

Finally, the experimental results are systematically analyzed and discussed. The findings serve as the basis for drawing informed conclusions regarding the efficacy and generalizability of the proposed modifications. Additionally, the thesis outlines practical recommendations and future research directions that may further advance the state of the art in latency-optimized controller placement methodologies.

The study excludes non-latency-related aspects of CPP, such as security or energy efficiency, to maintain a focused analysis on demand-serving optimization. By addressing the above research questions, this work aims to provide actionable insights into latency reduction strategies, bridging the

gap between theoretical CPP formulations and practical SDN deployments.

To concretize the analytical direction of this thesis and ensure structured investigation, the following research questions have been formulated:

- **What controller placement strategies in SDNs have been shown to improve request-processing latency?**
- **To what extent does the controller placement strategy influence request-processing latency in SDNs?**
- **Is it possible to devise a controller placement strategy that always minimizes request-processing latency regardless of network conditions?**
- **Does the topology of an SDN network directly impact the effectiveness of a given controller placement method in minimizing request-processing latency?**

The first question motivates the need for a systematic literature review and is directly addressed in the initial stage of the thesis. Its objective is to provide a critical synthesis of the state of the art and identify the techniques by which latency optimization has been previously approached.

The second and third questions guide the core experimental component of the work. While question two seeks to verify whether the choice of controller placement strategy truly affects latency values and in what scenarios, question three probes the existence of placement schemes that consistently yield optimal performance. These questions collectively form the conceptual framework for evaluating the baseline method and its proposed improvements.

The fourth question relates to the adaptability and robustness of placement strategies across diverse network topologies. It seeks to determine whether a method optimized for one class of topologies retains its performance characteristics in networks with different structural properties. Addressing this question is key to understanding the generalizability of algorithmic solutions and informs the selection of representative test cases in the simulation phase.

These research questions underpin the entire methodology of the thesis and are referenced throughout the subsequent chapters to ensure the coherence and relevance of the conducted work. They provide the analytical lens through which all literature evaluation, algorithmic modifications, and experimental results are interpreted.

1.4. Thesis Structure

This thesis is structured to progressively explore the Controller Placement Problem (CPP) in Software-Defined Networks, with particular emphasis on optimizing request-processing latency. The work is developed in a linear fashion, beginning with a theoretical foundation, moving through the analysis of existing methods, and culminating in original contributions and their evaluation.

The opening chapter (see chapter 1) introduces the essential concepts of Software-Defined Networking, describing its architecture, the distinction between data and control planes, and the challenges posed by both single- and multi-controller designs. This background sets the stage for

understanding the rationale behind studying CPP. The chapter also presents the purpose of this research, outlines the research questions that guide the inquiry, and explains the methodological approach and expected outcomes.

Following the introduction, the thesis delves into the theoretical background of the Controller Placement Problem (see chapter 2) and the metrics used to evaluate controller deployment strategies. This includes a detailed description of how controller placement impacts network latency and other performance indicators. Various categories of placement methodologies are reviewed, starting with optimal approaches, such as Integer Linear Programming and brute-force algorithms, which guarantee the best possible results but suffer from poor scalability. In contrast, heuristic and suboptimal techniques, including greedy algorithms, clustering methods, and game-theoretic models, are examined for their ability to deliver near-optimal solutions with reduced computational complexity. Other less conventional approaches are also briefly discussed to provide a broader context.

The next part of the thesis surveys selected existing works that offer concrete implementations of placement strategies (see chapter 3). A comprehensive taxonomy of CPP methodologies is developed, detailing the defining characteristics of each approach and categorizing them according to their underlying principles and mechanisms. Relevant literature is reviewed and cited to provide context and examples of how these methods have been implemented in practice. Performance comparisons are carried out to evaluate the relative efficiency of the surveyed methods. Finally, all identified categories are summarized, and both the strengths and weaknesses of each approach are critically assessed, providing a clear overview of the current state of the art in SDN controller placement.

Then, foundational concepts regarding problem discussed in this thesis are formalized (see chapter 4). This includes formal model: the SDN is represented as a graph structure, with nodes corresponding to switches and potential controller locations, and edges representing communication links. Constraints and assumptions underlying the optimization task—such as controller capacity, switch assignments, and communication limits—are clearly specified to allow for reproducibility and analytical rigor. Additionally, definitions of average-case and worst-case request-processing latency are established, and the mathematical expressions used to quantify these phenomena in controller-switch interactions are derived. This section provides the tools necessary to interpret experimental outcomes later in the thesis.

After presenting the underlying system model and formally defining the controller placement problem, this chapter (see 5) identifies two theoretical strategies—Independent Dominating Set (IDS) and K-Means clustering—as promising frameworks for further analysis. Both approaches are considered in light of their conceptual foundations, algorithmic design, and reported effectiveness in similar optimization tasks. Building on this theoretical groundwork, the subsequent sections describe the concrete implementation of CPP solutions based on these strategies: the High Degree with Independent Dominating Set method and the Advanced K-Means algorithm. Each method is examined in terms of its internal logic, computational steps, and suitability for the target SDN environment. The chapter

details the simulation environment and experimental setup, ensuring clarity and reproducibility of the results. Finally, a comparative evaluation is conducted, critically analyzing the performance and trade-offs associated with each approach and noting key observations.

A dedicated chapter (see 6) is devoted to acknowledging the limitations observed in the analyzed Advanced K-Means algorithm and to outlining potential improvement strategies. Drawing on the results of the comparative analysis, specific shortcomings—such as sensitivity to initialization, or challenges in adapting to heterogeneous network conditions—are systematically discussed. Based on these findings, the thesis proposes targeted directions for algorithmic enhancement, providing a clear rationale for the subsequent modifications and experiments.

Building on this evaluation, the thesis then presents the author's original contribution: targeted modifications to improve the performance of the selected baseline method (see chapter 7). The reasoning behind each modification is explained, along with the theoretical and technical details. Experimental scenarios are designed to test the impact of the enhancements under different topological conditions and the results are analyzed both quantitatively and qualitatively.

Following the experimentation phase, the observed improvements are assessed in light of the research questions (see chapter 8). The analysis considers not only the effectiveness of the enhancements in minimizing latency but also their consistency across various topologies and deployment scenarios. Particular attention is paid to determining whether a given strategy can deliver robust performance and how topological variation influences its reliability. Building on these insights, the chapter is concluded by offering practical recommendations for the design and selection of controller placement methods. The evaluation covers how the proposed method performs under different conditions, highlights its key characteristics, and identifies which strategies or placement methods are best suited for particular network scenarios or specific stages of the controller placement process.

Finally, the thesis summarizes (see chapter 9) the key outcomes of the research. The main achievements include the identification and critical assessment of promising controller placement strategies, the development and enhancement of selected algorithms, and the thorough experimental evaluation of their performance in diverse SDN scenarios. The conclusions drawn highlight not only the potential and challenges of the examined approaches, but also propose concrete directions for future work aimed at further optimizing latency and adaptability in Software-Defined Networks.

2. CONTROLLER PLACEMENT PROBLEM

The efficiency of Software-Defined Networks (SDNs) critically depends on the design of the control plane, which governs the operation and reconfiguration of the underlying data plane. In conventional SDN architectures, the logically centralized controller is responsible for processing flow requests, maintaining network state, enforcing policies, and reacting to topology changes. As the network scales in size and complexity, relying on a single controller becomes infeasible due to constraints related to performance, fault tolerance, and responsiveness. Consequently, modern SDNs adopt distributed or hierarchical controller deployments, which introduce the need for strategic controller placement.

2.1. Theoretical Background

The *Controller Placement Problem* (CPP) emerges as a fundamental design challenge in such distributed control architectures. Formally, the problem involves determining the optimal number and locations of controllers within the network, as well as the assignment of switches to controllers, in order to meet performance and reliability requirements. Since each controller-switch interaction incurs a communication delay, which directly impacts the flow-setup time, the placement configuration plays a pivotal role in the quality of service experienced by end-users.

The origins of CPP can be traced to the foundational work of Heller et al. [3], who investigated the effects of controller location on propagation delays in wide-area SDNs. Their study revealed that improper controller placement could significantly degrade performance, while optimized placement strategies could reduce average latencies by up to 50%. Since then, CPP has attracted considerable research interest, evolving into a multifaceted optimization problem with numerous variants and objective functions.

The CPP is widely recognized as NP-hard [7], which implies that exact solutions become computationally intractable for large-scale topologies. This has led to the development of a wide spectrum of solution approaches, ranging from exact optimization via Integer Linear Programming (ILP), through heuristic and metaheuristic methods, to game-theoretic and clustering-based strategies. Each of these methodologies offers a different balance between optimality, scalability, and implementation complexity.

Multiple performance metrics have been introduced in the literature to guide the evaluation of placement strategies. The most prominent include average and worst-case latency between switches and controllers [7, 8], controller load balancing [6], inter-controller synchronization overhead [4], and network resilience under failure scenarios [9]. More recent contributions also consider hybrid objectives, such as reliability-aware or energy-efficient placement schemes, addressing the diverse needs of emerging SDN applications.

Moreover, the performance of a given placement method is often topology-dependent. Different network structures—such as mesh, ring, tree, or Internet2-like graphs—introduce varying levels of complexity and bottlenecks, which affect the efficiency of controller allocation algorithms. Consequently, the generalizability of a method across multiple topologies is an important consideration in comparative evaluations.

Minimizing controller-to-switch latency has become increasingly central within modern CPP research, as it directly influences request handling time—one of the key determinants of SDN control plane responsiveness. Since every unmatched packet at a switch must traverse the control channel to the assigned controller for decision-making, reducing this latency translates into faster flow rule installation and better support for real-time applications. Accordingly, many recent works prioritize propagation and response delay as dominant objectives [4] [7] [8]. In the next chapter, we will examine selected approaches in detail and evaluate their potential to fulfill the latency minimization mission defined in this thesis.

3. TAXONOMY OF CONTROLLER PLACEMENT STRATEGIES

In response to the growing complexity and size of Software-Defined Networks, numerous strategies have been proposed to tackle the Controller Placement Problem. This chapter introduces a structured taxonomy of approaches, presenting representative methods from each category, their operational principles, and an evaluation of their strengths and weaknesses. The classification includes optimal methods, heuristic and metaheuristic strategies, clustering-based techniques, game-theoretic models, hybrid multi-objective optimization frameworks and reinforcement learning approaches.

3.1. Optimal Approaches

Optimal approaches to the CPP aim to provide exact solutions by formulating the problem as a mathematical optimization task, frequently leveraging models such as Integer Linear Programming (ILP), Mixed Integer Linear Programming (MILP), and Quadratic Programming (QP). These formulations strive to minimize performance-related metrics, such as average and worst-case latency, while conforming to topological and resource constraints. Due to their computational complexity, these methods are primarily applicable to small- or medium-sized network topologies, serving both as baselines for performance comparisons and as reference models for hybrid or heuristic strategies.

Heller et al. [3] pioneered the formalization of CPP as an optimization problem using ILP. Their model introduced three canonical variants: minimizing the worst-case latency (modeled as a k-center problem), minimizing the average latency (as a k-median problem), and constraining the maximum switch-controller latency (akin to a set cover problem). This foundational work demonstrated that controller placement has substantial impact on the delay characteristics of the network, and highlighted the NP-hard nature of the problem.

Killi et al. [10] extended Heller's model by incorporating controller failure scenarios, proposing the CPWP (Controller Placement With Planning) model. Their MILP formulation considers both regular operation and failover cases, ensuring that each switch is assigned to a primary and a backup controller. Although the MILP provides optimal solutions, it suffers from exponential runtime growth as the network size increases, making it impractical for large-scale topologies.

In the context of the present work, which focuses on reducing the latency of request handling in Software-Defined Networks, optimal approaches offer valuable theoretical baselines. Their ability to precisely minimize average or worst-case latencies aligns well with our performance objectives. However, the computational intractability of solving such models in real-time or for larger topologies makes them unsuitable for direct implementation in practical SDN scenarios. Although they guarantee the highest solution quality for the CPP, their reliance on exhaustive search renders them ineffective for networks beyond 50–60 nodes. Instead, they serve to define ideal performance targets, guiding the development and validation of well-scalable methods such as heuristic and hybrid algorithms.

3.2. Heuristic and Metaheuristic Approaches

To overcome the computational limitations of exact optimization models, heuristic methods aim to find approximate solutions using fast and scalable procedures that are often topology-agnostic. Classical heuristics are typically inspired by facility location theory, including greedy algorithms based on latency minimization, clustering, or capacity-constrained assignments. For instance, Qi et al. [7] proposed a density-based clustering strategy where switches are grouped around high-centrality nodes using closeness and degree metrics, leading to notable latency improvements over traditional k-means.

To improve the quality of placement in large-scale or complex SDNs, metaheuristic approaches leverage global search strategies driven by stochastic sampling. Algorithms such as Simulated Annealing, Genetic Algorithms (GA), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO) have been widely explored in the CPP context.

Recent contributions have extended the application of metaheuristics to consider realistic constraints such as controller failure resilience and control plane overhead. Singh et al. [9] employed a PSO-based framework with capacitated controller placement, ensuring that switch-to-controller assignments adhere to both reliability and latency thresholds. Their simulations on the Internet2 OS3E topology show that three well-placed controllers can sustain network operations even under n–1 controller failure conditions.

Despite their practical advantages, metaheuristic methods remain sensitive to parameter configurations, often requiring multiple executions to stabilize results. They lack guarantees of optimality and may exhibit inconsistent performance across different network topologies. Nonetheless, their scalability and adaptability make them essential tools in SDN controller placement, especially when dealing with large-scale or dynamically changing environments.

3.3. Clustering-Based Approaches

Clustering-based methods solve the CPP by partitioning the network into multiple local domains, where each cluster is governed by a separate controller. This is typically done using spatial, topological, or metric-based criteria to ensure proximity between switches and their assigned controller. The rationale is to reduce intra-cluster latency and simplify the assignment logic by minimizing the average distance from each switch to its controller.

One of the earliest and widely referenced clustering techniques is the K-Means and its optimized variants. These methods partition the network graph based on the geographic or topological proximity of nodes and select cluster centroids as controller locations. Although straightforward, standard K-Means struggles with topological irregularities and unbalanced clusters. To address these limitations, advanced adaptations have emerged. The Advanced K-Means algorithm [4] introduces heuristics for handling skewed node distributions and optimizing centroid positions iteratively to better reflect delay-sensitive objectives. Empirical evaluations have demonstrated that Advanced K-Means achieves lower average switch-to-controller delays than conventional K-Means and offers robust performance across

diverse topologies, making it a compelling candidate for further development and integration.

Several recent works have advanced partitioning strategies for controller placement that, while inspired by clustering principles, often blend classical clustering with graph-theoretic heuristics. Rather than forming clusters purely through iterative assignment as in K-Means, these approaches define controller domains by exploiting structural properties such as node degree, spatial density, or dominating sets. For example, the method presented in [11] employs a hybrid technique, initially prioritizing nodes with high degree centrality and subsequently applying Independent Dominating Set constraints. This ensures that controller domains are both well-distributed and mutually independent, thereby reducing latency and avoiding unnecessary overlap. Likewise, the Density-Based Controller Placement Algorithm (DCPA) [12] leverages local spatial density to identify candidate centers, allocating controllers at nodes that exhibit both high connectivity and low aggregate propagation cost to their respective domains.

While these methods do not always conform strictly to classical clustering algorithms, they embody the core principle of dividing the network into coherent subregions, each optimized for local latency and control efficiency. Such partitioning techniques have demonstrated particular effectiveness in topologies with non-uniform node distributions, where leveraging graph-theoretic insights allows for more adaptive and performance-oriented controller placements than purely distance-based clustering.

Spectral clustering techniques, as discussed in [13], further enhance the partitioning process by leveraging the eigenstructure of the network's Laplacian matrix to uncover natural clusters in complex graphs. This approach enables better handling of topological irregularities and has shown improved latency performance in experimental scenarios compared to conventional geometric clustering.

Another noteworthy method is the Modified Density Peaks Clustering (MDPC) approach introduced in [7], which selects cluster centers based on local density and distance metrics. This method is particularly adept at identifying optimal controller positions in irregular and large-scale SDN environments. Experimental results indicate that MDPC achieves a 35% improvement in average latency compared to K-Means and a 10% gain over Optimized K-Means, confirming its suitability for latency-critical deployments.

Overall, clustering-based strategies strike a balance between computational tractability and solution quality. Their ability to adapt to varied network structures and effectively reduce controller-switch latency makes them not only a practical choice for real-world SDN deployments but also a promising foundation for enhancing existing methods. In particular, techniques like Advanced K-Means provide a modular framework that can be extended with additional heuristics or integrated with hybrid strategies, thereby forming a solid basis for the main optimization framework proposed in this thesis.

3.4. Game-Theoretic Approaches

Game-theoretic models incorporate autonomous decision-making agents and have proven particularly effective in multi-controller environments with heterogeneous stakeholders. These models frame the controller placement problem (CPP) as a strategic interaction between rational players—

controllers or network domains—each seeking to optimize its own utility function under latency, cost, or load-related constraints. Non-cooperative approaches such as congestion games model selfish behavior of controllers, where each agent independently selects its optimal placement to minimize response time or communication cost. For instance, Venkatesan et al. [14] formulated the CPP as a non-zero sum game, showing that even under decentralized decision-making, stable Nash equilibria can be reached with minimal inter-controller interference and acceptable latency trade-offs. Their model supports resilience by allowing controllers to re-evaluate placement decisions dynamically in response to changes in network state.

In contrast, cooperative game-theoretic approaches assume willingness of agents to coordinate for mutual benefit. Ksentini et al. [15] employed the Nash Bargaining Solution to derive controller placements that minimize overall network delay while ensuring fairness across domains. Their method reduces inter-domain latency by fostering equitable load distribution and stable coalitions between administrative units. A recent study by Xu et al. [16] extends this idea through a collaborative competitive game that introduces bounded selfishness via a pricing mechanism. Their model effectively combines the robustness of non-cooperative strategy spaces with the performance gains of coalition-based bargaining, achieving significant latency reductions in heterogeneous multi-domain topologies. These approaches highlight the flexibility of game-theoretic formulations, which can model diverse SDN deployment scenarios, from vendor-neutral inter-domain networks to hierarchically managed service provider infrastructures.

These methods are inherently decentralized and adaptive, making them well-suited for dynamic and federated SDNs. Their major drawbacks include difficulty in designing payoff functions, challenges in convergence, and sensitivity to agent behavior modeling.

3.5. Multi-Objective and Hybrid Approaches

Multi-objective and hybrid optimization approaches aim to address the Controller Placement Problem by simultaneously optimizing multiple performance metrics, such as propagation latency, controller load balancing, fault tolerance, and throughput. These methods incorporate diverse algorithmic strategies, often combining heuristics with metaheuristics or leveraging advanced population-based optimization schemes.

The Pareto Optimal Controller Placement (POCO) model exemplifies an early approach to multi-objective controller deployment, where trade-offs between competing objectives are captured using Pareto front exploration [5]. POCO generates a set of non-dominated controller placements from which network operators can choose, depending on their performance priorities. This methodology highlights the essential nature of objective balancing in complex and large-scale SDN topologies.

The MODECP (Multi-Objective based Distributed Efficient Controller Placement) framework [6] extends this concept by explicitly incorporating delay, reliability, and load balance into a weighted composite cost function. It utilizes a two-stage optimization procedure: an initial delay-centric

selection of controller locations followed by a reliability-aware reassignment of switch-to-controller mappings. Simulations on realistic topologies demonstrate MODECP's superior adaptability under network dynamics, achieving reduced average response time and enhanced fault tolerance.

Firouz et al. [8] introduced a hybrid multi-objective Marine Predator Algorithm enhanced with NSGA-II crossover and mutation operators. Their approach, termed MOMPA (Multi-Objective Marine Predator Algorithm), tackles CPP by optimizing simultaneously for latency, controller load balance, and inter-controller reliability. The model is capable of discovering a diverse Pareto-optimal solution set, enabling flexibility in controller deployment decisions depending on application requirements. The integration of NSGA-II mechanics allows the algorithm to maintain solution diversity and avoid premature convergence. Experimental results highlight MOMPA's effectiveness in achieving substantial reductions in maximum and average latency, while also ensuring more evenly distributed load and improved fault resilience compared to standard evolutionary and heuristic baselines.

Recent advances in hybrid evolutionary strategies have further enhanced the multi-objective landscape. Hemagowri and Selvan [17] present the HEA-OCP (Hybrid Evolutionary Algorithm for Optimized Controller Placement), which combines artificial fish swarm optimization with Gaussian chaotic maps and opposition-based learning. The model optimizes across five criteria: propagation latency, load balancing, bandwidth, fault tolerance, and data transmission rate. Through comparative simulations against SA-FFCCPP and GSOCCPP baselines, HEA-OCP demonstrates superior packet delivery ratio, reduced jitter, lower average latency, and enhanced throughput.

These multi-objective and hybrid frameworks underscore the necessity of holistic optimization in controller placement. Rather than targeting a single metric, they recognize the multi-faceted nature of SDN performance and provide mechanisms to accommodate trade-offs, making them well-suited for real-world, heterogeneous network scenarios.

3.6. Reinforcement Learning Approaches

Recent advances in Artificial Intelligence, particularly in reinforcement learning (RL), have opened promising avenues for solving the CPP. Unlike traditional static heuristics or clustering methods, RL-based solutions are inherently adaptive and capable of learning optimal controller placements and switch-controller mappings based on environmental feedback, which is particularly advantageous under fluctuating traffic conditions.

A notable contribution in this domain is the D4CPP framework proposed by Wu et al. [18], which utilizes Deep Q-Networks (DQN) to dynamically adjust both the controller deployment and the switch-to-controller assignments. D4CPP formulates CPP as a multi-objective optimization problem aiming to minimize both average response latency and controller load imbalance. The system leverages a global DQN agent trained with flow fluctuation data to determine optimal actions, defined as placement and mapping strategies. Simulation results over realistic network topologies demonstrate that D4CPP outperforms classical methods like K-Means, achieving up to 13% reduction in latency

and 50% improvement in load balance.

However, DQN-based models suffer from overestimation bias, potentially limiting their decision quality. To address this, Aboelela and Sadek [19] introduce a Prioritized Double Deep Q-Network (PDDQN), which combines Double DQN and prioritized experience replay to enhance learning stability and convergence. Their method includes pre-clustering of switches and uses the Elbow method to select the optimal number of controllers. The placement decision considers latency and load metrics jointly in the reward function, promoting both low communication delay and equitable load distribution. The evaluation conducted on Abilene and Germany50 topologies confirms that PDDQN outperforms standard DQN with up to 52% lower latency and 22% better load balancing.

These results underline that reinforcement learning—especially with advanced DQN variants—provides a scalable and effective framework for CPP, particularly in non-stationary environments with high dynamism in traffic patterns.

3.7. Comparison of Controller Placement Methods

Table 3.1: Comparison of Controller Placement Problem (CPP) solution categories in SDN

Method Category	**Sources**	**Description**	**Strengths**	**Weaknesses**
Optimal Approaches	[3], [10]	ILP/MILP/QP models that provide exact solutions to the CPP, aiming to minimize latency.	Highest solution quality, ideal as a performance benchmark.	Very high computational cost, impractical for large-scale networks.
Heuristic and Metaheuristic Methods	[7], [9]	Approximate algorithms (e.g., PSO, GA, ACO) designed for fast and scalable controller placement.	Scalable, fast, effective in dynamic environments.	No optimality guarantee, sensitive to parameter settings.
Clustering-Based Approaches	[4], [7], [11], [12], [13]	Divide the network into local clusters managed by controllers (e.g., K-Means, MDPC, spectral methods).	Effectively reduce latency, flexible, suitable for large-scale topologies.	Struggle with irregular topologies, risk of unbalanced clusters.
Game-Theoretic Approaches	[14], [15],[16]	Decentralized models with autonomous agents using non-cooperative or cooperative strategies.	Well-suited for inter-domain and dynamic scenarios, support resilience.	Complex utility modeling, potential convergence issues.
Multi-Objective and Hybrid Methods	[5], [6], [8], [17]	Simultaneously optimize multiple criteria (latency, load, reliability), e.g., POCO, MODECP, MOMPA, HEA-OCP.	Address complex network requirements, achieve balanced performance.	Increased model complexity, possibly longer execution time.
Reinforcement Learning Approaches	[18], [19]	Adaptive models (e.g., DQN, PDDQN) that learn optimal controller placement based on feedback.	Dynamic adaptation, strong results under changing traffic.	Training complexity, overestimation bias in standard DQN.

3.8. Alignment of HDIDS and Advanced K-Means with Research Objectives

The central objective of this thesis is to optimize controller localization in SDN networks in order to minimize request handling latency, which could be achieved by reducing the propagation delay between switches and their assigned controllers. In this context, two approaches have emerged as especially well-aligned with the research goals: the High Degree with Independent Dominating Set (HDIDS) method [11] and the Advanced K-Means algorithm proposed by Zobary [4].

The HDIDS strategy embodies key principles directly supporting latency minimization. By leveraging node degree centrality to prioritize controller candidates and subsequently enforcing independence through the Independent Dominating Set, HDIDS efficiently distributes controllers across the network while targeting both average and worst-case response times. Its ability to dynamically determine the required number of controllers for a given latency threshold, and its proven advantage in sparse or hierarchical topologies, position HDIDS as a compelling approach for achieving low switch-to-controller latencies. The underlying heuristics of HDIDS—focused on maximizing reach and minimizing propagation delay—resonate strongly with the mission of this thesis.

Advanced K-Means, on the other hand, enhances the latency-oriented perspective by applying a clustering framework that partitions the network according to latency-aware distance metrics and selects controller locations based on centrality considerations. This algorithm not only outperforms standard K-Means in reducing both average and maximal propagation delays, but also offers a natural extension point for incorporating further optimization objectives, such as dynamic load balancing or resilience. Its modularity and modifiability make Advanced K-Means particularly suitable for research aimed at advancing latency minimization techniques.

Together, both HDIDS and Advanced K-Means contribute essential concepts and practical mechanisms toward the primary research objective: reducing SDN request handling latency. HDIDS introduces fundamental heuristics for effective latency-aware placement, while Advanced K-Means broadens the methodological toolkit, offering enhanced clustering and extensibility for future developments. The potential of these approaches underscores their relevance as both a theoretical and practical foundation for controller placement strategies focused on minimizing request handling delay.

4. PROBLEM FORMULATION

This chapter presents the formal system model that forms the foundation for both core approaches investigated in this thesis: the High Degree with Independent Dominating Set (HDIDS) method and the Advanced K-Means method. While both approaches share the same underlying network representation, their subsequent mathematical formulations diverge, particularly regarding decision variables and constraints. The Advanced K-Means approach introduces explicit decision variables and an optimization framework, while HDIDS applies a graph-theoretic heuristic, where selection of controller locations is performed algorithmically based on node degrees without formulating explicit optimization variables. This unified modeling framework enables direct comparison and consistent simulation, while further sections will introduce method-specific mathematical details—especially for Advanced K-Means, which is the primary subject of detailed experimental evaluation. The HDIDS method is utilized mainly as a reference and as a basis for developing and validating simulation tools for latency-aware Controller Placement Problems.

4.1. System Model and Overview

We represent the SDN network as an undirected graph $G = (V, E)$, which forms the structural backbone for both methods. This abstraction allows for flexible definition of topologies, explicit control over node and link characteristics, and rigorous mathematical treatment of controller placement decisions.

4.1.1. Network Topology

Let $G = (V, E)$ denote the undirected graph modeling the SDN:

- $V = \{v_1, v_2, \ldots, v_N\}$ is the set of N nodes, where each node represents a switch that may also be considered as a candidate location for controller deployment.
- E is the set of bidirectional links, where each edge $(v_i, v_j) \in E$ signifies a direct physical connection between nodes v_i and v_j.

To mathematically encode node network connectivity, we introduce the binary connectivity indicator α_{ij} for each pair of nodes $v_i, v_j \in V$:

$$\alpha_{ij} = \begin{cases} 1, & \text{if } (v_i, v_j) \in E, \\ 0, & \text{otherwise.} \end{cases} \tag{4.1}$$

This indicator α_{ij} forms the adjacency matrix of the network, providing a precise representation of which node pairs are directly connected.

The communication latency between any two nodes v_i and v_j is denoted as $d_{ij} \in \mathbb{R}_{\geq 0}$, representing the shortest path distance between v_i and v_j within the topology. The value of d_{ij} may reflect the number of hops, physical distance, propagation delay, or a combination of transmission

characteristics depending on the level of modeling detail. For each pair, d_{ij} is computed as the minimum sum of weights along any path connecting v_i to v_j, ensuring that the model accounts for the most efficient communication route:

$$d_{ij} = \min_{\text{paths}(v_i, v_j)} \sum_{(u,w) \in \text{path}} w_{uw} \tag{4.2}$$

where w_{uw} is the weight (delay) associated with each edge (u, w) on the path.

In the Advanced K-Means approach, the controller placement and switch-to-controller assignment are explicitly modeled by means of binary decision variables, and the entire optimization problem is formulated with precise constraints and objective functions. In contrast, the HDIDS approach does not introduce decision variables in the mathematical sense; instead, it algorithmically constructs a set of controller locations by prioritizing nodes with the highest degree, subject to independence and domination properties, and assigns each switch to its nearest selected controller. Thus, while both methods operate on the same underlying model, their mathematical treatment and the role of variables in the optimization process are fundamentally different.

4.1.2. Clustering the Topology

We denote K as the number of clusters or subnetworks in the topology. Accordingly, K controllers are to be placed in the network, and we define the set of controllers as $C = \{C_1, C_2, \ldots, C_K\}$, where each C_j corresponds to a distinct controller deployed at a selected node.

In the Advanced K-Means approach, the controller placement and switch assignment problem is formulated as a clustering task, where the goal is to partition the network into K clusters, each managed by a single controller. The following mathematical definitions and constraints refer specifically to this method.

4.1.3. Decision Variables

To precisely capture both controller deployment and switch-to-controller assignment, we introduce two sets of binary decision variables:

- y_{ij}, which indicates whether controller C_j is placed at node v_i:

$$y_{ij} = \begin{cases} 1, & \text{if controller } C_j \text{ is physically placed at node } v_i, \\ 0, & \text{otherwise.} \end{cases} \tag{4.3}$$

- x_{ij}, which indicates whether switch v_i is assigned to controller C_j:

$$x_{ij} = \begin{cases} 1, & \text{if switch } v_i \text{ is assigned to controller } C_j, \\ 0, & \text{otherwise.} \end{cases} \tag{4.4}$$

These variables enable the mathematical model to represent both the physical location of each controller and the logical assignment of switches to their respective controllers, forming the basis for subsequent constraint and objective function definitions.

4.1.4. Constraints

The placement and assignment configuration must satisfy the following set of constraints:

- **Exclusive assignment of each switch:** Each node v_i must be assigned to exactly one controller:

$$\sum_{j=1}^{K} x_{ij} = 1 \quad \forall v_i \in V \tag{4.5}$$

The variable x_{ij} determines the controller-to-switch mapping, and the summation over j ensures a unique assignment for every v_i.

- **Controller exclusivity per node:** At most one controller can be deployed at any single node:

$$\sum_{j=1}^{K} y_{ij} \leq 1 \quad \forall v_i \in V \tag{4.6}$$

For each node v_i, the sum of y_{ij} over all j must not exceed 1, ensuring that at most one controller is deployed per node.

- **Fixed number of controllers:** Exactly K controllers must be deployed across the network:

$$\sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} = K \tag{4.7}$$

The total number of deployed controllers in the network must equal K. This is enforced by summing all deployment indicators y_{ij} across the entire graph.

- **Switches can only be assigned to deployed controllers:** A switch may only be assigned to a controller if that controller has been deployed:

$$x_{ij} \leq y_{ij} \quad \forall i \in \{1, \ldots, N\},\ j \in \{1, \ldots, K\} \tag{4.8}$$

A node v_i can only be assigned to controller C_j (i.e., $x_{ij} = 1$) if that controller is actually deployed at some node (i.e., $y_{ij} = 1$). This ensures consistency between assignment and deployment.

- **Node eligibility based on connectivity:** To promote robust deployment, only nodes with sufficient degree are eligible to host controllers. Let D_i denote the degree of node v_i. The average node degree across the network is:

$$D_i \geq \rho \quad \text{for controller-eligible nodes } v_i \tag{4.9}$$

Here, D_i is the degree of node v_i, and ρ is the average degree of the network defined as:

$$\rho = \text{round}\left(\frac{1}{N} \sum_{i=1}^{N} D_i\right) \tag{4.10}$$

This constraint restricts controller deployment to nodes that are at least as well connected as the network average, favoring topologically central placements.

4.1.5. Objective Function

The optimization objective is to minimize the average propagation latency between all switches and their assigned controllers. Let d_{ij} be the latency between node v_i and controller C_j, and let $x_{ij} = 1$ if v_i is assigned to C_j. Then:

1. **The propagation latency for each cluster** is defined as follows:

$$L(C_j) = \sum_{i=1}^{N} d_{ij} \cdot x_{ij}. \tag{4.11}$$

2. **The average latency across the entire network** is defined as:

$$L_{\text{avg}}(K) = \frac{1}{N-K} \sum_{j=1}^{K} L(C_j). \tag{4.12}$$

The denominator $(N - K)$ accounts for the fact that K nodes serve as controllers and are not managed by other controllers themselves.

4.1.6. Goal of the Optimization

The final goal of the optimization is therefore:

$$\min_{x_{ij}, y_{ij}} \; L_{\text{avg}}(K). \tag{4.13}$$

4.2. On Request Handling Latency in CPP

When transitioning to a distributed controller architecture in SDN, effective communication between controllers and their assigned switches becomes essential. This communication, realized through well-defined interfaces, is critical for establishing forwarding rules, configuring security policies, and gathering information about network changes, such as path recomputation following node failures or topology updates. Each signal exchanged between a controller and its switches introduces a certain amount of latency. In large-scale networks, a high volume of requests can overwhelm a controller, leading to degraded network performance—manifesting as poor QoS metrics or even disrupted connections. Therefore, it is important to ensure that each controller operates within its processing capacity and handles a manageable volume of requests.

However, in extensive topologies, it is not always possible to position controllers so that every switch enjoys ideal conditions with minimal latency and balanced load. In such cases, additional controllers may need to be deployed to alleviate the burden on overloaded controllers and to maintain acceptable performance levels. Mamushiane et al. [20] pose a fundamental question: *Given an SDN topology, how many controllers are needed and where should they be placed?* In this study, we revisit this question with a focus on achieving efficient request handling and minimizing delays. It is necessary to consider not only communication latency (i.e., propagation delays) but also the maximum load that each controller can sustain. Simultaneously optimizing controller placement with respect to both latency

and load balancing is therefore expected to significantly improve overall request handling performance in SDN networks.

4.2.1. Minimizing Propagation Latency

By minimizing the objectives formulated by Zobary [4], the propagation delay in controller-switch communication can be significantly reduced, which directly contributes to faster flow rule installation and better control plane responsiveness in SDN. In these architectures, every time a data-plane switch encounters a packet for which it lacks a matching flow entry, a control message must be generated and sent to its assigned controller. The controller then performs the necessary policy and forwarding logic computations, and responds with appropriate instructions, typically via flow-modification messages. This process — known as request handling — is critically latency-sensitive, as the end-to-end reaction time is governed by the round-trip delay between the switch and the controller.

By minimizing propagation latency in the controller placement process, the overall delay experienced during these request handling cycles is significantly reduced. This has two important implications. First, it lowers the time required for packet forwarding decisions to be enacted, which is vital for short-lived flows and latency-critical applications such as voice, video, or real-time telemetry. Second, it enables the controller to respond more rapidly to bursts of control requests from multiple switches, thereby reducing queuing delays and mitigating the risk of control plane bottlenecks.

Furthermore, a lower propagation delay improves the effectiveness of adaptive traffic engineering strategies, as more timely feedback from the network allows the controller to update routes or policies with minimal lag. This enhances the reactivity and agility of the SDN architecture. Consequently, optimizing controller-to-switch latency not only meets local performance goals but contributes systemically to global SDN control efficiency.

4.2.2. Evaluation of the Number of Controllers

Determining the optimal number of controllers (K) remains a central and challenging aspect of the controller placement problem in SDN networks. As highlighted by Zobary [4], there is no universal answer to the question of how many controllers are needed, since the optimal value of K depends on multiple network-specific objectives and trade-offs, including propagation latency, load distribution, and deployment cost. While increasing the number of controllers generally reduces average propagation latency and enhances network responsiveness, it also introduces additional complexity and cost.

5. SELECTED APPROACHES AND THEIR CONTRIBUTIONS

Having surveyed a wide range of strategies to address it and introducing general model and problem to solve, this chapter narrows the focus to two selected methods—High Degree with Independent Dominating Set (HDIDS) and Advanced K-Means. These approaches have been chosen due to their promising characteristics with regard to minimizing request handling latency, which constitutes the principal performance criterion of this thesis.

The structure of the chapter reflects the analytical pathway required to understand the internal mechanisms of both methods. As HDIDS is fundamentally grounded in the idea of Independent Dominating Set (IDS), we first introduce the theoretical basis for IDS and explain how it enables effective and latency-aware controller distribution. Next, we present the underlying concepts of the classical K-Means clustering algorithm, which serves as the foundation for the enhancements introduced in the Advanced K-Means variant.

By examining these core principles before analyzing the full implementations of HDIDS and Advanced K-Means, we lay the groundwork for a deeper understanding of their placement logic, performance characteristics, and suitability for SDN environments where low-latency responsiveness to flow setup requests is essential.

5.1. Independent Dominating Set Approach

The concept of the Independent Dominating Set (IDS) offers a graph-theoretic strategy for selecting special nodes in a network to ensure both full coverage and minimized overlap between their control domains. To make the idea more intuitive, we again represent the network as an undirected graph $G = (V, E)$, where V is the set of all nodes (vertices), and E is the set of edges (links) that connect pairs of nodes.

A subset $D \subseteq V$ is called a *dominating set* if every node in the network either belongs to D, or has at least one direct neighbor in D. In simple terms, every node that is not in D must be directly connected (by an edge) to at least one node from D. Formally, this is expressed as:

$$\forall v \in V \setminus D, \quad \exists u \in D \quad \text{such that} \quad (u, v) \in E \tag{5.1}$$

This means that all nodes in the network are either selected for D or are "covered" by being a neighbor to someone in D.

The *independence* condition further refines the selection by requiring that no two nodes in D are directly connected to each other. In other words, none of the chosen nodes in D should be neighbors:

$$\forall u_1, u_2 \in D, \quad (u_1, u_2) \notin E \tag{5.2}$$

This ensures that the selected nodes are as far apart as possible (at least two hops away), which helps

prevent overlap and redundancy—especially important in SDN, where these nodes typically represent controller locations.

Summarizing, an Independent Dominating Set D must satisfy:

$$\textbf{Dominating:} \quad \forall v \in V \setminus D, \quad \exists u \in D \quad \text{with} \quad (u, v) \in E$$

$$\textbf{Independent:} \quad \forall u_1, u_2 \in D, \quad (u_1, u_2) \notin E$$

This dual property guarantees that every node is covered, while ensuring that the selected nodes (e.g., controllers) are not clustered together, but distributed across the network.

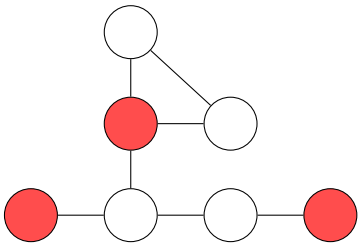


Fig. 5.1. Example of an Independent Dominating Set (IDS) in a simple network graph. Red nodes belong to the IDS.

The central challenge associated with finding an IDS is its computational complexity, as determining an independent dominating set of minimal cardinality is classified as an NP-hard problem. Consequently, exact solutions become computationally infeasible for large-scale networks, necessitating the use of heuristic or approximation algorithms. The relevance of IDS in network contexts, particularly within SDN, arises from its ability to balance coverage with independence, thereby minimizing the overlap of control domains and effectively reducing complexity in inter-controller communication. Specifically, the independence criterion limits potential interference or redundant communication among controllers, while the dominance condition guarantees comprehensive network coverage and responsive interactions between controllers and switches.

5.2. K-Means Approach

K-Means is a widely used clustering algorithm in data analysis and machine learning [21], originally designed to partition a set of points into K groups (clusters) such that points within each cluster are as close as possible to the cluster's centroid. The algorithm iteratively assigns each point to its nearest centroid and then updates the centroids as the mean position of all assigned points. This process continues until the cluster assignments stabilize.

In the context of controller placement in SDN networks, the K-Means approach adapts this classical method to partition the network into K subnetworks (clusters), each managed by a controller. Here, "closeness" is measured not by geometric distance, but by the network's shortest path distances, and centroids correspond to actual network nodes that can host controllers. The primary goal is to group switches in a way that minimizes the communication latency between them and their assigned

controller, mirroring the clustering intuition of the original K-Means algorithm but applying it to the topology of the network.

To formalize this, we again model the physical network as an undirected graph $G = (V, E)$, where V is the set of nodes (switches) and E is the set of links. The objective is to divide V into K disjoint subsets $\{V_1, V_2, \ldots, V_K\}$, each corresponding to a subnetwork managed by a controller:

$$\bigcup_{i=1}^{K} V_i = V, \quad V_i \cap V_j = \varnothing \quad \forall i \neq j \tag{5.3}$$

Each subnetwork should remain connected, ensuring that communication between switches and their controller is always feasible.

In the classical K-Means algorithm, centroids are updated by calculating the mean of data points within each cluster. However, in a network context, the centroid must correspond to an actual node in the graph—since only existing nodes can host controllers. Therefore, the centroid of each cluster C_i is selected as the node within C_i that minimizes the total distance to all other nodes in that cluster:

$$c_i = \arg\min_{v \in C_i} \sum_{u \in C_i} d(u, v) \tag{5.4}$$

where $d(u, v)$ denotes the shortest path distance between nodes u and v in the network.

The algorithm proceeds iteratively through two main steps:

1. **Assignment step:** Each switch is assigned to the cluster whose centroid (controller candidate) is nearest in terms of shortest path distance:

$$v \in C_i \text{ if } d(v, c_i) < d(v, c_j) \ \forall j \neq i \tag{5.5}$$

2. **Update step:** For each cluster, a new centroid is chosen according to the rule above, ensuring the centroid remains a real node and optimally located within the cluster.

These steps are repeated until cluster assignments stabilize and no further changes occur in centroids.

A practical challenge arises in the initialization of centroids. Random selection can lead to highly variable and sometimes suboptimal clustering results, with large discrepancies in controller-switch latency. To address this, improved methods such as the approach proposed by Wang et al. [22] use a deterministic initialization strategy inspired by the Katsavounidis method [23]: the node that is farthest from current centroids is chosen as the next initial center. This ensures that controllers are well-dispersed throughout the network, significantly reducing the worst-case latency compared to random initialization.

Despite its efficiency and scalability, the K-Means approach is not without limitations. While it is highly effective at reducing communication latency and ensuring that each controller serves a distinct, geographically coherent group of switches, it does not directly account for additional operational constraints such as controller capacity, reliability, or load balancing. Nevertheless, its clear and flexible framework makes it a strong foundation upon which further optimization objectives can be incorporated.

5.3. High Degree with Independent Dominating Set (HDIDS) Method

The High Degree with Independent Dominating Set (HDIDS) method offers an efficient heuristic for controller placement in SDN, uniting the coverage guarantees of the Independent Dominating Set (IDS) with the practical benefits of prioritizing nodes that play a central topological role. The rationale behind this approach is that high-degree nodes—those with the most direct neighbors—tend to occupy strategic positions in the network, making them well-suited as controller locations to minimize switch-to-controller latency.

5.3.1. Algorithmic Reasoning and Heuristic Integration

Unlike the classical IDS, which focuses solely on coverage and independence, HDIDS explicitly incorporates topological heterogeneity by selecting candidate controller nodes based on their degree. This bias towards high-degree nodes ensures that the resulting set of controllers not only covers the entire network, but also leverages the structural advantages of network hubs. In the undirected network graph $G = (V, E)$, with V as the set of nodes and E as the set of links, the degree $d(v)$ of a node $v \in V$ denotes the number of its direct neighbors. Alowa et al. [11] formulate the objectives of the HDIDS approach by minimizing request-handling latency in SDN, directly targeting two key metrics: the average response time (RS_{avg}) and the maximum response time (RS_{max}) between controllers and switches. These metrics are defined as follows:

- **Average response time** (RS_{avg}): This metric measures the mean shortest-path delay between each switch and its assigned controller, thereby reflecting the overall responsiveness of the control plane in the network. Formally, it is given by:

$$RS_{avg} = \frac{1}{|V|} \sum_{v \in V} d(v, c) \tag{5.6}$$

  where c denotes the controller assigned to node v and $d(v, v)$ is the shortest-path distance (e.g., number of hops or aggregated link latency) from v to its controller.

- **Maximum response time** (RS_{max}): This metric captures the worst-case (longest) delay from any switch to its nearest assigned controller. It represents the upper bound on the request handling latency experienced in the network, and is formally defined as:

$$RS_{max} = \max_{v \in V} d(v, v) \tag{5.7}$$

  Minimizing RS_{max} is crucial to ensure that no switch in the network experiences unacceptably high delays when communicating with its controller, thus improving the overall quality of service and network reliability.

By optimizing these objectives, the algorithm aims to reduce both the typical and the worst-case delays encountered when processing flow requests, thereby enhancing SDN control plane performance [11].
The HDIDS algorithm can be characterized by the following steps:

1. **Initialization:** Set the dominating set $D = \emptyset$; mark all nodes as uncovered.

2. **Selection Loop:** While there exists at least one node in V not dominated by D (i.e., not in D nor adjacent to any node in D):
   (a) Among all uncovered nodes not adjacent to any node in D, select the node v^* with the highest degree $d(v^*)$.
   (b) Add v^* to the dominating set D.
   (c) Mark v^* and all its neighbors as covered.
3. **Assignment:** After constructing D, each node in D is designated as a controller. All remaining (non-dominating) nodes are assigned to the nearest controller in D.

This construction maintains the independence property—no two controllers are directly connected—while ensuring full network coverage: every node is either a controller or a neighbor to one. By focusing on nodes with maximal degree, the algorithm tends to select network hubs, yielding placements that, in empirical evaluations, significantly reduce both RS_{avg} and RS_{max} compared to baseline IDS or K-Means-based methods [11]. As a result, HDIDS directly minimizes the propagation component of request-handling latency, making it well suited to SDN scenarios where both average and worst-case responsiveness are critical for efficient network operation.

5.3.2. Properties and Limitations

The principal advantage of HDIDS lies in its dual optimization: it preserves the theoretical guarantees of the IDS (minimal, non-overlapping coverage) and pragmatically biases selection toward topological cores, leading to improved communication reach and reduced propagation delay. This results in a typically smaller set of controllers required to satisfy stringent latency constraints, as shown in comparative studies.

However, the method's reliance on degree centrality may be less effective in networks with homogeneous degree distributions or randomized topologies, where the distinction between node centralities is weak. Moreover, as an inherently greedy, heuristic method, HDIDS does not guarantee global optimality and may require further adaptation to incorporate capacity, load balancing, or resilience constraints in complex operational scenarios.

5.3.3. Illustrative Example of HDIDS Execution

To illustrate the HDIDS algorithm [11] in detail, consider a representative network composed of 12 interconnected nodes. The algorithm proceeds by iteratively selecting controller nodes based on the degree heuristic, combined with an independence constraint.

In the first iteration, as shown in Figure 5.2, node 5 is selected as the first controller. This node has the highest degree in the graph ($d = 5$), making it an influential hub within the topology. The choice ensures a broad initial coverage, and the node is not adjacent to any existing controller, satisfying the IDS condition.

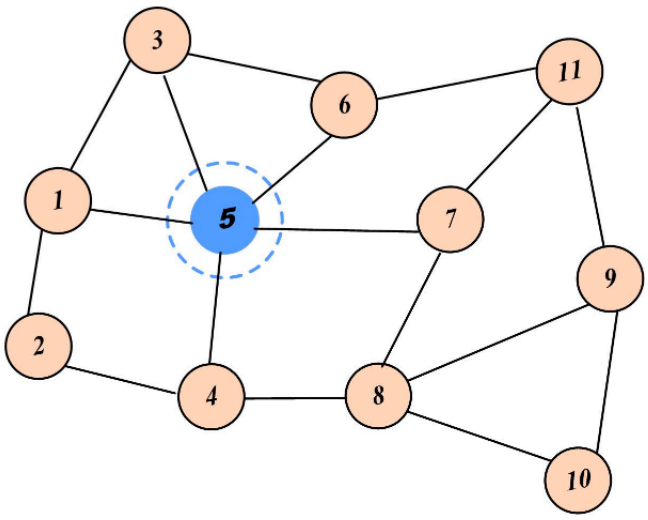


Fig. 5.2. Step 1: Selection of node 5 as the first controller (degree = 5)

Next, the algorithm seeks a second controller among nodes not directly connected to node 5. Among these, node 8 stands out due to its high degree ($d = 4$) and its relatively low aggregate shortest path length to the rest of the network. It is selected as the second controller, ensuring expanded coverage while preserving independence. Node 8 serves a distinct segment of the network, as visualized in Figure 5.3.

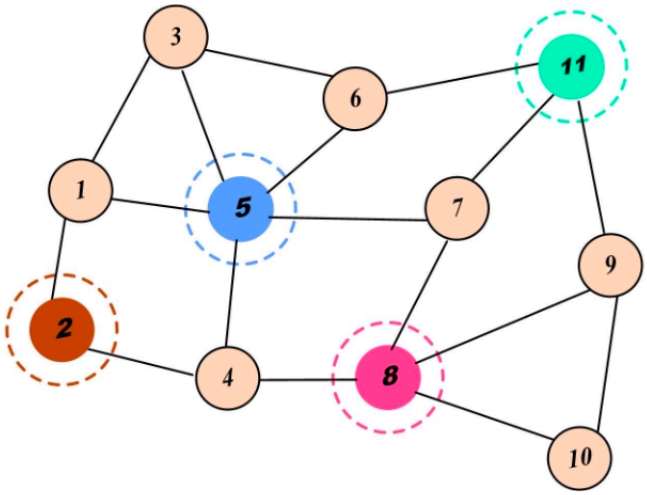


Fig. 5.3. Step 2: Selection of node 8 and other intermediate controllers

The algorithm continues by selecting node 11 as the third controller, due to its sufficient degree and independence from previously selected controllers (nodes 5 and 8). Finally, node 2 is chosen as the fourth controller since it remains undominated and is not adjacent to any prior controller node.

The culmination of the HDIDS process results in the selection of four controllers (nodes 5, 8, 11, and 2). Each non-controller node is then assigned to the closest controller based on shortest path distance, effectively partitioning the network into distinct control domains. This is depicted in

Figure 5.4, where each domain is clearly separated, ensuring efficient local control.

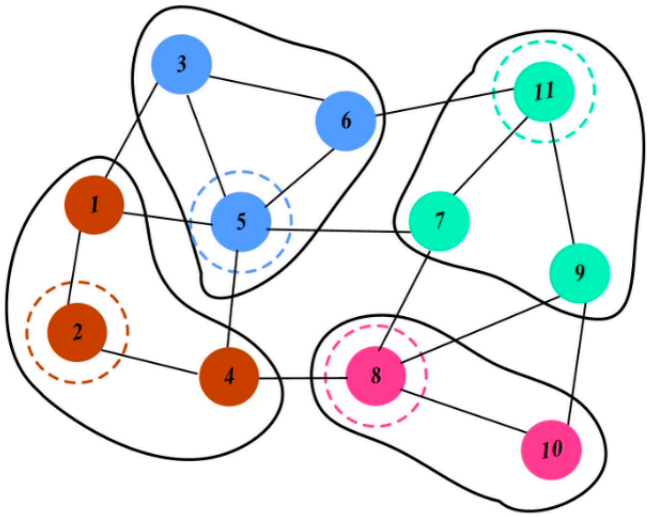


Fig. 5.4. Step 3: Final partitioning of the network after four controller selections

This method ensures that the selection process respects topological prominence (high node degree), while avoiding clustering overlap through independence enforcement. As a result, HDIDS produces a near-optimal controller placement with strong performance in terms of response time and control plane coverage.

5.3.4. Experimental Evaluation and Performance Results

The HDIDS method was evaluated through extensive simulations on four real-world topologies from the Internet Topology Zoo: Internet2 OS3E, Bell Canada, ATT North America, and TataNID. These networks vary significantly in size and structural complexity, with node counts ranging from 25 to 145. The simulations were conducted on a system with Intel i7 CPU and 16GB RAM, using distance-based propagation delays derived from fiber-optic infrastructure and the A* shortest path algorithm with Haversine heuristics.

The evaluation focused on two performance metrics: average response time and maximum response time. HDIDS was benchmarked against the Optimized K-Means (OK-Means) algorithm, with results plotted against the number of deployed controllers.

Average Response Time Analysis

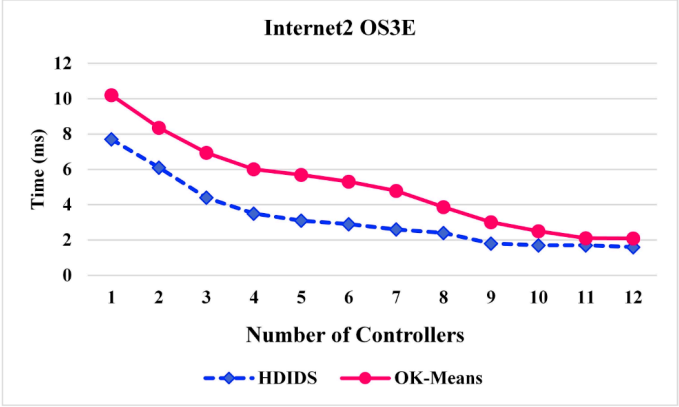


Fig. 5.5. Average Response Time on Internet2 OS3E

In the Internet2 OS3E topology (Figure 5.5), HDIDS consistently outperforms OK-Means [22] across controller counts up to 12. For instance, with 10 controllers, HDIDS achieves an average response time of 1.83 ms, while OK-Means yields 2.49 ms. The performance gap narrows as controller count increases, reflecting convergence in placement sets.

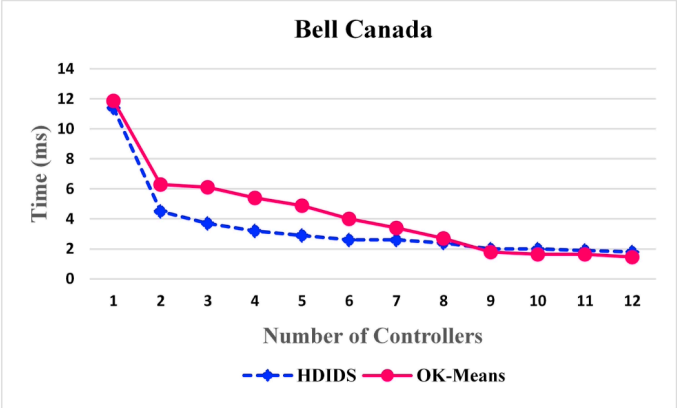


Fig. 5.6. Average Response Time on Bell Canada

In Bell Canada (Figure 5.6), HDIDS dominates for up to 8 controllers. Beyond this point, OK-Means performs marginally better. This suggests that HDIDS is particularly advantageous under constrained controller budgets.

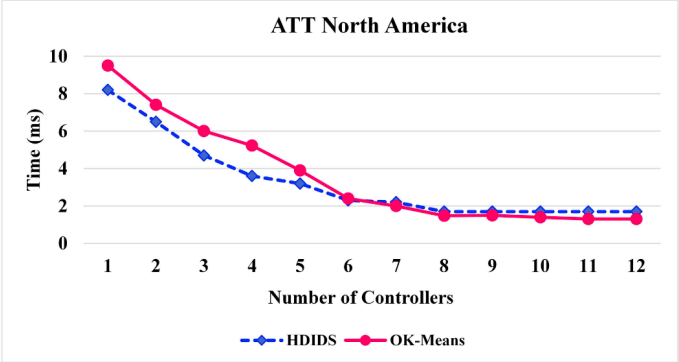


Fig. 5.7. Average Response Time on ATT North America

ATT North America results (Figure 5.7) mirror this trend: HDIDS performs better up to 6 controllers, then converges with OK-Means. This supports the view that HDIDS is effective for modest-scale deployments.

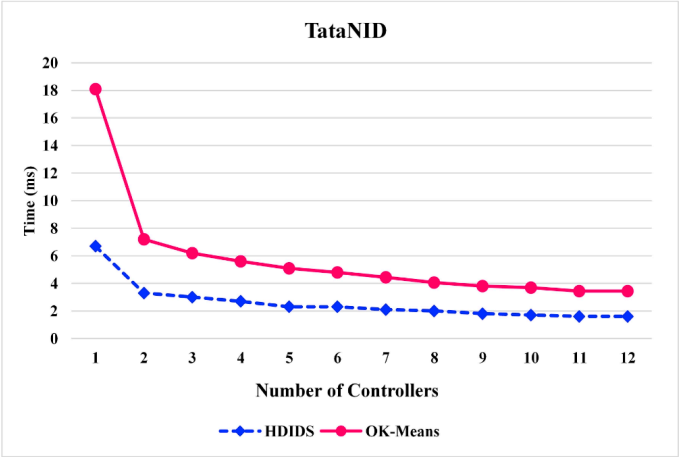


Fig. 5.8. Average Response Time on TataNID

In the large TataNID topology (Figure 5.8), HDIDS clearly outperforms OK-Means across all controller counts. For 11 controllers, HDIDS reaches 1.84 ms compared to 3.79 ms by OK-Means, underscoring HDIDS's scalability.

Maximum Response Time Analysis

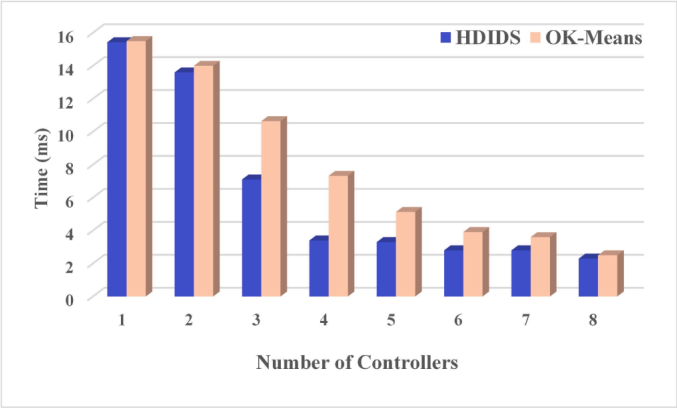


Fig. 5.9. Maximum Response Time on Internet2 OS3E

For maximal response time, HDIDS shows a consistent reduction across all topologies. On OS3E (Figure 5.9), response time drops from 15.5 ms to 3.4 ms as controller count increases.

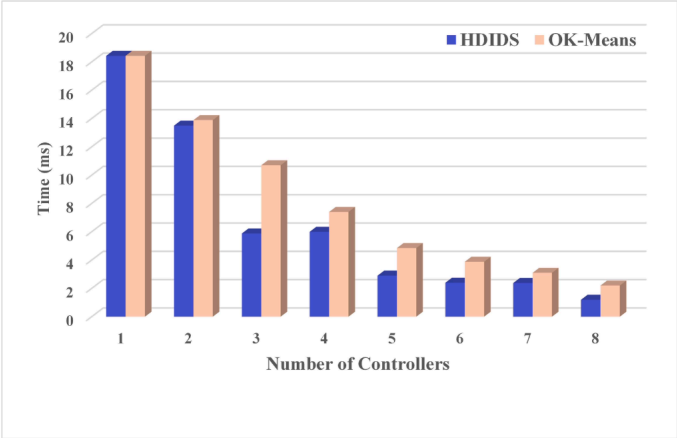


Fig. 5.10. Maximum Response Time on Bell Canada

Bell Canada (Figure 5.10) sees similar improvements, dropping from 18.43 ms to 6 ms with 4 controllers under HDIDS.

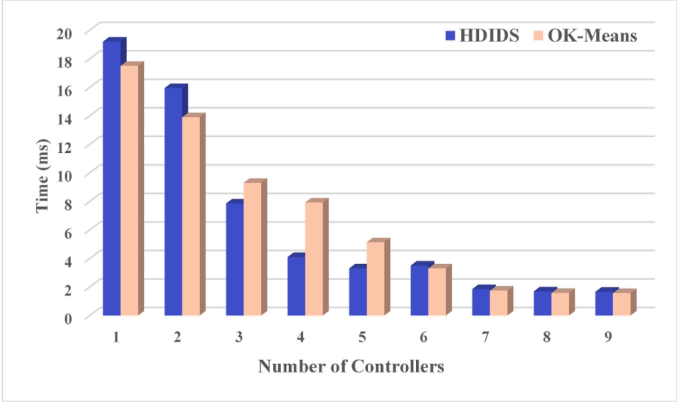


Fig. 5.11. Maximum Response Time on ATT North America

Interestingly, ATT (Figure 5.11) shows that HDIDS underperforms OK-Means at very low controller counts, but surpasses it with 3 or more controllers.

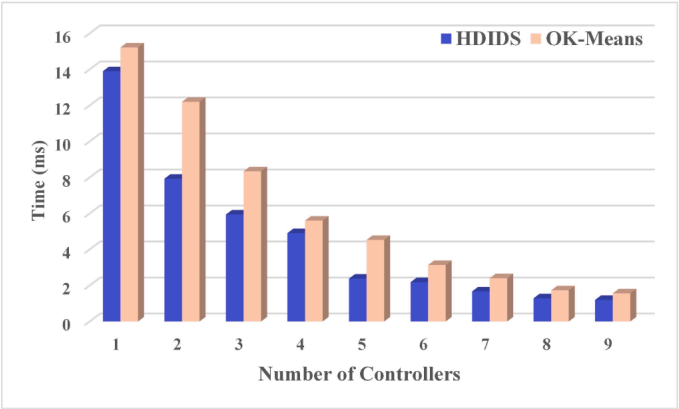


Fig. 5.12. Maximum Response Time on TataNID

TataNID (Figure 5.12) confirms HDIDS's suitability for large networks: maximal delay drops from 13.9 ms to 1.2 ms across 10 controllers.

Threshold-Based Controller Requirements

Summary tables in [11] pictures the number of controllers needed to maintain average or maximal response time below fixed thresholds (2ms, 3ms, 4ms). HDIDS consistently requires fewer controllers than OK-Means to stay within these bounds, particularly in the TataNID case where reductions up to 80% are observed. These results highlight HDIDS's strong performance under both average and worst-case delay metrics, particularly in sparse and large-scale networks. However, they also suggest potential diminishing returns in topologies with uniform structure or when deploying large controller sets.

5.3.5. Factors Contributing to HDIDS Performance

The performance of the HDIDS method is determined by several interrelated factors, as revealed by extensive evaluation across diverse real-world network topologies. Key contributors include:

- **Topological Centrality Exploitation:** By systematically selecting high-degree nodes as candidate controller locations, HDIDS leverages network hubs that are naturally well-connected. This ensures that controllers are positioned in regions offering maximal communication reach, directly contributing to the reduction of both average and worst-case switch-to-controller response times.
- **Iterative Independence Enforcement:** The independence constraint—no two controllers are adjacent—prevents unnecessary overlap of control domains. This strategic distribution enhances coverage efficiency, enabling each controller to manage distinct areas of the network with minimal redundancy.
- **Greedy Selection and Tie-Breaking Mechanism:** In cases where multiple nodes have the highest degree, HDIDS further refines selection by choosing the node with the minimum total

shortest-path distance to all other nodes. This dual-criterion promotes not only local connectivity but also global proximity, which is particularly effective in minimizing propagation delays.

- **Scalability Across Network Sizes:** Empirical results show that HDIDS consistently outperforms Optimized K-Means in both medium-scale (e.g., Internet2 OS3E, ATT North America, Bell Canada) and large-scale (e.g., TataNID) networks, especially when the number of controllers is constrained. The method demonstrates substantial gains in reducing the number of controllers needed to meet strict latency thresholds, often requiring up to 50% fewer controllers than clustering-based approaches for similar performance targets.
- **Adaptation to Topological Heterogeneity:** The method is most effective in heterogeneous networks with pronounced hub structures, where degree centrality meaningfully distinguishes nodes. In large, geographically dispersed, or hierarchically organized networks, this advantage is especially significant.
- **Response to Latency Constraints:** For both average and maximum response time thresholds, HDIDS requires fewer controllers to satisfy given bounds, particularly in large or sparse topologies. For example, in the TataNID network, HDIDS achieved between 33% and 80% reduction in controller count for average response time thresholds between 2 ms and 4 ms, compared to optimized K-Means.
- **Limitations in Uniform Topologies:** The method's effectiveness diminishes in highly regular or random graphs, where the variance in node degrees is minimal and centrality provides little discriminative power. In such cases, HDIDS may offer only marginal improvement over alternative heuristics.

In summary, HDIDS achieves superior controller placement by harnessing topological features—particularly node degree—while maintaining independent coverage. Its greedy, adaptive selection strategy ensures efficient scalability and performance, especially in networks exhibiting structural heterogeneity and demanding stringent latency requirements [11].

5.3.6. Potential Bottlenecks and Limitations of HDIDS

Despite its demonstrated effectiveness and theoretical foundation, the HDIDS method exhibits several potential limitations, as discussed in [4]. Our own analysis, conducted after an extensive review and classification of various CPP strategies and methods, allowed for a critical evaluation of certain inherent limitations of HDIDS:

- **Dependence on Node Degree Exclusively:** HDIDS relies solely on degree centrality as a criterion for selecting controller nodes. In topologies with uniform degree distributions or the absence of distinct hub nodes, this approach becomes less discriminative, potentially leading to suboptimal controller placements and higher latency outcomes.
- **Lack of Load Consideration:** HDIDS optimizes primarily for latency based on node degrees and does not explicitly account for controller load or traffic patterns. In scenarios with uneven traffic

distribution, selected controllers may become overloaded, causing queuing delays and reducing the actual performance benefits of low-latency placements.

- **Static Topology Assumption:** The method assumes a static network topology during controller placement computation. Therefore, in dynamic environments or scenarios prone to node and link failures, predetermined placements can quickly become suboptimal, negatively impacting latency and network resilience.
- **Greedy Heuristic and Optimality Guarantees:** HDIDS employs a greedy algorithm to construct the Independent Dominating Set. While computationally efficient, this heuristic offers no global optimality guarantees due to the inherent NP-hardness of the underlying placement problem. Consequently, it can yield locally optimal solutions that are globally suboptimal.
- **Neglect of Traffic Synchronization and Multi-Objective Scenarios:** Current implementations of HDIDS do not consider inter-controller synchronization, reliability, or energy efficiency explicitly. Thus, its placements might not satisfy broader operational objectives necessary in comprehensive SDN deployments.

Given these limitations, HDIDS performs strongly in stable, degree-heterogeneous networks focused on latency minimization, but additional measures—such as load balancing, reliability-enhancement strategies, or multi-objective hybrid approaches—are recommended for mission-critical and dynamic network environments.

5.3.7. Critical Assessment and Value of Degree-Based Approach

Although the HDIDS algorithm demonstrates certain limitations—particularly its exclusive reliance on node degree and lack of explicit multi-objective or dynamic adaptation—its core heuristic remains highly relevant for controller placement in SDN. As highlighted in the comparative study by Zobary [4], HDIDS, as a state-of-the-art (SOTA) degree-based approach, provides a competitive and conceptually transparent solution to minimizing switch-to-controller request handling latency. The key intuition—selecting controllers based on their connection degrees and minimizing the average response time between each controller and its forwarding nodes—offers a strong foundation for further algorithmic enhancements and hybridization.

The strategic use of node degree allows HDIDS to rapidly identify network hubs, which, in many realistic topologies, act as natural aggregation points for efficient control-plane communication. This principle aligns well with the mission-critical objective of reducing request handling delays in SDN, as controllers placed on high-degree nodes are statistically more likely to serve a larger fraction of the network with shorter propagation times. The success of this heuristic, repeatedly validated in both synthetic and real-world topologies, underscores its value as a baseline for latency-oriented controller placement.

However, as contemporary SDN environments grow in size and heterogeneity, the need arises to address the aforementioned shortcomings—such as sensitivity to uniform topologies, static operation,

and potential load imbalance—by integrating degree centrality with additional heuristics. Future work can focus on multi-metric algorithms that complement node degree with betweenness and closeness centrality, explicit load-awareness, and adaptive or dynamic reassessment, thereby extending the effectiveness of HDIDS beyond its original scope. Such hybrid approaches promise to further minimize request handling latency while maintaining the computational efficiency and conceptual clarity that are the hallmarks of degree-based selection.

In summary, while HDIDS in its current form is not without weaknesses, its focus on high-degree nodes as controller candidates represents both a state-of-the-art and a practically effective baseline for latency-aware controller placement. This design philosophy, as confirmed in recent research and empirical comparison with Advanced K-Means [4], positions degree-based heuristics as a robust starting point for developing even more capable solutions to the Controller Placement Problem in SDN.

5.4. Advanced K-Means Method

The Advanced K-Means method represents a significant evolution of the classical K-Means clustering approach, specifically tailored to address the complex requirements of controller placement in Software-Defined Networks. Building upon the traditional iterative assignment and centroid update mechanism, this method incorporates a series of enhancements that make it sensitive to both network structure and operational constraints. Most notably, it integrates additional optimization criteria such as controller capacity limits, node importance weighting, and topological features—including metrics like closeness and betweenness centrality—directly into the clustering and controller assignment process [4].

Unlike the basic K-Means, where clusters are formed solely on the basis of proximity or minimal distance, the Advanced K-Means approach adaptively reshapes cluster boundaries in response to dynamic network conditions. This can involve real-time adjustment of controller locations, redistribution of switches in accordance with changing demand patterns, or the prioritization of topologically critical nodes as new centroids. By doing so, the algorithm is capable of distributing load more evenly across controllers, enhancing fault tolerance, and maintaining low-latency communication even in the face of fluctuating or geographically skewed traffic demands.

The integration of topology-aware metrics into centroid selection is a key innovation, as it ensures that cluster centers are chosen not merely for their geometric position, but for their strategic influence over the network. As a result, both average and worst-case switch-to-controller latencies are minimized, and the resilience of the control plane to failures or sudden traffic spikes is substantially improved.

5.4.1. Algorithmic Details and Operational Principles

The Advanced K-Means method refines traditional clustering by incorporating network-specific metrics and intelligent center initialization to achieve latency-aware controller placement. The algorithm

operates on the SDN network topology modeled as an undirected graph $G = (V, E)$, where V represents the nodes (switches) and E the links. The main objective is to partition the network into K clusters—each managed by a controller—in a way that minimizes the average propagation delay between switches and controllers.

The method starts with a careful initialization of cluster centers, rather than random selection. The node degree is calculated for each switch, and the average node degree is determined. The initial center is chosen as the node with the highest degree that also meets or exceeds the average degree, thus avoiding the selection of peripheral or weakly connected nodes. When multiple candidates exist, the algorithm selects the node with the minimum sum of shortest path distances to all other nodes, ensuring both connectivity and centrality.

Subsequent centers are selected iteratively. At each step, the next center is chosen as the node farthest in terms of shortest path from all previously selected centers, again ensuring it satisfies the degree threshold. This guarantees a well-dispersed set of controllers, reducing cluster overlap and maximizing coverage.

Once initial centers are established, nodes are assigned to the nearest cluster center based on shortest path distance. Each cluster's centroid is then recalculated: within each cluster, the node with the minimum total shortest path distance to all other cluster members is chosen as the new center. The assignment and update steps are repeated until cluster centers stabilize.

This approach—grounded in topological metrics and intelligent initialization—ensures that controllers are placed in both well-connected and geographically dispersed positions. The algorithm not only minimizes propagation latency but also promotes load balancing by preventing controller clustering and excessive overlap in switch assignments. By updating centroids with respect to shortest path rather than Euclidean distance, the method respects the true structure of the network and avoids infeasible placements.

In summary, the Advanced K-Means method brings together the speed of clustering, the strategic insight of topological analysis, and the adaptability required for real-world SDN deployments. Its iterative, metric-driven selection process ensures robust, latency-optimized controller placement that consistently outperforms naive or random clustering methods in both average and worst-case scenarios [4].

5.4.2. Pseudocode of the Advanced K-Means Algorithm

The operational flow of the Advanced K-Means method is encapsulated in two algorithmic stages: initialization of the first cluster center and the main partitioning process. Below, we present detailed pseudocode for both stages, following the original specification from [4]. The pseudocode outlines the logic for selecting optimal initial centers and iteratively partitioning the network into latency-minimizing clusters. All steps operate on a graph $G = (V, E)$, with N denoting the number of nodes and K the number of controllers to deploy.

Algorithm 1 Selecting the First Initial Center

Require: Network graph $G = (V, E)$, number of nodes N

Ensure: First cluster centroid

1: **for** each $i \in V$ **do**

2: Compute node degree D_i for node i

3: **end for**

4: Compute average node degree $\rho \leftarrow \text{round}\left(\frac{1}{N}\sum_{i=1}^{N} D_i\right)$

5: $S \leftarrow \{\ i \in V$ such that $D_i \geq \rho\ \}$

6: **Select** node $i^* \in S$ with minimal $\sum_{j \in V} d(i, j)$

7: **return** i^* as the first cluster centroid

Algorithm 1 aims to select the most suitable initial node (center) for clustering the network. It first calculates the degree (number of connections) for each node, then determines the average node degree. Only nodes with at least average connectivity are considered as candidates, ensuring that controllers are not placed on peripheral or weakly connected nodes. From these candidates, the algorithm chooses the node that is most "central"—that is, the one whose total distance to all other nodes is minimal. This ensures that the first controller is well connected and optimally positioned to serve as a cluster center, providing a robust foundation for the clustering process.

Algorithm 2 Network Partitioning (Advanced K-Means Clustering)

Require: Network graph $G = (V, E)$, number of nodes N, number of controllers K, average degree ρ, degree balance parameter β, first cluster centroid

Ensure: Final controller placements and cluster assignments

$C \leftarrow$ {first initial center}
Compute shortest path distance matrix d_{ij} for all $i, j \in V$
while $|C| < K$ **do**
  $S \leftarrow V \setminus C$
  for each $i \in S$ **do**
    if $D_i \geq \rho$ **then**
      Compute $min_{c \in C} d(i, c)$
    end if
  end for
  Select $i^* \in S$ (with $D_{i^*} \geq \rho$) having **maximal** $min_{c \in C} d(i^*, c)$
  $C \leftarrow C \cup \{i^*\}$
end while
repeat
  for each node $v \in V$ **do**
    Assign v to the cluster with nearest center in C (by $d(v, c)$)
  end for
  for each cluster **do**
    Update cluster center: select node with minimal sum of distances to all other cluster members
  end for
until cluster centers no longer change and degree condition $D_c \geq \rho$ holds for all centers
return Cluster assignments and final centers C

Algorithm 2 describes the main clustering process for the Advanced K-Means method. Starting from the initial center found by Algorithm 1, the algorithm iteratively selects additional centers to maximize their separation in the network, ensuring good coverage and minimal overlap. Nodes are assigned to their closest center, forming preliminary clusters. The centers are then updated within each cluster to the most central node—one that has the lowest total distance to other cluster members—guaranteeing efficient communication within the cluster. This process repeats, refining both cluster assignments and centers, until no further improvements are possible. Throughout, a minimum node degree constraint is enforced for all centers, making sure controllers are always located at well-connected, robust positions in the network.

5.4.3. Experimental Evaluation and Performance Results

Experimental Setup

To rigorously assess the efficacy of the Advanced K-Means algorithm for SDN controller placement, a series of simulation experiments were conducted. The evaluation was performed using MATLAB R2018a on a workstation equipped with an Intel Core i5-12500H CPU (2.50 GHz) and 16 GB RAM. The primary testbed was the Internet2 OS3E topology, obtained from the Internet Topology Zoo [24][1], which is widely recognized as a realistic and challenging benchmark for SDN research. The OS3E network consists of 34 nodes and 41 edges, closely mirroring real-world operational conditions encountered by research and education network infrastructures.

The primary goal of the experimental study was to minimize the average propagation delay between controllers and switches, while also considering controller load balancing. The optimal number of controllers ($K = 5$) was determined by evaluating the cost-benefit ratio and analyzing the trade-off between network performance and resource expenditure, as described by the author [4].

The Advanced K-Means algorithm was benchmarked against four state-of-the-art controller placement algorithms:

- **DBCP** [25]: Density-Based Controller Placement
- **POCO** [5]: Pareto-based Optimal Controller Placement
- **CNPA** [26]: Clustering-based Network Partition Algorithm
- **HDIDS** [11]: High Degree with Independent Dominating Set

The shortest-path calculations used Dijkstra's algorithm to compute all-pairs distances within the graph. Evaluation metrics focused on (i) average propagation latency and (ii) the maximum load per controller (i.e., the maximum number of switches managed by any one controller).

[1]The original source referenced by the author is no longer available. Therefore, the citation here refers to currently accessible online resources.

Average Propagation Latency

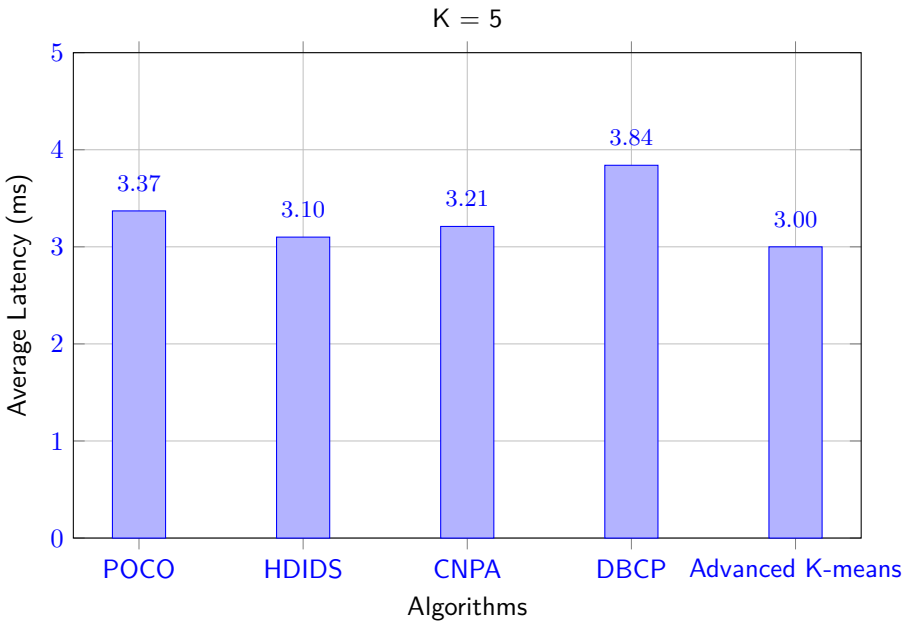


Fig. 5.13. Comparison of average propagation latency (K = 5) across methods.

The first set of experiments compared the average propagation latency obtained by the Advanced K-Means algorithm with the competing approaches for $K = 5$ controllers. As shown in Figure 5.13, the Advanced K-Means achieved the lowest average latency (3.00 ms), outperforming POCO (3.37 ms), CNPA (3.21 ms), HDIDS (3.09 ms), and DBCP (3.85 ms). These results correspond to relative reductions in average latency of 11% compared to POCO, 7% to CNPA, 3% to HDIDS, and a substantial 22% compared to DBCP [4].

The primary factor driving this improvement is the algorithm's use of both node degree and shortest-path centrality when selecting cluster centers (controller locations), ensuring controllers are optimally positioned for both connectivity and reachability. Unlike DBCP and CNPA, which focus primarily on cluster density and may neglect topological centrality, or POCO, which can overemphasize inter-controller latency at the expense of controller-switch latency, the Advanced K-Means approach achieves a balanced and latency-aware deployment. The close performance of HDIDS is attributed to its degree-based selection, but HDIDS does not incorporate shortest-path distances in its optimization, resulting in slightly inferior latency performance.

Maximum Controller Load

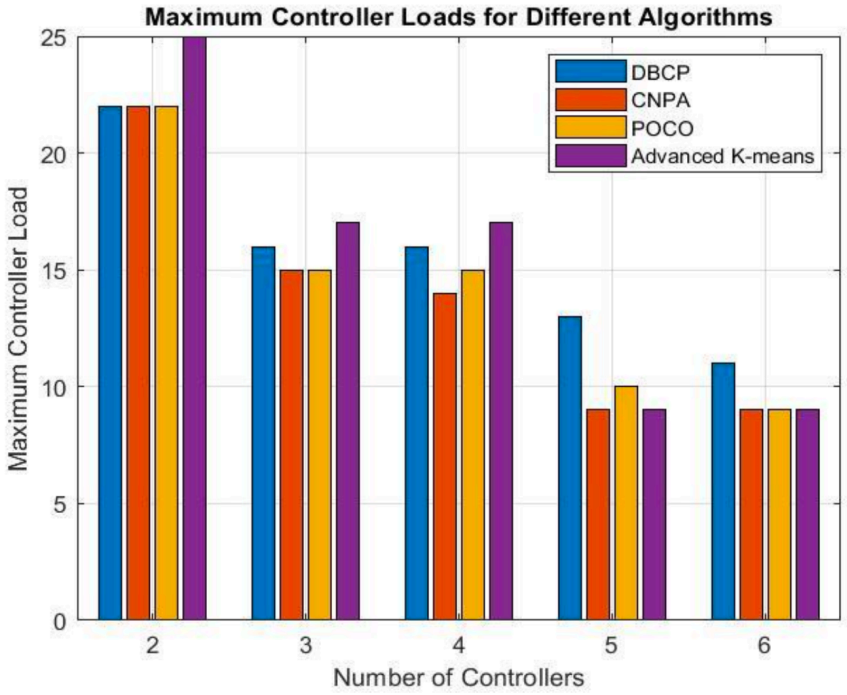


Fig. 5.14. Comparison of maximum controller load for different approaches as a function of the number of controllers in OS3E.

The second major performance metric concerns load balancing, quantified as the maximum number of switches assigned to any single controller. In the OS3E topology, Advanced K-Means demonstrates competitive or superior load balancing compared to existing solutions. For small numbers of controllers ($K = 2$), Advanced K-Means exhibits a slightly higher maximum load (25 switches), but as K increases, the difference diminishes. At $K = 5$, the algorithm achieves parity with CNPA and surpasses both POCO and DBCP in reducing the maximum controller load [4].

The reason for this improvement lies in the algorithm's iterative refinement of cluster centers: by strategically spacing controllers apart and reassigning nodes to their closest controller in terms of network distance, the load can be distributed more evenly across controllers. The combined use of node degree and shortest-path criteria ensures that controllers are located not only at well-connected nodes but also in positions that minimize clustering imbalance. As a result, Advanced K-Means is able to achieve effective traffic load balancing while simultaneously reducing latency.

5.5. Discussion and Key Observations

The experimental results demonstrate that Advanced K-Means provides a robust solution to the controller placement problem, consistently reducing both latency and load imbalance in comparison to state-of-the-art algorithms. The improvement is most significant in average propagation latency, which is a direct consequence of the algorithm's hybrid use of topological and distance-based heuristics

for cluster center selection. Moreover, the approach exhibits strong adaptability as the number of controllers increases, closing the load gap with other methods. As noted in the discussion by Zobary [4], these enhancements are achieved without introducing excessive computational complexity or parameter tuning, and the methodology remains applicable to real-world, large-scale topologies.

Despite its advantages, the Advanced K-Means algorithm is not without limitations. One of the principal challenges emerges in sparse topologies, where the number of nodes with degree above the average threshold ($D_i \geq \rho$) is inherently limited. In such environments, the algorithm's insistence on the degree constraint may compel the placement of controllers on nodes that are not truly optimal from a latency perspective, resulting in increased average propagation delays. Moreover, even in networks that are not overtly sparse, the diversity of real-world topologies means that node degree alone may not sufficiently capture the nuances of centrality or communication efficiency required for robust controller placement.

The rigid focus on node degree neglects other topological properties, such as betweenness and closeness centrality, which can be crucial for effective network management. In the Advanced K-Means approach, closeness centrality is only invoked as a tie-breaker in cases where multiple nodes share the maximum degree—serving a secondary, rather than an equally-weighted, role in the selection process. As a result, nodes with moderate degree but high betweenness or closeness—potentially offering more strategic controller placement, particularly in heterogeneous or irregular topologies—may still be overlooked in favor of purely degree-based candidates. Introducing an adaptive strategy that elevates the importance of alternative centrality measures in the initial center selection could further improve performance, especially in networks with diverse structural characteristics.

Another important limitation lies in the algorithm's strategy for selecting subsequent cluster centers after the initialization phase. While the initial center is chosen with respect to both node degree and centrality, the prevailing heuristic for subsequent selections aims to maximize the spatial separation between centers by picking nodes that are as far as possible from those already chosen. Although this "maximum dispersion" heuristic intuitively promotes coverage and distribution of controllers across the network, it may also lead to suboptimal placements, particularly in networks with non-uniform or irregular connectivity patterns. In such cases, enforcing maximal separation does not always coincide with minimizing overall latency or optimizing load balance—especially if it causes the algorithm to disregard local clusters of demand or regions with specific communication bottlenecks. Moreover, this approach can prove insufficiently adaptive when confronted with networks where high-demand areas or critical interconnects are not uniformly spaced. As a result, there is a risk that subsequent controller placements will not adequately reflect the real operational demands or topological idiosyncrasies of the underlying network.

To address these challenges, future enhancements could involve relaxing the degree threshold or introducing a hybrid selection metric that integrates multiple centrality measures—such as a weighted combination of degree, betweenness, and closeness centrality. This would allow the algorithm to flexibly

adapt to the structural characteristics of the underlying network and optimize controller placement accordingly. Assigning tunable weights to these metrics could be particularly valuable, enabling the method to be tailored to specific network scenarios and assessed for efficiency gains on a per-case basis. Such an adaptive strategy would not only mitigate the risk of suboptimal placement in sparse or atypical topologies, but also promote broader applicability and improved performance of the Advanced K-Means algorithm across diverse SDN environments.

In summary, while Advanced K-Means demonstrates strong overall performance, both the initialization and the iterative selection of cluster centers can introduce sensitivities to network structure and demand patterns. Recognizing and addressing these limitations—particularly the balance between dispersion and topological relevance in center placement—offers a promising direction for further refinement and broader applicability of the method. It is also worth considering that the selection of subsequent centers need not rely solely on maximizing their distance from existing centers. Instead, future improvements could introduce a more flexible and context-aware selection mechanism, where candidate centers are chosen based on a combination of network-driven metrics, each weighted according to the specific requirements and traffic characteristics of the underlying topology. Such an approach would enable controller placement strategies to better reflect the operational realities and unique demands of diverse networks, fostering more robust, efficient, and adaptable solutions.

6. LIMITATIONS AND ENHANCEMENTS STRATEGY

This chapter critically examines the limitations of the Advanced K-Means algorithm identified through both theoretical analysis and experimental evaluation. While the method demonstrates clear advantages in reducing latency and balancing controller loads in Software-Defined Networks, several algorithmic choices present opportunities for further enhancement. By systematically reviewing these limitations, we aim to outline promising directions for future research and potential modifications to the controller placement strategy. The discussion highlights not only the observed performance gaps but also motivates the exploration of new hybrid and adaptive mechanisms to address them.

6.1. Identified Gaps in Advanced K-Means

As introduced in the previous chapter—despite its hybrid approach—the Advanced K-Means algorithm exhibits notable gaps in both the selection of the initial center and the process of choosing subsequent cluster centers. The initialization phase relies primarily on the maximum node degree criterion, selecting as the first controller candidate the node with the highest connectivity in the network. While closeness centrality is used as a tie-breaker when multiple nodes share the maximum degree, it does not serve as a principal or equally weighted factor in the initial selection. Consequently, in networks where degree centrality does not adequately reflect other forms of node importance—such as betweenness or closeness—nodes that are highly influential for communication flow but possess only moderate degrees may still be systematically overlooked. This limitation can potentially lead to suboptimal coverage and higher propagation delays.

The process of selecting subsequent centers is driven by a heuristic of maximizing the distance from previously chosen centers—i.e., picking the node that is farthest from all current centers. This maximum dispersion strategy aims to achieve broad spatial distribution of controllers, thereby promoting network coverage. However, this approach may prove inadequate in topologies with irregular or non-uniform node distributions, where maximal separation does not necessarily align with minimizing average latency or balancing controller load. In such cases, the algorithm may place controllers in locations that are poorly suited to handle local traffic concentrations or critical communication paths, thereby undermining the effectiveness of the overall deployment.

Together, these identified gaps suggest that the Advanced K-Means algorithm, while effective in many scenarios, may not fully exploit the potential of network-aware placement strategies. Addressing these issues—such as by integrating additional centrality measures or refining the selection process for subsequent centers—represents a key opportunity for enhancing the adaptability and performance of the algorithm in diverse SDN environments.

6.2. Assessment of Possible Improvements

A critical aspect of controller placement in Software-Defined Networks is the selection of cluster centers, as the choice of these nodes directly impacts both network performance and resource utilization. Traditionally, the Advanced K-Means algorithm initiates this process by selecting the node with the highest degree as the initial center. While intuitive, this approach does not always capture the multifaceted importance of nodes within a network, especially when compared to broader centrality concepts.

6.2.1. Strategies for Initial Center Selection

The selection of the initial center is a critical step in clustering-based controller placement algorithms, as it can strongly influence the overall quality of subsequent cluster formation. This subsection presents a comparative analysis of various centrality measures—such as node degree, closeness, and betweenness—and discusses their potential impact on the choice of the initial controller location. By examining the theoretical foundations and practical implications of each metric, we aim to elucidate how different centrality-driven strategies can affect latency, coverage, and the overall effectiveness of controller deployment in SDN networks.

Node Degree

Node degree, defined as the number of direct connections a node has to other nodes in the graph, is the simplest and most local measure of centrality. Formally, the degree of node v_i in an undirected graph $G = (V, E)$ is given by:

$$D_i = |\{v_j \in V : (v_i, v_j) \in E\}|$$

Nodes with high degree are generally well-connected and often located in dense regions of the network, making them appealing candidates for controller placement, particularly in topologies where robustness and fault tolerance are prioritized. However, high degree does not necessarily equate to a strategic or central position within the network's global structure. For example, a node might have many local connections but be peripheral with respect to critical communication paths, thereby providing limited benefit in reducing overall propagation latency.

Closeness Centrality

Closeness centrality quantifies how near a node is to all other nodes in the network, emphasizing global reachability. It is defined as the inverse of the sum of the shortest path distances from node v_i to all other nodes:

$$C_{closeness}(v_i) = \frac{1}{\sum_{v_j \in V, j \neq i} d(v_i, v_j)}$$

A node with high closeness centrality can communicate rapidly with all other nodes, making it an ideal candidate for minimizing average latency when chosen as a controller location [27]. Nodes central in

this sense often facilitate efficient network management, as they can serve a large portion of the network with minimal delay.

Betweenness Centrality

Betweenness centrality, on the other hand, measures the frequency with which a node appears on the shortest paths between pairs of other nodes:

$$C_{betweenness}(v_i) = \sum_{s \neq v_i \neq t} \frac{\sigma_{st}(v_i)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from node s to node t, and $\sigma_{st}(v_i)$ is the number of those paths that pass through v_i [28]. High betweenness nodes are critical for the flow of information, acting as bridges or bottlenecks within the network. Selecting such nodes as controllers can be particularly advantageous in topologies with pronounced communication bottlenecks or hierarchical structures, as these nodes facilitate efficient routing and load distribution.

Interplay and Practical Implications

The relationships among degree, closeness, and betweenness centrality are nuanced. While a node with high degree may also have high closeness or betweenness, this is not guaranteed—especially in irregular or heterogeneous topologies. For example, in a star topology, the central hub has maximal degree, closeness, and betweenness, but in more complex graphs, these measures may diverge. Relying solely on degree for initial center selection can lead to suboptimal controller placement, particularly when nodes with moderate degree occupy more central positions in terms of network communication.

Empirical studies and theoretical analyses highlight that integrating multiple centrality metrics, potentially through a weighted selection strategy, can yield superior outcomes for controller placement [29, 30]. For instance, prioritizing nodes that are both well-connected (high degree) and centrally located (high closeness), or that facilitate key communication flows (high betweenness), can help ensure that the selected controllers are robust, efficient, and adaptable to network demands.

In summary, advancing the methodology beyond degree-centric heuristics to incorporate closeness and betweenness centrality offers a promising avenue for enhancing controller placement strategies. Such integrative approaches enable the identification of truly central and influential nodes, improving both latency and resilience in SDN deployments.

6.2.2. Strategies for Selecting Subsequent Centers

The procedure for selecting subsequent centers in the clustering process is critical for the effectiveness of controller placement in Software-Defined Networks. In the Advanced K-Means algorithm by Zobary, each next cluster center is chosen as the node that is maximally distant—according to the shortest path—from all already selected centers. The underlying intuition is that maximizing the spatial separation of controllers promotes broader coverage and, potentially, better load balancing.

However, this heuristic, while simple and computationally efficient, may exhibit significant limitations. Selecting only the single most distant node as the next center can cause the algorithm to overemphasize separation at the expense of other topological properties. In highly irregular or non-uniform topologies, such a policy may place controllers in areas with low node density or far from traffic hotspots, reducing the effectiveness of the placement in minimizing latency and balancing loads. This rigid approach is also sensitive to outliers and can fail to adapt to topological features that are crucial in real-world networks, such as clusters with high internal connectivity but moderate global distance to existing centers.

K-Means++ Clustering Innovation

A more sophisticated alternative—widely used in data clustering—is the k-means++ initialization scheme [31]. In k-means++, the first center is selected randomly, while each subsequent center is chosen from the remaining nodes with a probability proportional to the square of its shortest distance to the closest already-selected center:

$$P(v_i) = \frac{d(v_i, C)^2}{\sum_{v_j \notin C} d(v_j, C)^2}$$

where $d(v_i, C)$ is the shortest distance from node v_i to the set of already chosen centers C. This strategy means that nodes far from existing centers are more likely to be picked, but the process remains probabilistic, allowing for a diversity of configurations and reducing the risk of pathological placements.

The k-means++ approach has several proven theoretical advantages:

- It provides an $O(\log K)$-approximation guarantee with respect to the optimal clustering cost [31], which is significantly better than purely random or purely deterministic "farthest" selection.
- The probability-weighted selection ensures that centers are well-dispersed, but not in a strictly deterministic way—so the algorithm is more robust to atypical network structures, outliers, and non-uniform node distributions.
- Empirical studies, including applications in SDN controller placement [32], show that k-means++ consistently results in lower average latency and better load balancing compared to the farthest-node strategy, especially as network size and heterogeneity increase.

Practical Implications for Controller Placement

Adopting the k-means++ strategy for selecting controller locations in SDN can mitigate the weaknesses of the pure "maximum dispersion" heuristic. The probabilistic selection mechanism allows the algorithm to better adapt to local traffic concentrations and nuanced topological features, without being overly sensitive to a single outlying node. In effect, k-means++ balances the need for center separation with the need to serve dense or high-demand regions efficiently.

In practice, this leads to more flexible and adaptive controller placements that are less likely to result in pathological cases—such as controllers placed in sparsely connected or peripheral regions

solely for the sake of maximizing distance. The diversity introduced by the probabilistic selection also provides an opportunity for repeated runs and consensus-based selection, further improving robustness and the quality of solutions [32].

A key strength of the k-means++ approach, as demonstrated in the work of Jacob et al. [32], is its ability to achieve high-quality clustering outcomes, which can be quantitatively assessed by the silhouette score. The silhouette coefficient measures how similar an object is to its own cluster compared to other clusters, with values closer to 1 indicating well-separated and cohesive clusters. In their experiments on a variety of real-world topologies from the Internet Zoo dataset, k-means++ consistently achieved higher silhouette scores than alternative clustering methods such as spectral clustering or Partitioning Around Medoids (PAM), especially for small- and medium-scale networks. This indicates that the clusters formed by k-means++ are both internally compact and externally well-separated, which directly translates to effective partitioning of the network for controller placement.

Beyond statistical clustering quality, the method also exhibits several practical advantages in SDN controller placement. First, the flexibility in center selection inherent to k-means++ enables the algorithm to better accommodate the actual communication demands of the network, thereby supporting lower average delay and jitter, as reported in the performance evaluations. This is particularly valuable in heterogeneous networks, where traditional heuristics might either neglect dense subregions or over-prioritize network peripheries. Second, the approach scales well with network size, as the probabilistic nature of the selection step reduces the likelihood of convergence to poor local optima and allows for efficient adaptation to varying topological conditions.

Moreover, the findings of Jacob et al. [32] highlight that k-means++ combined with an effective controller placement refinement—such as the meeting point algorithm—outperforms other methods not only in clustering cohesion but also in end-to-end network performance metrics. This synergy results in improved throughput, lower average latency, and more stable quality of service parameters under dynamic or escalating traffic conditions.

In summary, the k-means++ clustering method, by virtue of its probabilistic, distance-sensitive selection of cluster centers, delivers superior clustering structure as validated by silhouette score analysis, enables better alignment with real network demands, and supports enhanced SDN performance on practical metrics such as delay and throughput. Its robustness, scalability, and adaptability make it a preferred foundation for modern controller placement strategies in Software-Defined Networking.

In summary, while the "farthest node" strategy offers simplicity and guarantees dispersion, it may be too rigid for heterogeneous and real-world network topologies. The k-means++ method, with its probabilistic, distance-based selection, is theoretically and practically superior for controller placement in SDN, offering both improved performance guarantees and greater adaptability to network demands.

6.3. Direction of Enhancements

Building upon the analyses and observations presented in this chapter, the primary direction for enhancing the Advanced K-Means algorithm involves the integration of greater flexibility and adaptivity in the selection of cluster centers. To this end, the first step will be to introduce a weighted decision system for the initial center selection, wherein node degree and various centrality measures (such as closeness and betweenness) are combined according to manually adjustable weights. This mechanism will enable fine-grained control over the influence of each metric, allowing the algorithm to be tailored to the specific requirements or structural properties of different SDN topologies. By transitioning from a strictly degree-based or tie-breaker approach to a controlled, multi-metric framework, the algorithm's capacity for context-aware initialization is expected to be substantially improved.

Furthermore, the current heuristic for selecting subsequent centers—namely, always choosing the node maximally distant from existing centers—will be replaced with a probabilistic selection mechanism inspired by the k-means++ algorithm. In this approach, each candidate node's probability of being selected as the next center is made proportional to the square of its shortest-path distance from the set of already-chosen centers. This method introduces an element of controlled randomness, helping to avoid deterministic pitfalls such as the consistent selection of peripheral or outlier nodes and better accommodating the diversity and irregularity of real-world network structures.

An important aspect of this enhancement strategy will be the systematic investigation of how the proposed modifications compare to the original Advanced K-Means approach. Experimental results will be carefully analyzed to assess whether the modified algorithm delivers measurable improvements in latency reduction, load balancing, or robustness. However, even in cases where performance gains are not observed, the comparative study itself offers substantial research value. By evaluating the influence of different center selection strategies, this work aims to deepen understanding of the relationship between controller localization methods and request handling latency—ultimately contributing new insights and potential pathways for future advancements in SDN controller placement.

This approach, therefore, not only targets algorithmic improvement but also provides a structured methodology for exploring and quantifying the impact of placement strategies on key network performance indicators, solidifying the research's relevance to both theoretical development and practical deployment.

7. OWN CONTRIBUTIONS

This part of the study presents the original contributions of this thesis to the field of SDN controller placement, with a particular focus on improving the Advanced K-Means algorithm. Building upon the theoretical analysis and identified limitations from previous chapters, a set of novel enhancements has been designed, implemented, and experimentally validated. The primary aim of these improvements is to increase the flexibility and effectiveness of controller localization in minimizing request handling latency.

The following sections provide a comprehensive overview of the experimental research environment, including the selection of realistic network topologies and the development of a dedicated simulation framework. Subsequently, the realized algorithmic enhancements are detailed: the introduction of weighted metrics for initial centroid selection, as well as the adoption of a probabilistic, k-means++-inspired approach for selecting subsequent cluster centers.

A series of experiments was conducted to rigorously evaluate the impact of the proposed modifications on key performance indicators, including average latency and controller load balancing. The results of these experiments are systematically compared with the original Advanced K-Means method and other baseline approaches discussed earlier in the thesis. Through this process, the research not only validates the utility of the enhancements but also offers new insights into the relationship between controller placement strategy and request handling performance in Software-Defined Networks.

7.1. Realized Enhancements

This section details the key algorithmic improvements developed as part of this research to advance the state-of-the-art in SDN controller placement. The proposed enhancements build upon the foundational Advanced K-Means approach by introducing increased flexibility and adaptability in the selection of controller locations. Special emphasis is placed on the integration of centrality-weighted strategies for the initial centroid selection and the adoption of a probabilistic, distance-based rule for subsequent center selection. Through these modifications, the **Enhanced K-Means++** algorithm aims to achieve lower propagation latency and improved load balancing across a wide spectrum of network topologies. The following subsections describe the design principles, implementation details, and rationale behind each enhancement.

7.1.1. Centroid Selection Weights

The first key enhancement introduced in the *Enhanced K-Means++* algorithm involves the method for selecting the initial cluster center. In the original `Advanced K-Means` algorithm, this is performed by the `best_initial_center(G)` function:

```python
def best_initial_center(G):
    degrees = dict(G.degree())
    avg_degree = compute_node_average_degree(G)
    candidates = [n for n in nodes if satisfies_degree(n, degrees, avg_degree)]
    max_deg = max(degrees[n] for n in candidates)
    # In case of tie, use sum of shortest paths
    ...
```

Listing 7.1: Advanced K-Means: Initial Center Based on High Degree

Here, the algorithm deterministically selects the node with the highest degree (subject to an average degree constraint). In the event of a tie (multiple nodes with the same maximal degree), the node with the lowest total shortest-path distance to all other nodes is chosen as a tie-breaker. This prioritizes local connectivity but does not always select globally optimal nodes for controller placement, especially in topologies where degree centrality does not fully capture strategic network positions.

The *Enhanced K-Means++* variant generalizes this logic by integrating multiple centrality metrics—degree, betweenness, and closeness—using a tunable, weighted combination. The relevant code is as follows:

```python
def best_weighted_initial_center(
    G, betweenness, closeness,
    w_degree, w_betweenness, w_closeness
):
    ...
    for n in candidates:
        d_norm, b_norm, c_norm = normalize_metrics(n, degrees, betweenness,
            closeness)
        score = w_degree * d_norm + w_betweenness * b_norm + w_closeness * c_norm
        ...
```

Listing 7.2: Enhanced K-Means++: Weighted Initial Center

In this approach, each candidate node is scored based on normalized degree, betweenness, and closeness centrality, with the weights `w_degree`, `w_betweenness`, and `w_closeness` allowing manual control over the influence of each metric. The node with the highest composite score is selected as the initial center, with the sum of shortest-path distances serving as a tie-breaker in the event of equal scores. This strategy offers much greater flexibility and enables the algorithm to better adapt the center selection policy to the unique characteristics of the underlying network.

7.1.2. Probabilistic Subsequent Center Selection

The second major enhancement targets the process of choosing subsequent cluster centers. The standard `Advanced K-Means` uses a deterministic "maximum dispersion" strategy, implemented in `select_farthest_node`:

```python
def select_farthest_node(nodes, centers, degrees, avg_degree, path_lengths):
    farthest_node = None
    max_min_dist = -1
    for n in nodes:
        if n in centers or not satisfies_degree(n, degrees, avg_degree):
            continue
        min_dist = min(path_lengths[n][c] for c in centers)
        if min_dist > max_min_dist:
            max_min_dist = min_dist
            farthest_node = n
    return farthest_node
```

Listing 7.3: Advanced K-Means: Farthest Subsequent Node Selection

Here, the next center is always chosen as the node with the largest minimum distance to any of the existing centers, provided it satisfies the degree constraint. This ensures spatial separation of controllers but is inherently rigid and may not align with network demand or topological diversity.

The *Enhanced K-Means++* algorithm replaces this with a stochastic, probabilistic selection inspired by the k-means++ approach, using `select_stochastic_next_center`:

```python
def select_stochastic_next_center(G, centers, rng):
    ...
    for n in candidates:
        min_dist = min(path_lengths[n][c] for c in centers)
        dists.append(min_dist ** 2)
    total = sum(dists)
    if total == 0:
        return rng.choice(candidates)
    chosen_idx = rng.choices(range(len(candidates)), weights=dists, k=1)[0]
    return candidates[chosen_idx]
```

Listing 7.4: Enhanced K-Means++: Probabilistic Center Selection

This function assigns a probability to each candidate node proportional to the square of its minimum distance to the current centers, then samples the next center according to this distribution. This probabilistic selection mechanism increases the algorithm's ability to avoid local optima and pathological placements (such as consistently choosing outlier nodes or missing clusters of demand), and is more robust to variations in network structure. As a result, controller locations are chosen in a way that balances spatial distribution with adaptability to the unique characteristics and requirements of the network.

Overall, these enhancements—weighted multi-metric selection for the initial center and probabilistic selection for subsequent centers—make the *Enhanced K-Means++* algorithm more flexible, adaptive, and effective for minimizing latency and improving controller load balancing in realistic SDN environments.

7.2. Research Environment

This section provides an overview of the experimental environment designed for the evaluation of proposed enhancements to the controller placement algorithms. It describes the selection and characteristics of network topology datasets used as test cases, details the implementation of the simulation platform and algorithms, and outlines the integration of supporting scripts that automate experimental runs. Following parts are devoted to the procedures for collecting performance results and the subsequent generation of charts and output files. This ensured that all experiments are conducted in a reproducible and consistent manner.

7.2.1. Topology Test Data

The evaluation of controller placement algorithms was conducted on a diverse set of real-world network topologies, each characterized by different size, connectivity, and geographical spread. Almost all[1] topologies are sourced from the Internet Topology Zoo [24] [34] repository and are commonly used in SDN research. Below, each network is briefly described.

Atmnet

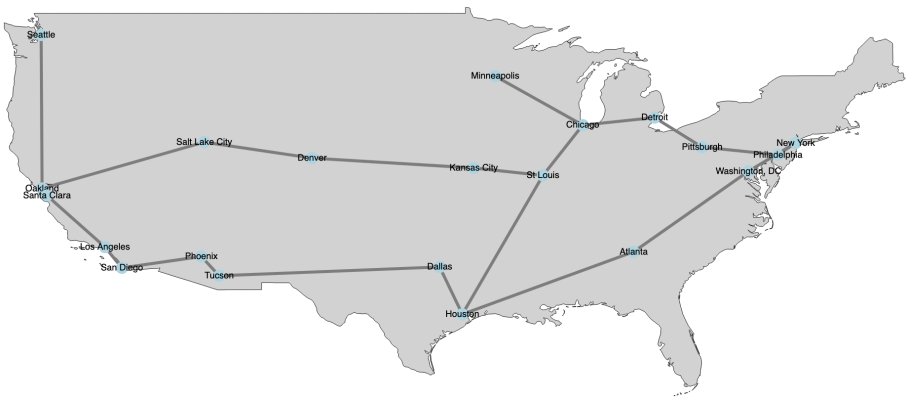


Fig. 7.1. Atmnet Network Topology (USA)

The Atmnet topology consists of 21 nodes and 22 edges, representing a larger-scale network deployed across the United States. Its interconnection structure is relatively sparse, with most nodes linked by long-haul connections rather than forming dense clusters. All nodes are annotated with geographic coordinates, allowing for spatial analysis. The network covers a wide area—spanning approximately 17.85 degrees latitude and 48.32 degrees longitude—which reflects considerable geographical diversity but limited node density.

[1] Internet2 OS3E topology data was sourced from [33] and converted into a GML file format for use in the simulation experiments.

Internet2 OS3E

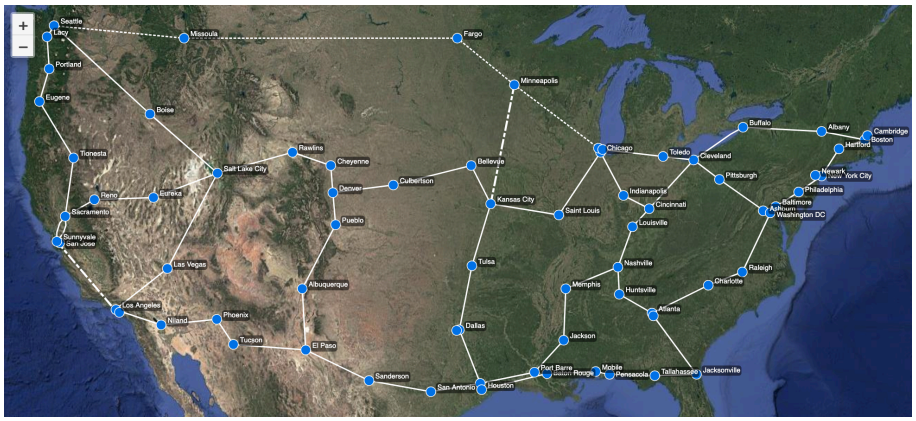


Fig. 7.2. Internet2 Network Topology (USA)

The Internet2 OS3E topology is a large, wide-area research network backbone deployed across the United States, composed of 34 nodes and 42 edges—making it notably larger and denser than the Atmnet topology. All nodes include latitude and longitude information, with the network spanning about 23.53 degrees latitude and 52.06 degrees longitude. Unlike Atmnet, Internet2 OS3E features regions of higher node concentration, resulting in locally dense clusters within the broader, geographically dispersed network. This structure makes it particularly suitable for evaluating latency-aware controller placement in realistic and heterogeneous SDN environments. Figure 7.2 was sourced from the official Internet2 infrastructure map [35].

Geant2012



Fig. 7.3. Geant 2012 (Europe)

Geant2012 is a large-scale European backbone topology, composed of 37 nodes and 58 edges. The network features a dense mesh structure with most nodes corresponding to major national or regional hubs, and a high concentration of connections in central and western Europe. Nodes are geographically distributed across nearly all of Europe—from Portugal and Iceland to Russia and Israel—covering 32.64 degrees of latitude and 59.52 degrees of longitude. The topology is characterized by both tightly connected core regions and several long-haul links that bridge peripheral nodes, resulting in a robust and heterogeneous backbone suited for evaluating controller placement strategies in geographically diverse networks.

IIJ



Fig. 7.4. IIJ (USA, Japan)

The IIJ (Internet Initiative Japan) topology comprises 28 nodes and 54 edges, forming a medium-scale backbone network with notably dense local interconnection. A distinguishing feature of this topology is its pronounced irregularity: while most nodes are clustered in relatively compact geographic regions, the network also contains a few extremely long-range links that connect distant parts of the topology. These long-haul connections effectively bridge two geographically separated "clusters" of nodes, resulting in a dual-cluster structure joined by intercontinental or interregional links. This unique organization introduces both densely connected local areas and sparsely bridged remote regions, making IIJ a particularly challenging and realistic scenario for evaluating controller placement algorithms in the presence of spatial and structural heterogeneity.

PionierL3

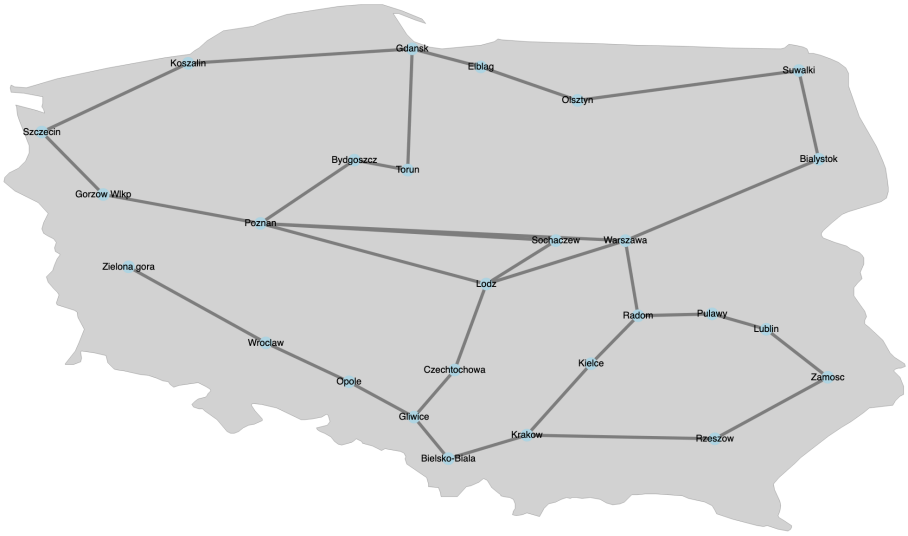


Fig. 7.5. Pionier L3 (Poland)

Pionier L3 is a regional backbone topology located in Poland with 27 nodes and 32 edges. All nodes are geocoded, and the network extends over 4.53 degrees latitude and 8.7 degrees longitude, representing a more localized deployment with moderate connectivity. This makes it suitable for experiments focusing on controller placement in regional SDN deployments.

Belnet2003

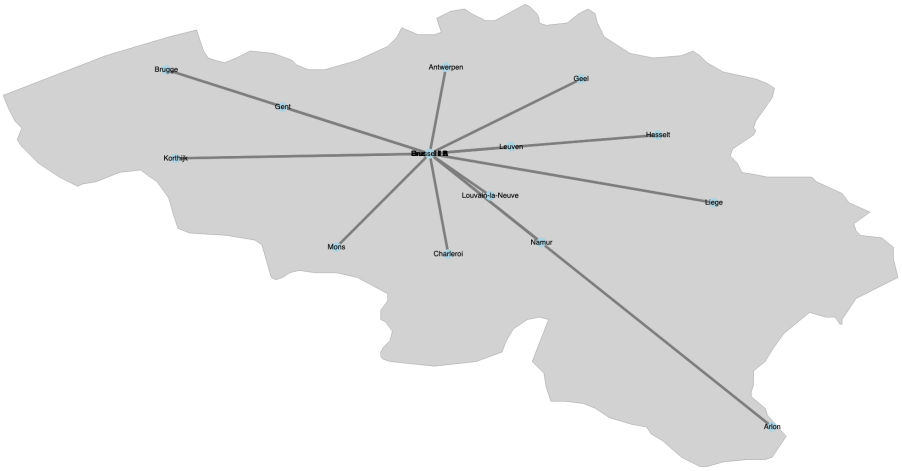


Fig. 7.6. Belnet 2003 (Belgium)

The Belnet2003 topology models the Belgian national research and education network. It consists of 17 nodes and 32 edges, with all major academic and research centers in Belgium directly connected to Brussels as the central hub. The topology exhibits a pronounced star-like structure, resulting in a very short path length between most nodes, which can minimize average latency but may

lead to central node congestion or a single point of failure. The network's small size, limited average degree (3.76), and compact geographical range make it well-suited for experiments focused on the effect of core-centric controller placement and its impact on network resilience and delay.

Bellsouth

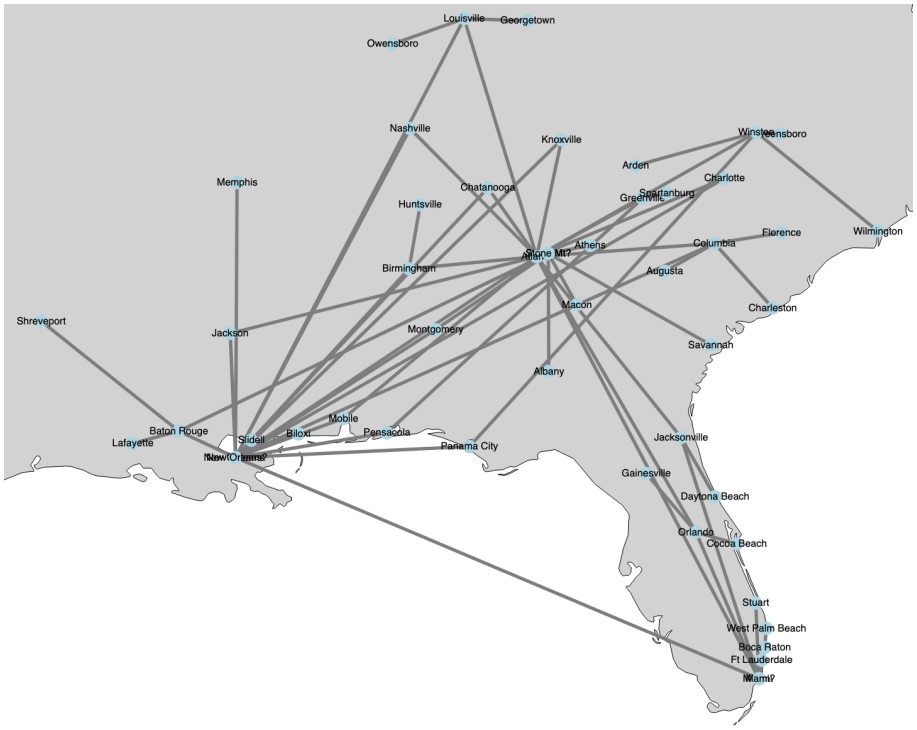


Fig. 7.7. Bellsouth (USA, Southeast)

Bellsouth is a large-scale, regional network deployed in the southeastern United States, featuring 50 nodes and 64 edges. Its topology is highly irregular, with a mixture of dense local clusters and long-haul links, and a maximum degree of 17. The network is geographically dispersed, with nodes spanning across multiple states. The average degree (2.56) indicates moderate connectivity overall, but the substantial graph diameter (by link lengths) and long average link lengths (over 350 km) highlight the presence of extensive long-range connections. This structure makes Bellsouth a valuable testbed for analyzing the trade-offs between geographic distance, latency, and controller placement in heterogeneous and spatially expansive networks.

Cesnet 1993

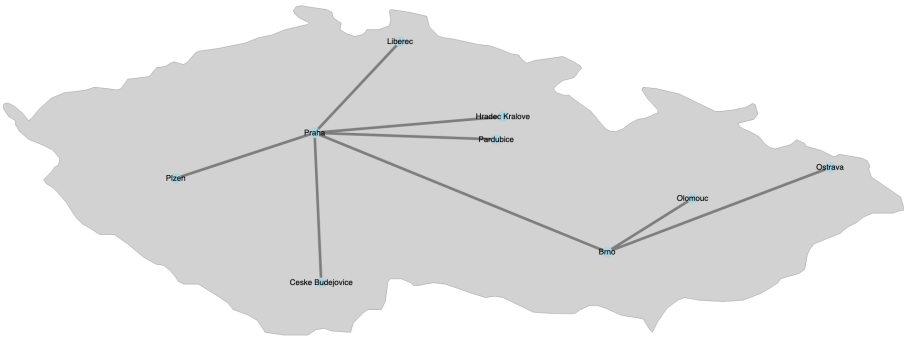


Fig. 7.8. Cesnet 1993 (Czech Republic)

Cesnet 1993 represents the backbone of the Czech national research network at the start of the 1990s. The network comprises 9 nodes and 8 edges, forming a small and very sparse topology with an average degree of just 1.78. Most nodes are directly connected to Prague, which acts as the network's central point. The topology is highly centralized, with a short graph diameter and minimal average hop count between nodes. This structure, while effective for minimizing latency in small national networks, also highlights vulnerability to failures of central nodes and makes it suitable for exploring the effects of hub-based controller placement in simple, regional environments.

Table 7.1: Summary of Test Topologies Used in Experiments

Topology	Nodes	Edges	Lat. Span	Lon. Span	Distinctive Features
Atmnet	21	22	17.85°	48.32°	US backbone; sparse, mostly long-haul links, wide geographic spread
Internet2 OS3E	34	42	23.53°	52.06°	US research backbone; large, denser, with local clusters and broader spread
Geant2012	37	58	32.64°	59.52°	European backbone; very dense core, major regional hubs, some long-haul links
IIJ	28	54	16.72°	263.49°	Dense local clusters with a few very long links connecting two main regions (USA-Japan)
Pionier L3	27	32	4.53°	8.70°	Regional Polish backbone; compact, moderate connectivity
Belnet2003	17	32	1.97°	5.13°	Belgium national network; star-like, short paths, centralized on Brussels
Bellsouth	50	64	12°	20°	Large, irregular, many long-haul links, moderate avg. degree; US (Southeast)
Cesnet1993	9	8	4.41°	4.41°	Czech research network; very small and sparse, highly centralized on Prague

These varied topologies provide a comprehensive testbed for benchmarking controller placement methods, capturing a range of network sizes and geographic distributions relevant to real-world SDN applications.

7.2.2. Simulator

The simulation software was developed in Python version 3.13 [36] and executed on a macOS Sequoia 15.5 platform. The hardware environment consisted of an Apple MacBook Pro equipped with an M4 Pro processor and 24 GB RAM. The principal tool for manipulating network topologies was the NetworkX library [37], which provides comprehensive data structures and algorithms for the creation, manipulation, and analysis of complex networks. NetworkX was used to import and represent topological resources described in the Graph Modelling Language (GML) format [38], mapping them onto Python's built-in graph data structures.

Additional Python libraries were utilized to support the simulation process:

- **random** [39] for generating pseudo-random numbers and selections required in probabilistic algorithms and initializations,
- **numpy** [40] for efficient numerical operations and vectorized computations,
- **Matplotlib** [41] for the generation of line charts and visualization of experimental results.

Experimental outcomes were visualized as line plots, and detailed measurement results were saved in JSON files. The use of JSON as a storage format facilitates flexible post-processing, enabling further data analysis with dedicated programs or the creation of custom scripts for additional result interpretation.

7.2.3. Project Structure

The repository of the project is organized with a modular architecture that facilitates systematic development and evaluation of controller placement algorithms. Its organization reflects a clear separation between algorithmic modules, experiment orchestration, data processing, and result visualization. The key components of the implementation are described below:

- **Algorithms**
  - `advanced_k_means.py` – implementation of the basic *Advanced K-Means* algorithm. The function `advanced_k_means()` iteratively selects controller centers and performs a local K-Means cycle (node assignment and center update) to reduce propagation delay.
  - `enhanced_k_means.py` – an original *Enhanced K-Means++* variant based on the advanced approach. It uses normalized centrality metrics and probabilistic center selection.
  - `helpers.py` – auxiliary routines for graph processing, including path-length computation, node assignment, center updates and node-selection strategies.
- **Experiments**. The `experiments` directory provides `experiments_runner.py`, which runs comparative tests and stores plots per topology under `plots/`.
- **Utilities**. The `utils` package contains:
  - `plot_utils.py` – functions for producing latency plots, saved in `plots/<topology>`.
  - `load_utils.py` – computation and storage of controller loads in `load/<topology>`.
  - `results_utils.py` – export of experiment results to JSON files in `results/<topology>`.
- **Input data**. The `topologies` folder contains GML files describing networks; `data_utils.py` loads them into graphs weighted by delay.
- **Main script**. `main.py` enumerates available topologies and launches the experiments using the above modules.

7.3. Experiments

A comprehensive series of experiments was conducted using the described research environment, test data, and algorithms, including both baseline and enhanced methods. Eight distinct topologies

were evaluated, each characterized by different structural properties and representing a range of real-world network environments. As previously discussed, these topologies capture a diversity of scale, connectivity, and spatial distribution, thereby reflecting network scenarios where latency requirements and constraints may differ substantially.

The experimental study was structured in two main series:

1. In the first series, for values of k ranging from 1 to 10, comparative experiments were performed. Results from both algorithms were visualized on plots as a function of k, the number of controllers in the network. The Y-axis represents, respectively, the average delay and the maximum delay—each reported on separate charts.
2. In the second series, also for k from 1 to 10, comparative experiments were repeated, but for Enhanced K-Means++ each value of k involved 10 independent runs. Due to the algorithm's stochastic nature, this allowed the computation of mean results and standard deviations for each k. While in the first series values were purely randomized, the second series provides a more detailed statistical characterization of the algorithm's performance variability.

For the Enhanced K-Means++ algorithm, the experiments arbitrarily adopted the following weights for the initial center selection: $w_{\text{degree}} = 0.2$, $w_{\text{betweenness}} = 0.4$, and $w_{\text{closeness}} = 0.4$. This configuration was chosen to distribute the influence among centrality measures and to investigate whether reducing the importance of node degree—relative to betweenness and closeness centrality—would yield different or potentially improved results.

7.3.1. Results

Atmnet

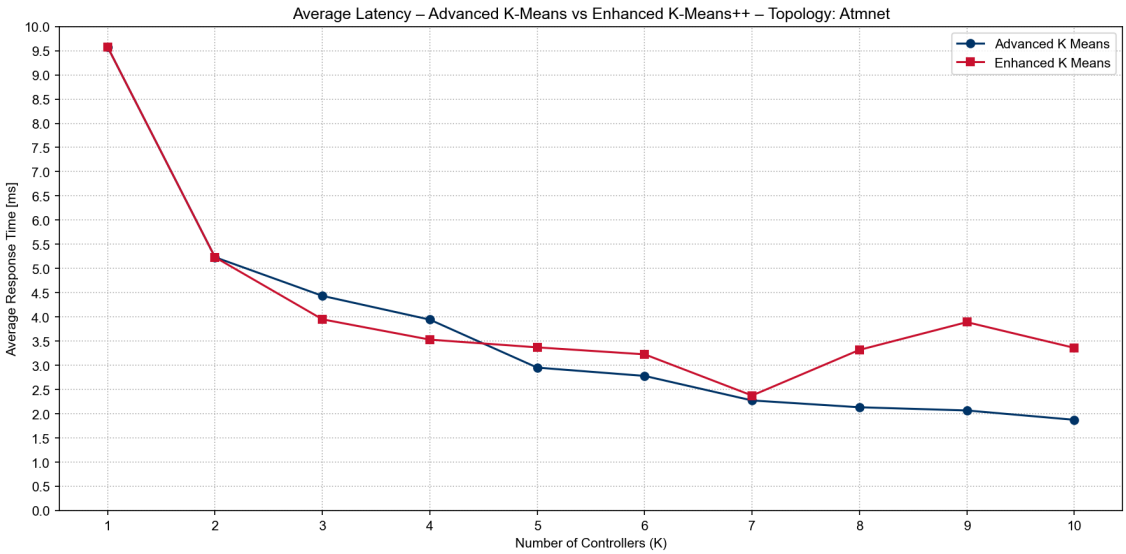


Fig. 7.9. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology.

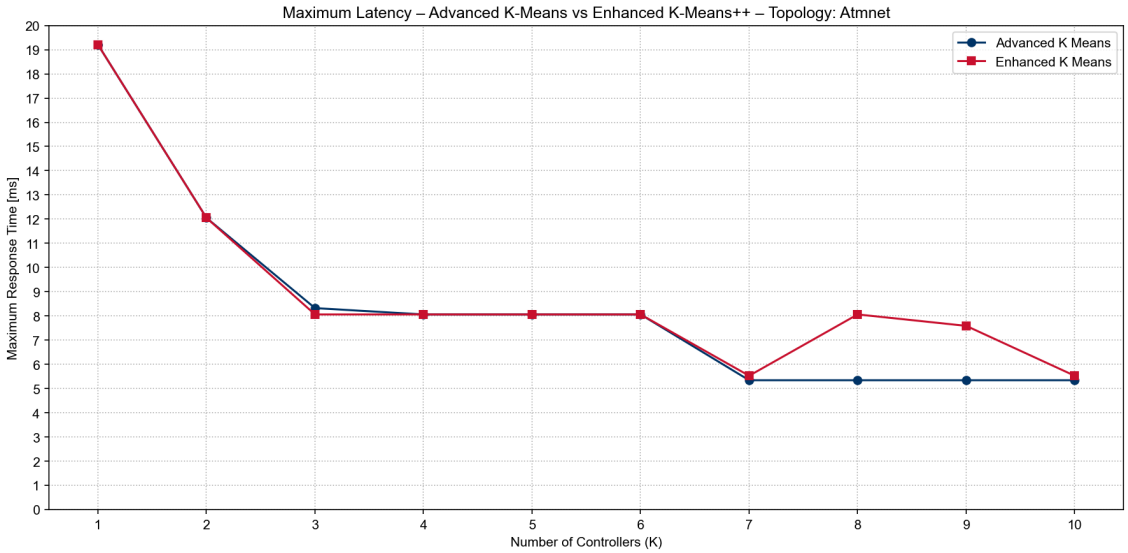


Fig. 7.10. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology.

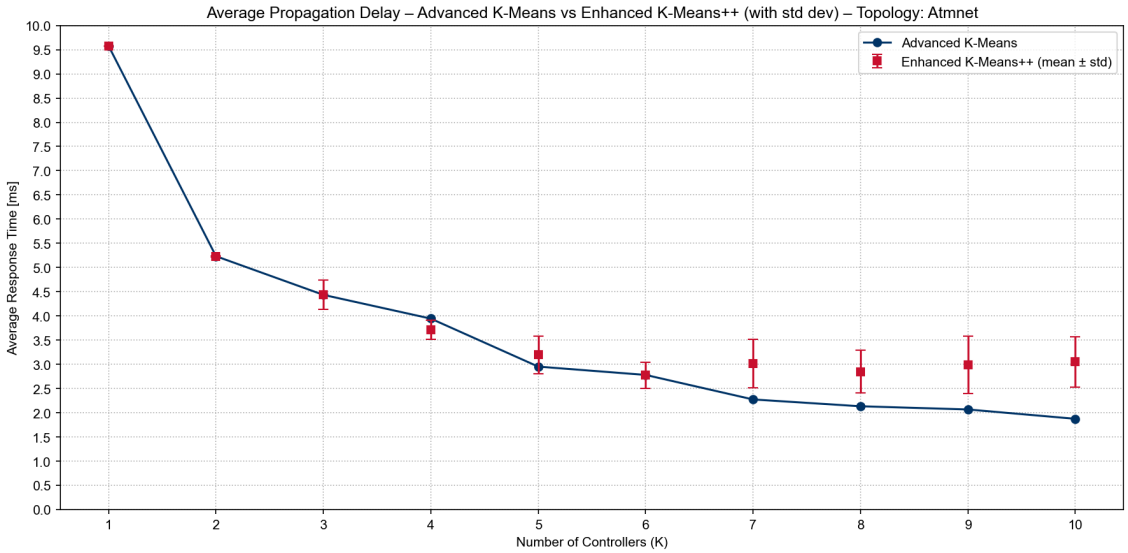


Fig. 7.11. Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology.

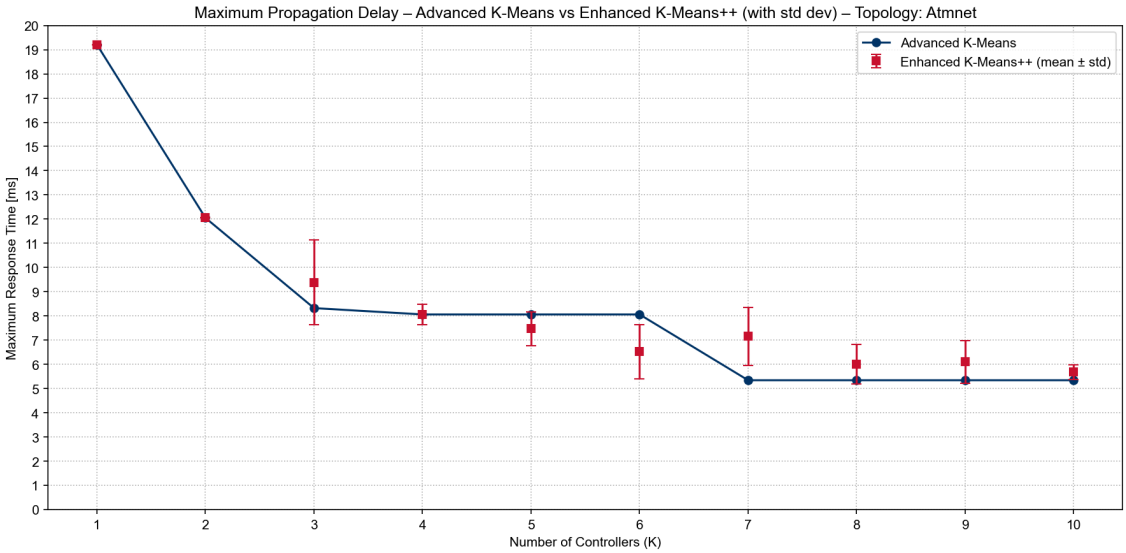


Fig. 7.12. Maximum Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology.

The comparative results for the Atmnet topology reveal several key patterns:

- Both Advanced K-Means and Enhanced K-Means++ exhibit a systematic reduction in average and maximum latency as the number of controllers k increases, with the largest improvements observed for small values of k.
- For $k = 3$ and $k = 4$, Enhanced K-Means++ achieves lower average latency than Advanced K-Means (roughly 0.5 ms), indicating that the probabilistic center selection and centrality weighting can produce more favorable placements in certain configurations.
- For higher values of k (particularly $k \geq 7$), the results begin to diverge: Advanced K-Means continues to achieve slightly lower average latency, while Enhanced K-Means++ stabilizes or exhibits minor increases in average latency relative to the deterministic variant.
- The standard deviation of results for Enhanced K-Means++ remains low across all tested values of k, confirming the robustness of its stochastic center selection in this topology.
- Both algorithms deliver nearly identical results for $k = 1$ and $k = 2$, as expected due to limited placement options in a small network.

Internet2 OS3E

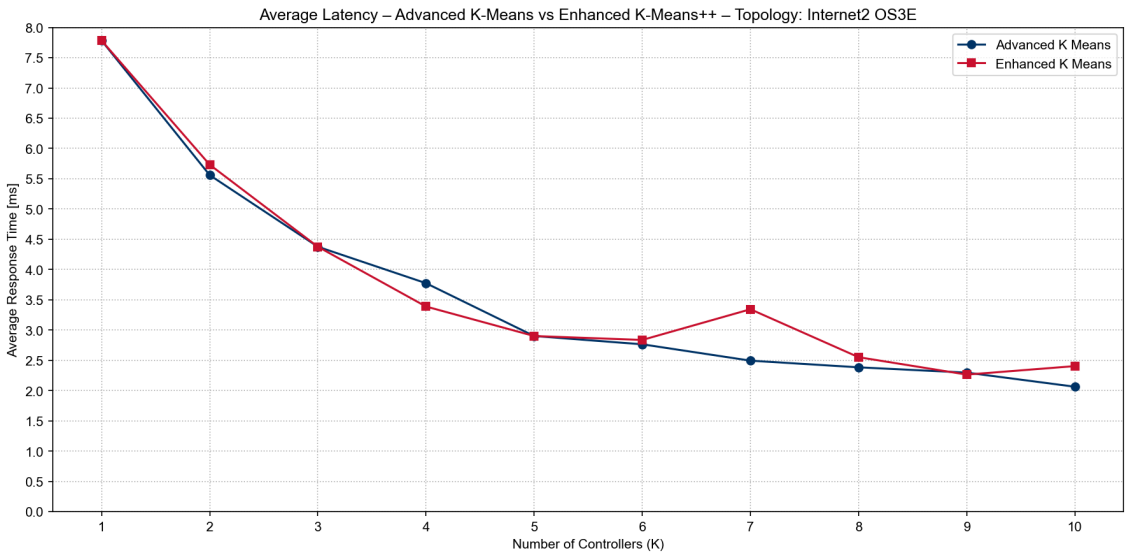


Fig. 7.13. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology.

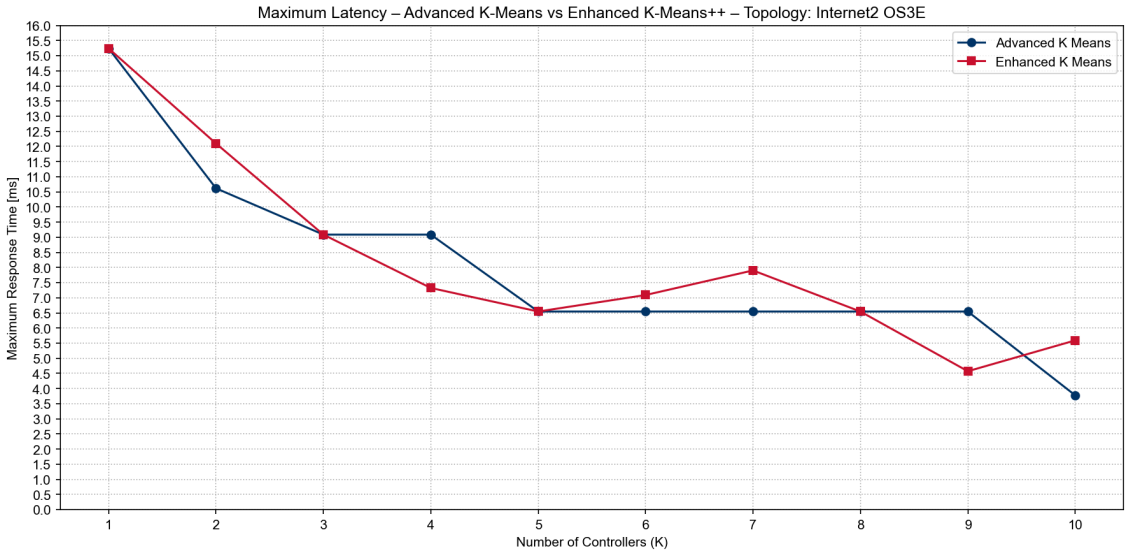


Fig. 7.14. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology.

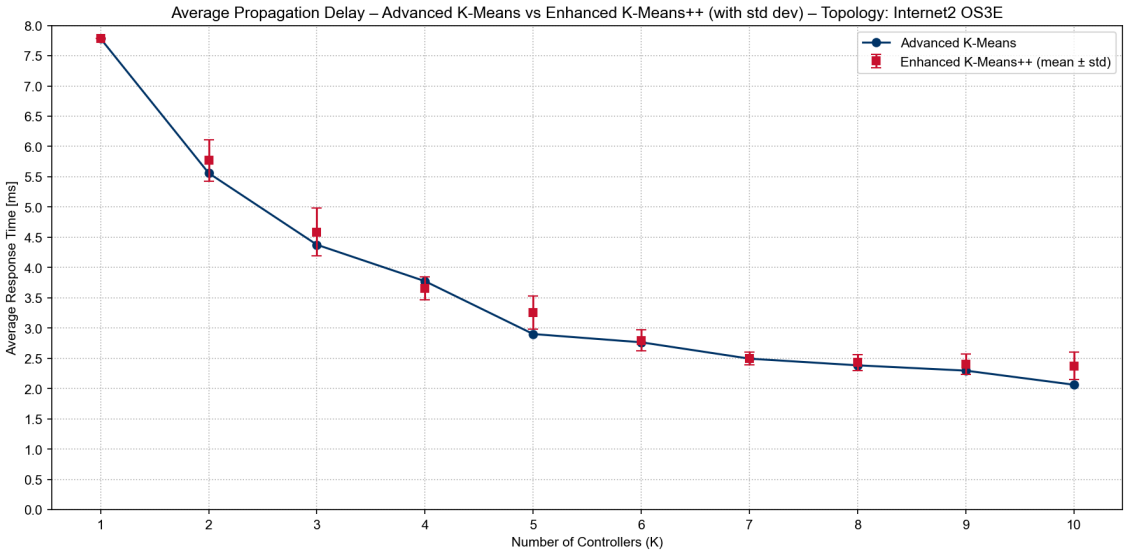


Fig. 7.15. Average Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology.

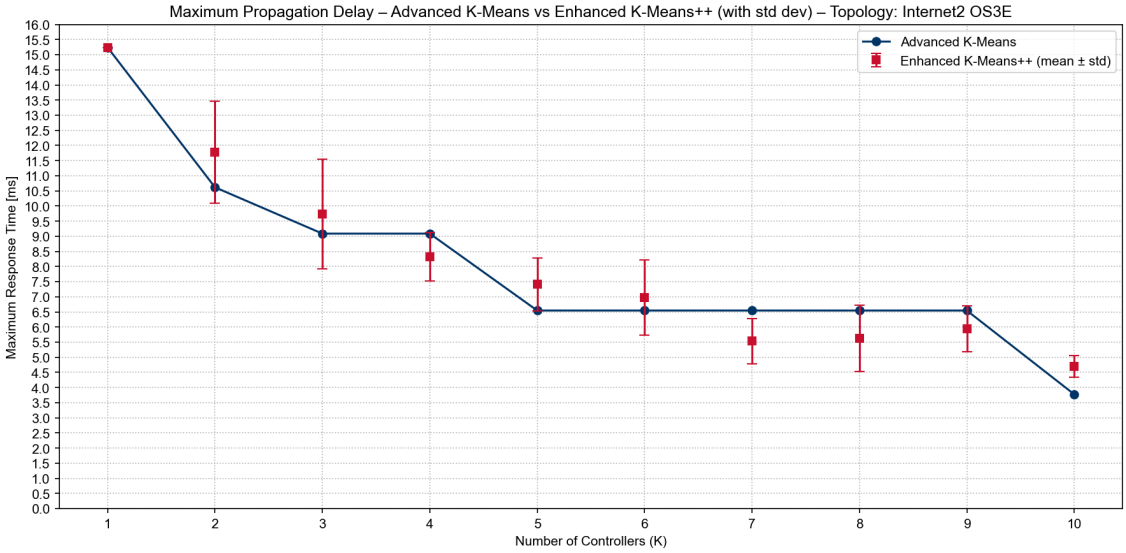


Fig. 7.16. Maximum Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology.

The experimental results for the Internet2 OS3E topology reveal several noteworthy observations:

- Both Advanced K-Means and Enhanced K-Means++ consistently reduce average and maximum latency as the number of controllers k increases, with the most pronounced improvement occurring for small values of k.
- For larger k (particularly $k \geq 7$), Advanced K-Means tends to outperform Enhanced K-Means++ in terms of average latency, with the enhanced method displaying a tendency to plateau or slightly increase in latency relative to the baseline method.
- In contrast to the Atmnet topology, the maximum latency for Enhanced K-Means++ in Internet2 OS3E exhibits more pronounced variability, particularly for larger k, as shown by the increased

standard deviation and higher fluctuations in the measured values. This suggests that the stochastic nature of center selection can lead to a wider spread of outcomes in more complex or denser topologies.

- Importantly, when comparing Atmnet and Internet2 OS3E—both representing similar geographic regions but with differing network scale and density—Enhanced K-Means++ exhibits slightly less consistent performance in the larger topology, as evidenced by larger standard deviations and occasional increases in both average and maximum latency. This highlights a potential sensitivity of the enhanced method to increased network size and complexity.

Geant 2012

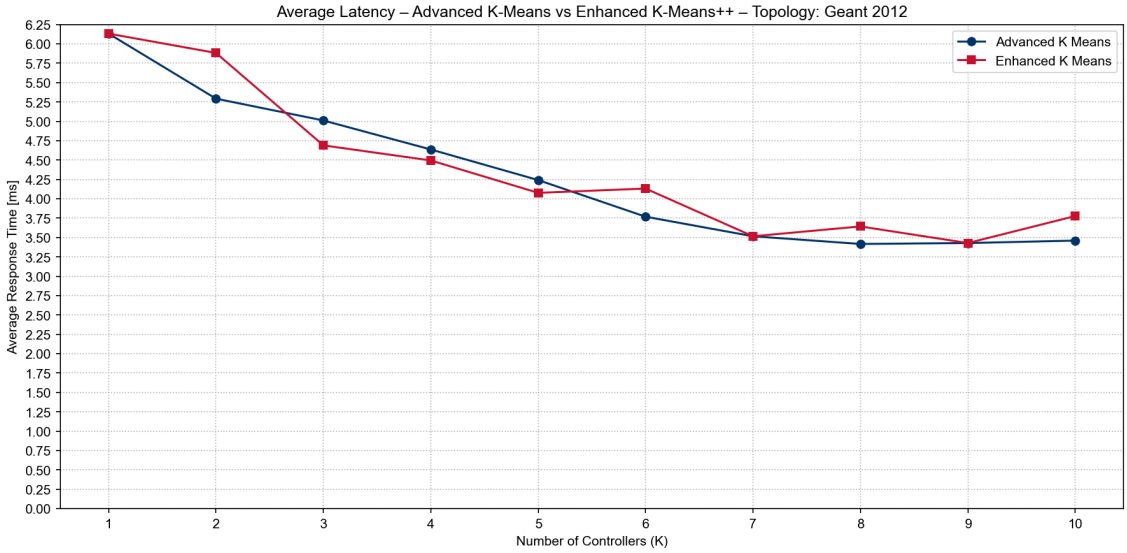


Fig. 7.17. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology.

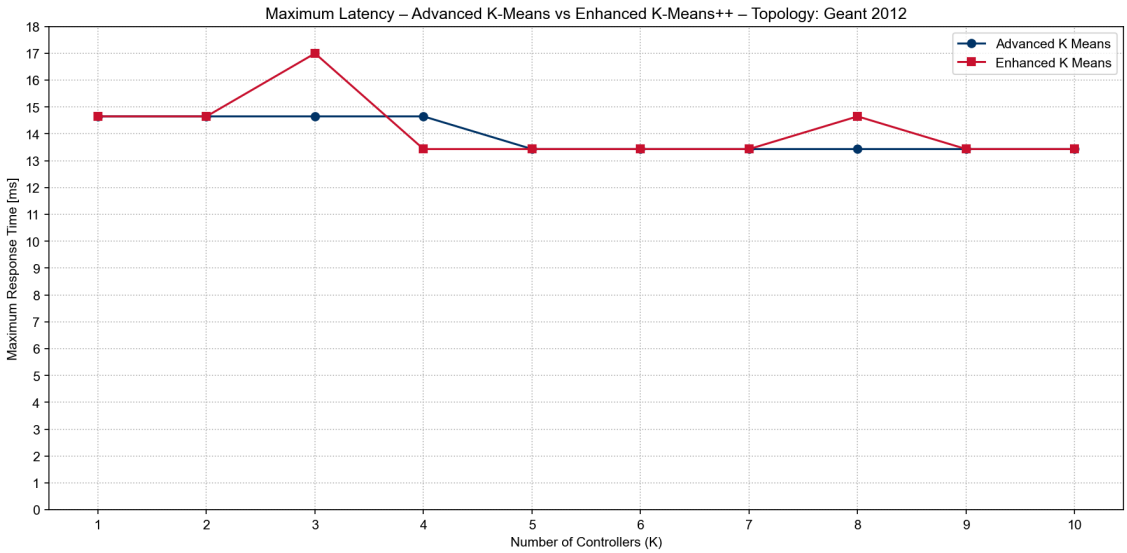


Fig. 7.18. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology.

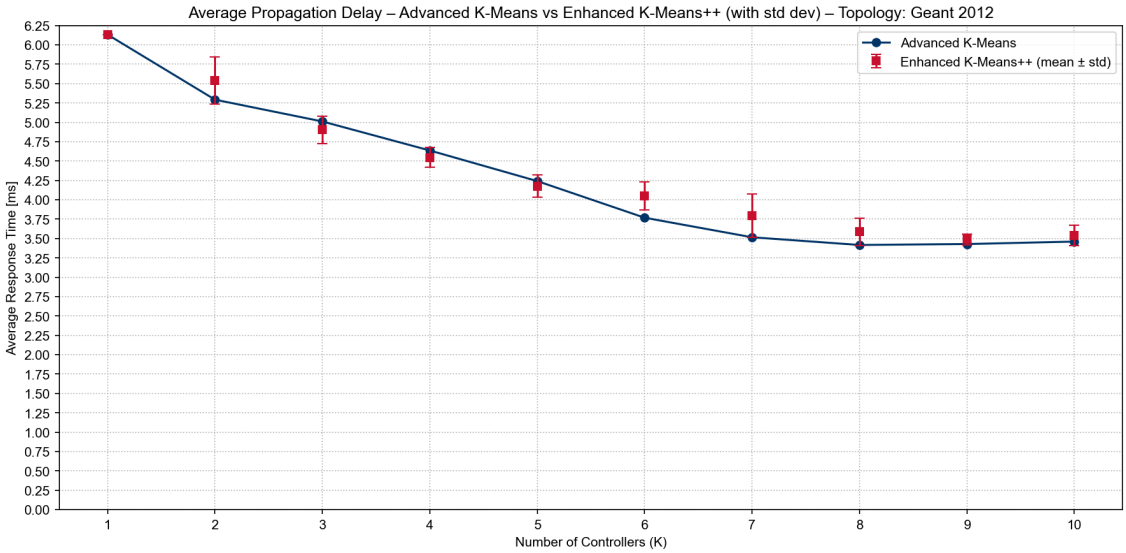


Fig. 7.19. Average Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology.

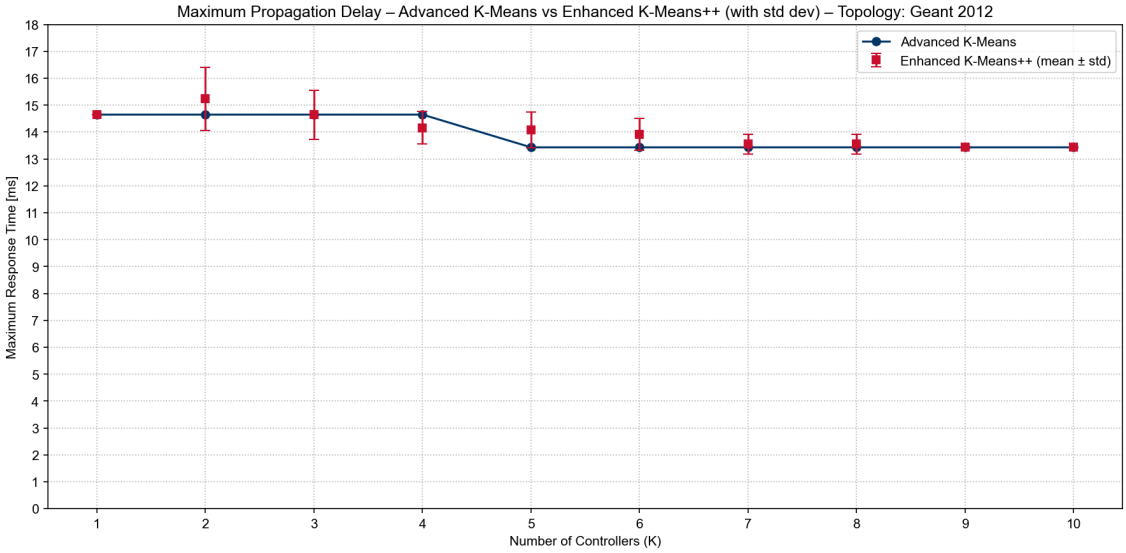


Fig. 7.20. Maximum Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology.

For the Geant2012 topology, several distinct trends emerge in the comparative results between Advanced K-Means and Enhanced K-Means++:

- For $k = 3, 4$, and even $k = 5$, the Enhanced K-Means++ algorithm achieves slightly lower average response times than Advanced K-Means, indicating a potential advantage of the stochastic selection in moderately partitioned network states. This effect is consistent with observations on previous topologies.
- As k increases further ($k \geq 6$), both methods converge and exhibit similar average latency, with Advanced K-Means maintaining marginally lower values for larger k. For the highest tested values, the difference is negligible.

- The minimization of maximum response time is much less pronounced in this topology. Enhanced K-Means++ demonstrates increased sensitivity, resulting in significant spikes in maximum latency for $k = 3$ and $k = 8$. In these cases, the placement randomness may have led to less favorable controller assignments for distant nodes, increasing the worst-case response time.
- The Geant2012 network is characterized by a dense core and a few peripheral nodes that are significantly distant from the central cluster. This structural property poses challenges for controller placement methods based on probabilistic dispersion, as isolated or peripheral nodes may occasionally be assigned suboptimal controllers, bringing cluster to distant regions with more latency.

IIJ

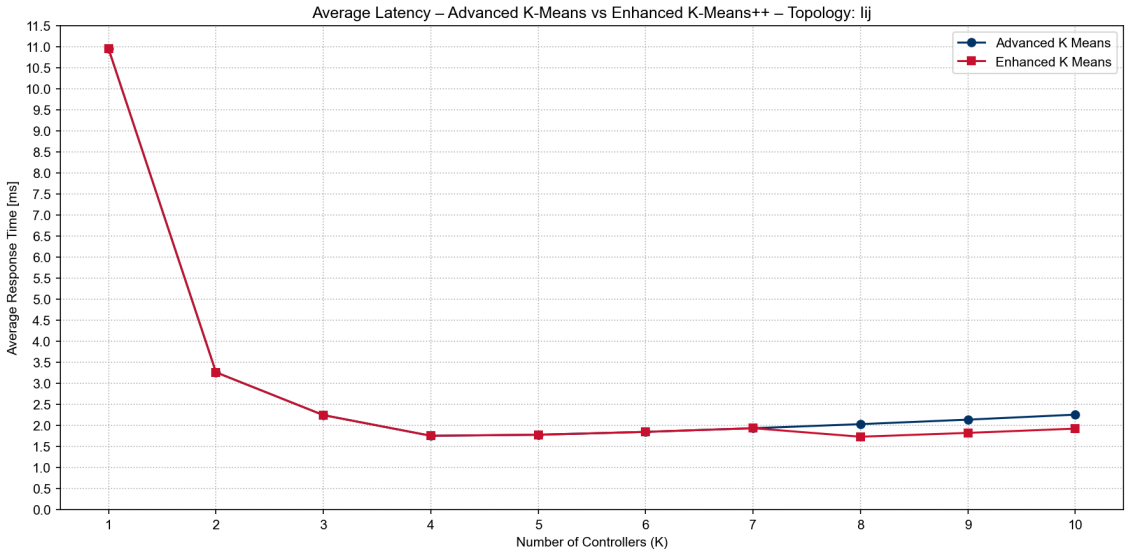


Fig. 7.21. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology.

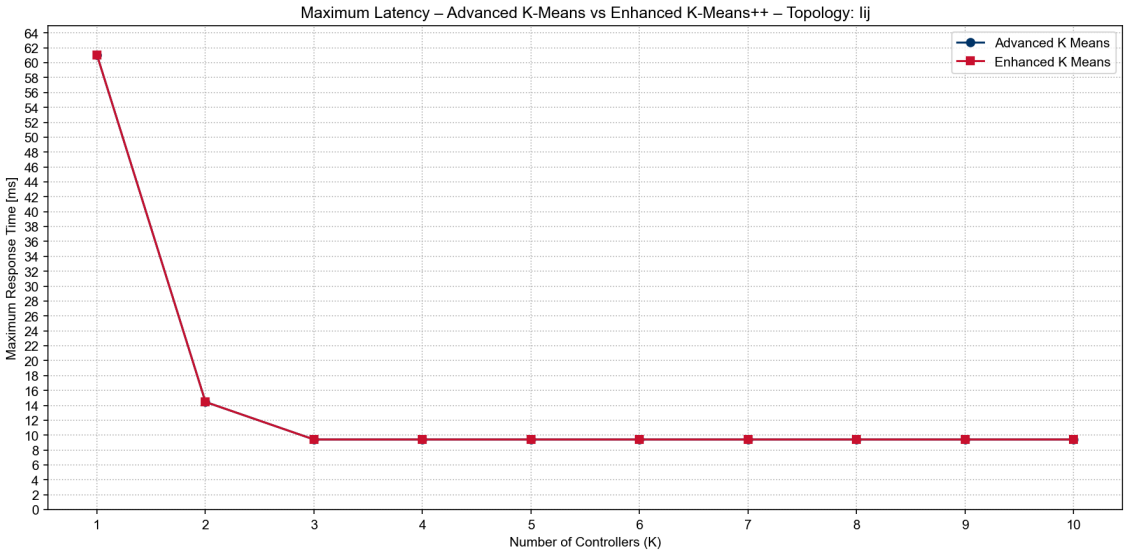


Fig. 7.22. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology.

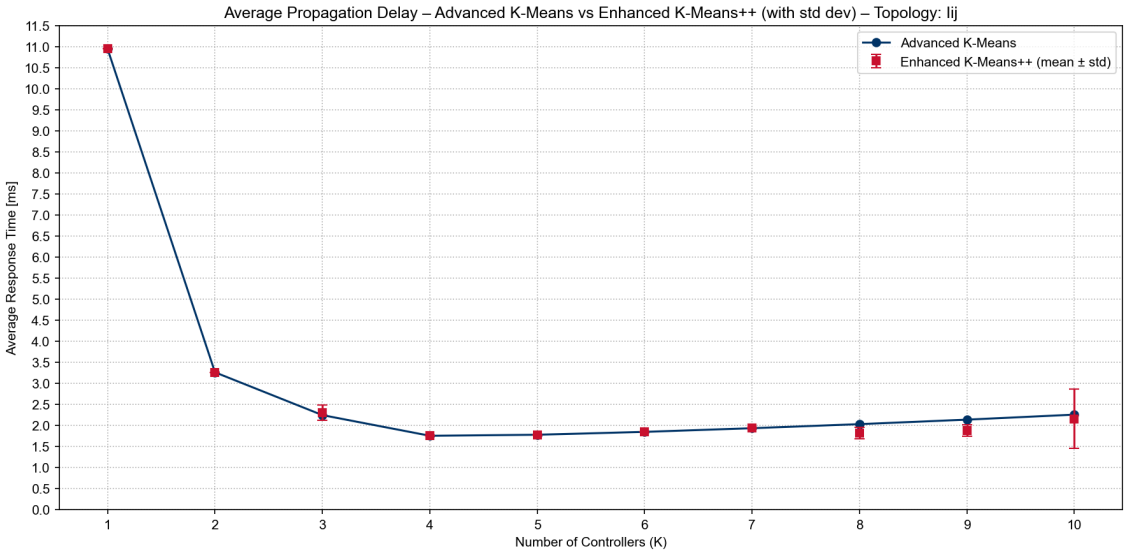


Fig. 7.23. Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology.

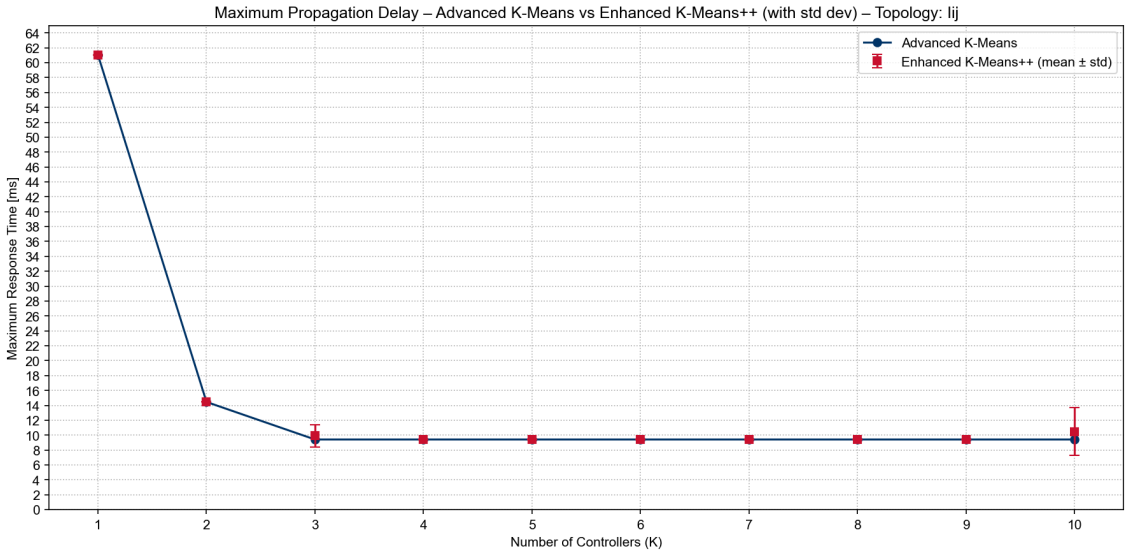


Fig. 7.24. Maximum Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology.

The IIJ network provides a particularly challenging scenario due to its highly irregular, dual-cluster structure connected by extremely long intercontinental links. This is reflected in the experimental results:

- For $k = 1$, both algorithms position the sole controller in one of the two geographically distant clusters, resulting in extremely high average and maximum latencies (exceeding 60 ms) due to the need to serve nodes across the globe.
- Introducing a second controller ($k = 2$) allows each algorithm to assign a controller to each of the two remote clusters. This immediately reduces both average and maximum latency by over 75%, demonstrating the pronounced impact of geographically-aware placement in this topology.
- For $k = 3$ and $k = 4$, both algorithms continue to deliver nearly identical performance, as additional controllers are placed to cover remaining nodes in both clusters, leading to further reduction and then stabilization of latency values (but with maximum delay being moderately high – roughly 10%)
- For $k = 5$ through $k = 7$, average and maximum latency plateau for both methods, indicating that the marginal benefit of adding more controllers is limited once both clusters are well-covered.
- Notably, for larger values ($k = 8$ to $k = 10$), Enhanced K-Means++ achieves marginally better average latency results, while maintaining similar or identical maximum latency to Advanced K-Means.
- Across all values of k, the variance in latency is very low (as evidenced by minimal standard deviation bars), which is attributable to the strong structural separation and deterministic assignment of controllers to natural clusters in the network.

Pionier L3

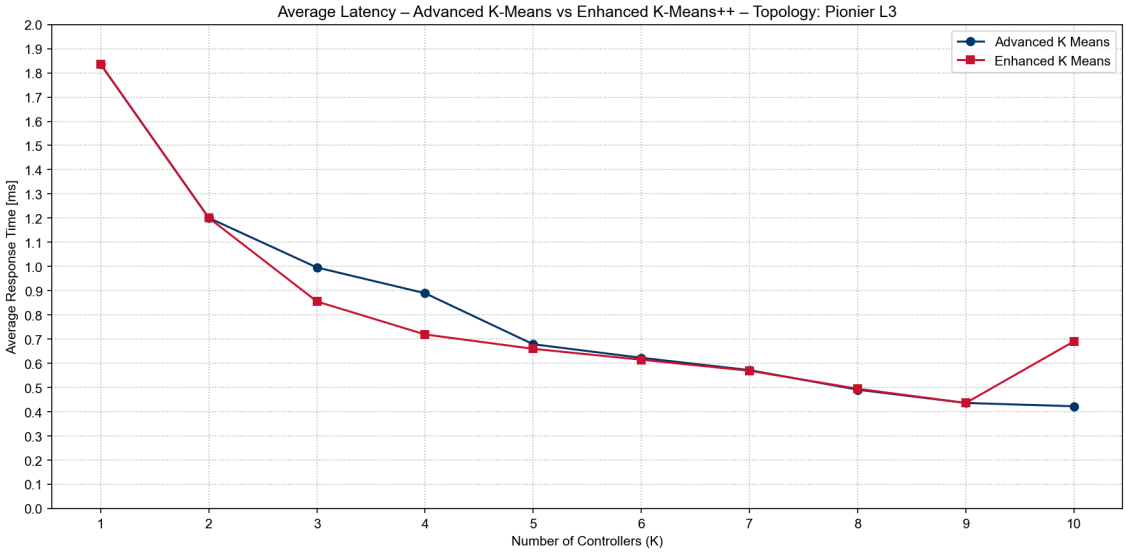


Fig. 7.25. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology.

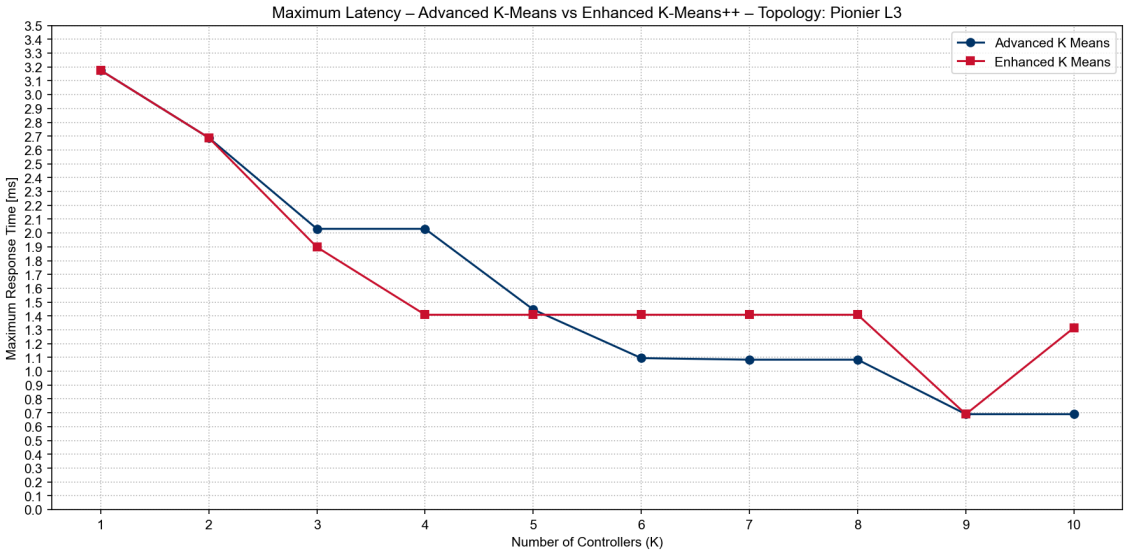


Fig. 7.26. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology.

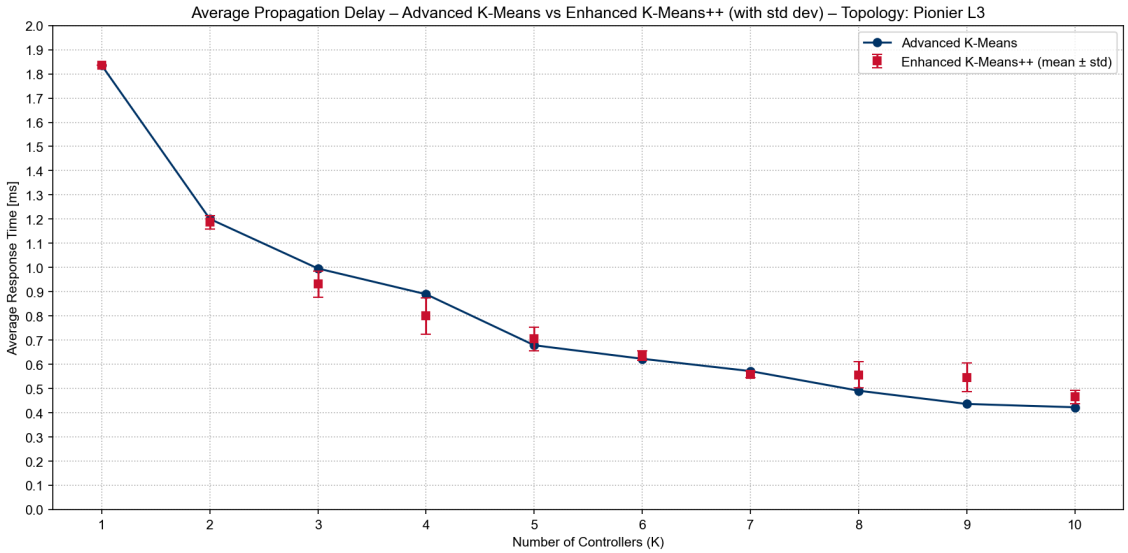


Fig. 7.27. Average Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology.

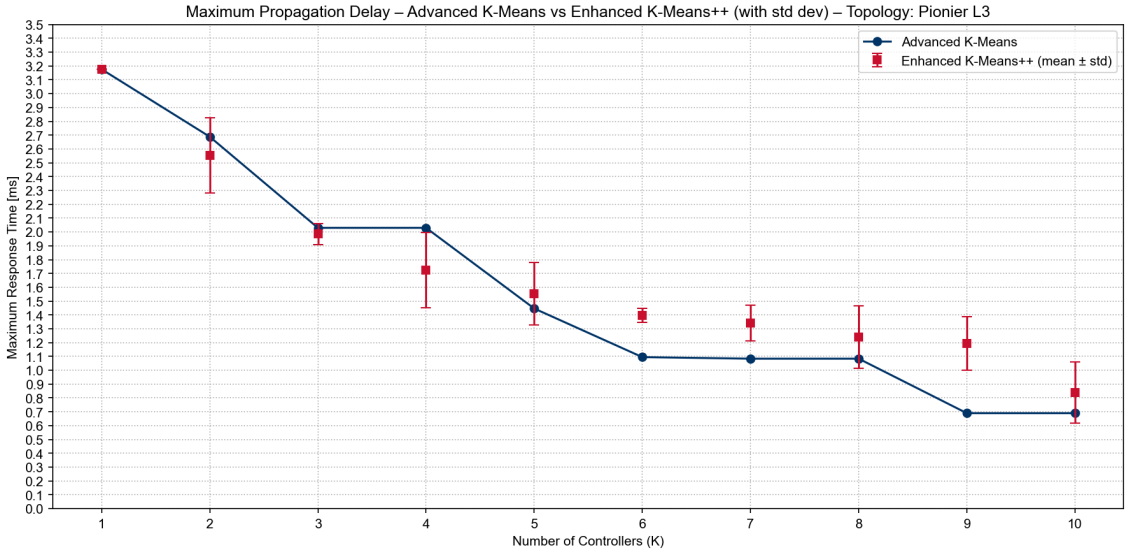


Fig. 7.28. Maximum Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology.

Analysis of the Pionier L3 network—a relatively small and spatially compact Polish backbone—reveals several recurring patterns in the performance of both algorithms:

- For $k = 3$ and $k = 4$, Enhanced K-Means++ achieves marginally lower average and maximum latency compared to the Advanced K-Means baseline. This is consistent with results on other smaller and more regular networks, suggesting that the stochastic center selection can occasionally exploit local optima.
- For larger values of k (especially $k \geq 8$), the performance of Enhanced K-Means++ begins to diverge, with occasional increases in both average and maximum latency relative to Advanced K-Means. This is likely due to the relatively uniform and short inter-node distances in the Pionier

L3 topology—when the number of controllers is high, the differences between candidate nodes become minimal, and the probabilistic selection may, in some runs, deviate from the optimal, more deterministic dispersion.

- Standard deviation grows for higher k, reflecting increased randomness and less predictable outcomes in Enhanced K-Means++ as candidate locations for controllers become less differentiated.
- Both algorithms converge to very low latency values, confirming that in dense, small-scale networks, the selection strategy becomes less critical when sufficient controllers are available. Nonetheless, Advanced K-Means tends to be slightly more stable and deterministic, which translates to marginally lower and more consistent delay for high k.

Belnet 2003

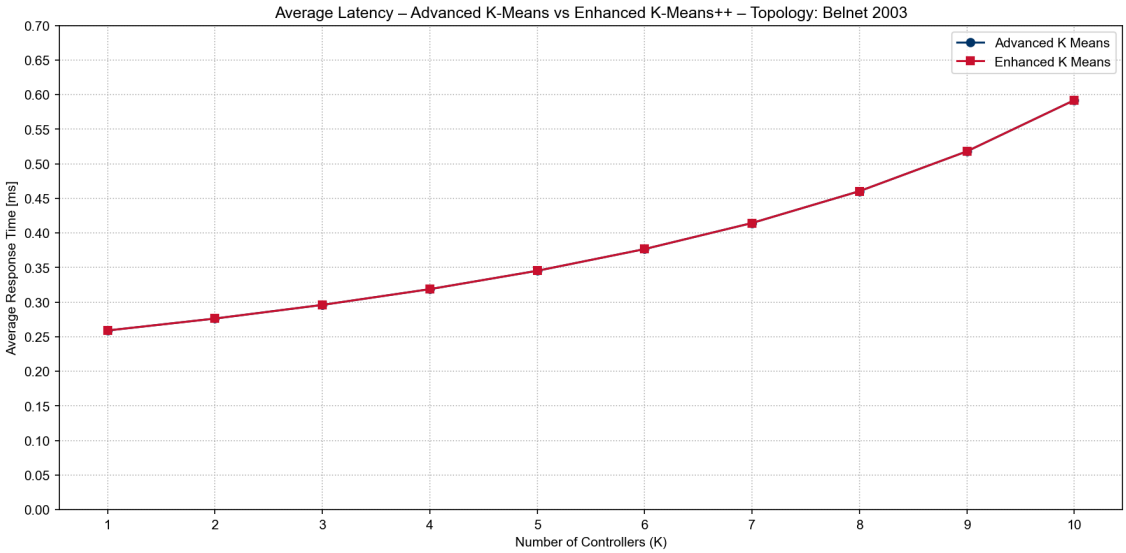


Fig. 7.29. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Belnet 2003 topology.

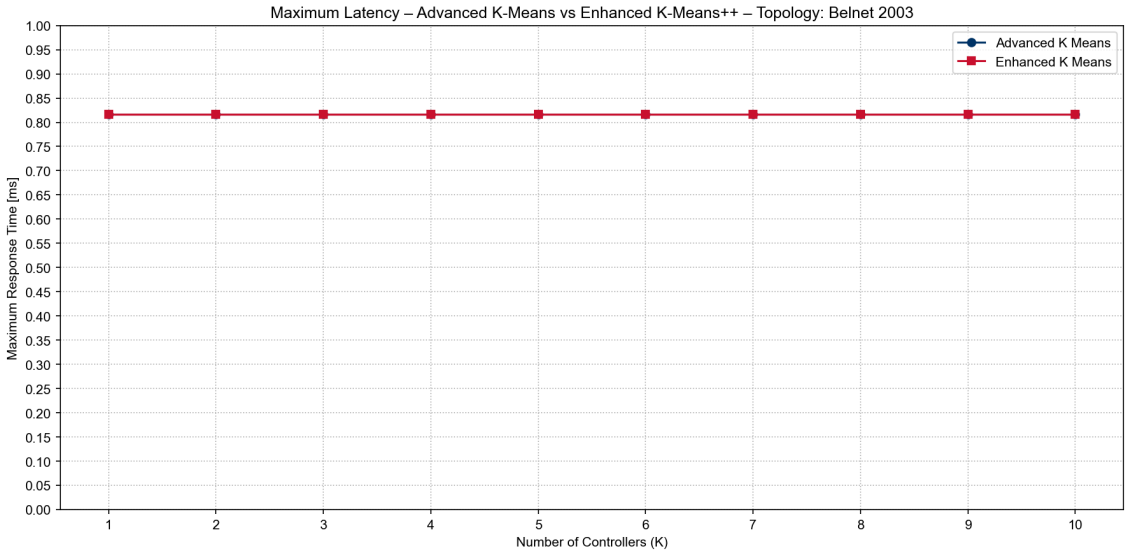


Fig. 7.30. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Belnet 2003 topology.

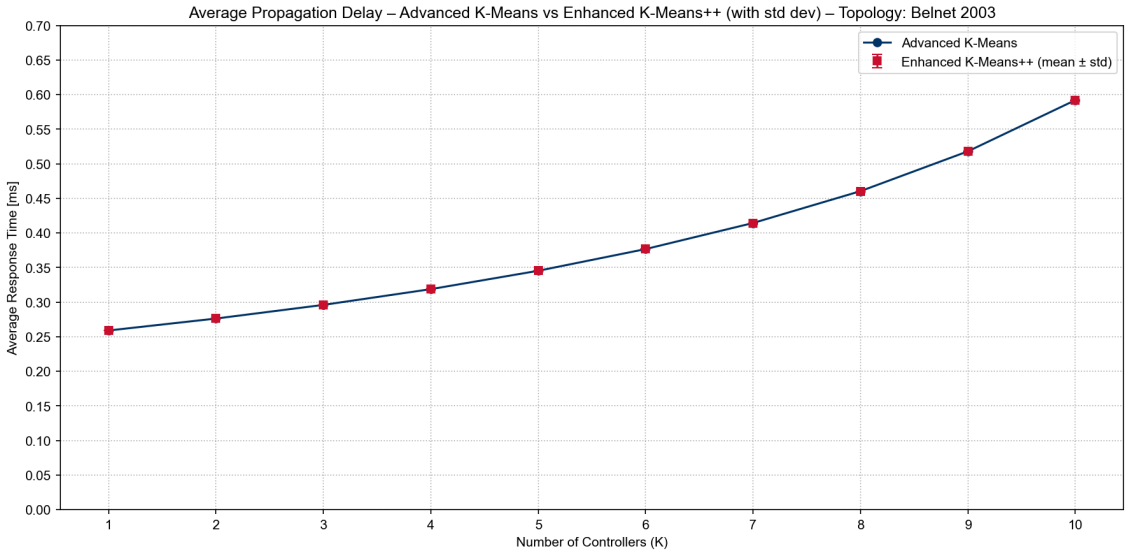


Fig. 7.31. Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Belnet 2003 topology.

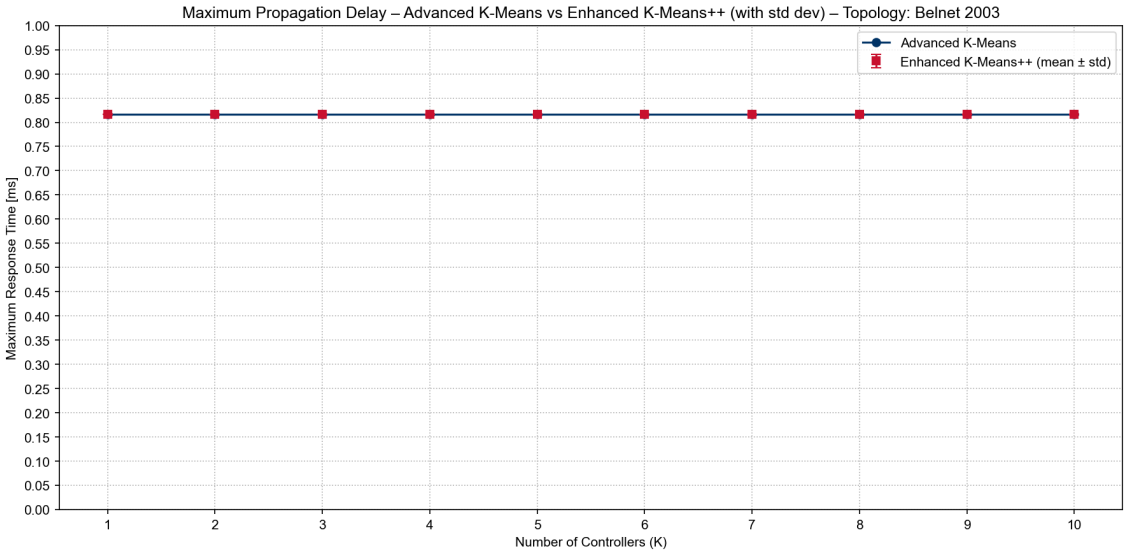


Fig. 7.32. Maximum Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means$++$ for the Belnet 2003 topology.

The Belnet 2003 topology presents a unique and instructive scenario for controller placement algorithms due to its pronounced star-like structure. In this network, the majority of peripheral nodes are directly connected to a highly central hub, creating a topology with minimal multi-hop connectivity. The experimental results for both average and maximum response times are strikingly distinct from the other tested networks:

- For all values of k from 1 to 10, both Advanced K-Means and Enhanced K-Means$++$ return virtually identical results. Average latency does not decrease with increasing k; in fact, it grows slightly, which is counter-intuitive compared to other topologies. The maximum latency remains perfectly flat across all k, staying at a constant value of approximately 0.82 ms. This can be attributed to the inability of the algorithms to meaningfully partition the network—since all non-central nodes are already as close as possible to the central hub, additional controllers cannot improve node-to-controller distances.
- A careful inspection of the algorithms' behavior reveals that after the selection of the initial center (the central hub), no other candidate node is able to draw assignments from surrounding nodes, since their shortest paths to the central hub are minimal and all assignments remain with the first center. As a result, new candidate centers are never "adopted," and the algorithm converges prematurely with only one active controller, regardless of the value of k. Both algorithms thus fall into a local minimum specific to star topologies, where the addition of further centers brings no topological benefit.
- This outcome highlights a fundamental limitation of distance-based and centrality-driven clustering methods in pathological network configurations such as stars, where structural redundancy for alternative controller locations is essentially absent.

Bellsouth

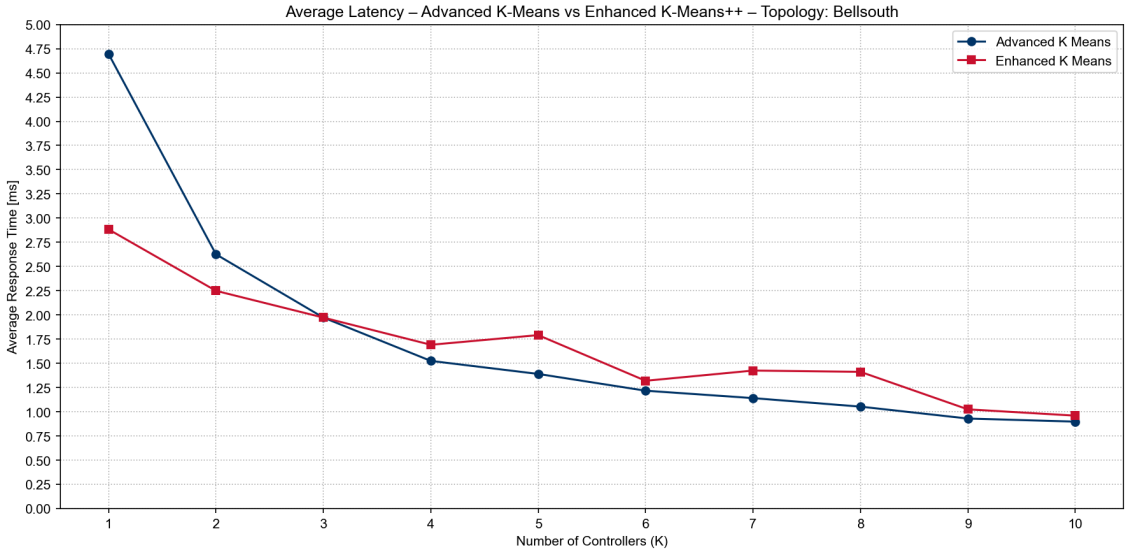


Fig. 7.33. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology.

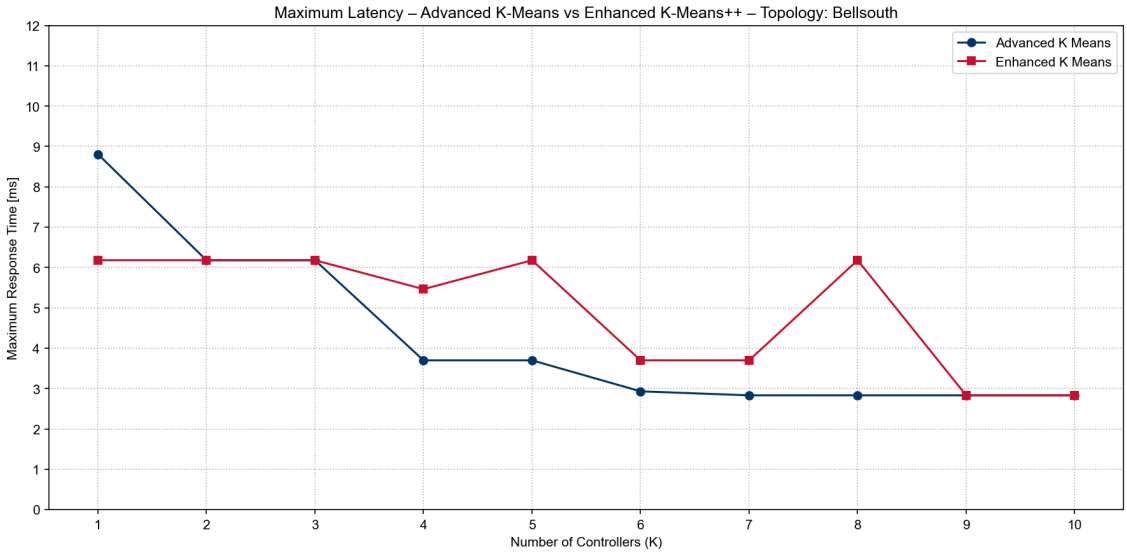


Fig. 7.34. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology.

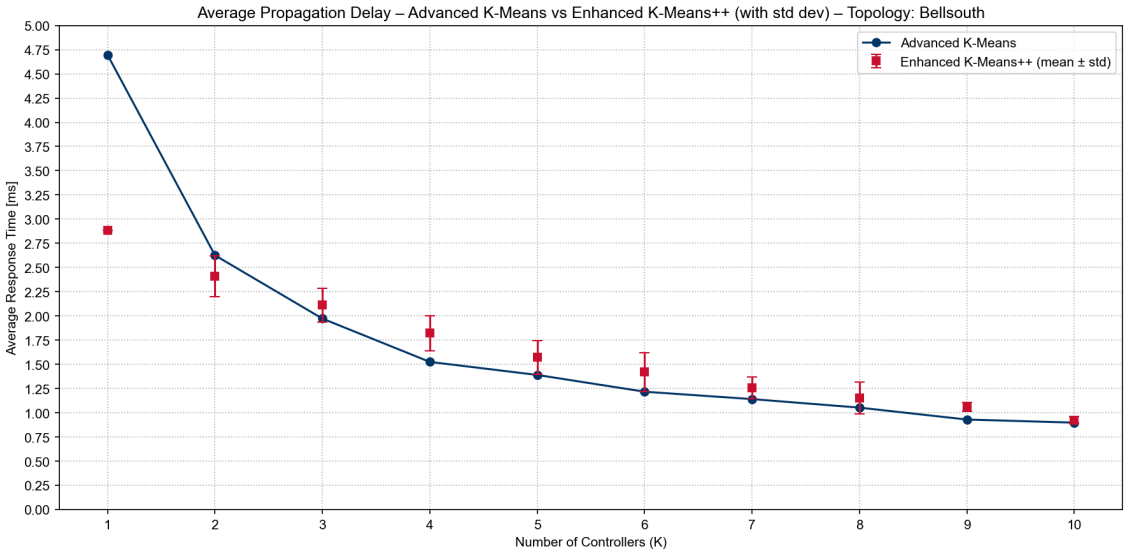


Fig. 7.35. Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology.

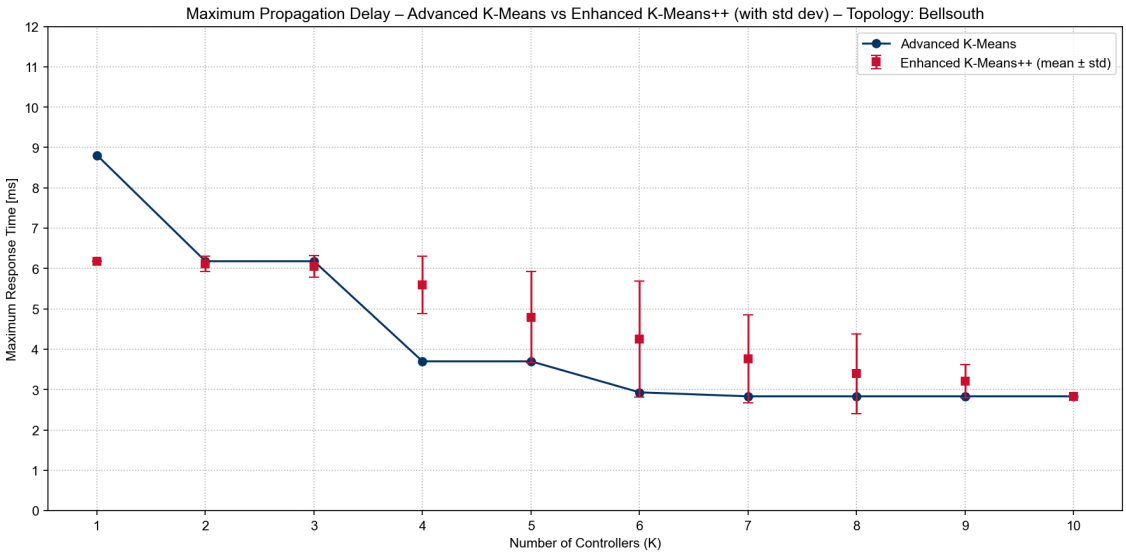


Fig. 7.36. Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology.

The results for the Bellsouth topology reveal several notable dynamics:

- Initially, the Enhanced K-Means++ algorithm demonstrates a substantial improvement over Advanced K-Means for $k = 1$, achieving a reduction in average latency of over 1.5 ms. This can be attributed to the strong influence of the betweenness and closeness centrality metrics, which, with their combined weight set to 100%, drive the selection towards globally central nodes rather than just the highest-degree node. However, as k increases, this advantage diminishes. Enhanced K-Means++ exhibits significant fluctuations for higher values of k, ultimately converging to results similar to those obtained by Advanced K-Means.

- It is particularly evident that while the weighting of centrality metrics ($w_{\text{betweenness}}$ and $w_{\text{closeness}}$) can substantially impact the very first center selection, this effect becomes marginal in the subsequent iterations of the algorithm. This observation is supported by further tests in which setting the degree weight to 100% leads to identical $k = 1$ results for both algorithms, but does not alter their behavior for larger k. The performance gap for average latency closes quickly after the first controller is added, and for larger k values, both algorithms yield nearly identical results. Furthermore, the stochastic nature of Enhanced K-Means++ manifests in larger standard deviations for maximum latency in the mid-range of k, but does not produce systematically lower delay values.
- Overall, these findings indicate that the only tangible benefit of using centrality metrics in the initial center selection is at $k = 1$, and that the stochastic k-means++ selection mechanism in subsequent center choices has a far greater impact on the results than any weighting scheme. The results confirm that, after the initial placement, the algorithm's convergence is dominated by the topological constraints and the minimization of local distances, rendering the influence of the centrality metrics negligible in practical scenarios.

Cesnet 1993

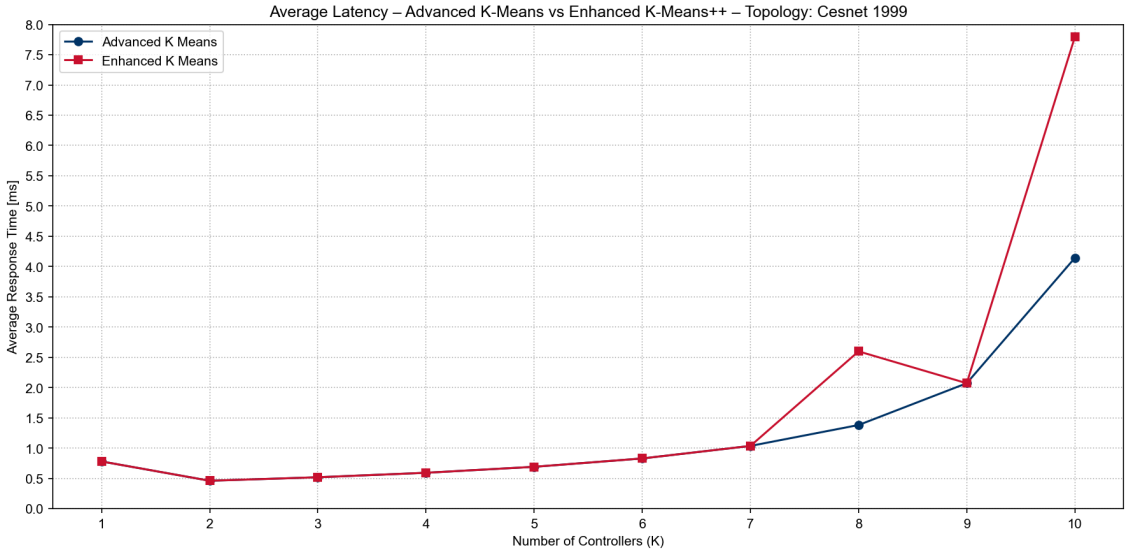


Fig. 7.37. Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Cesnet 1993 topology.

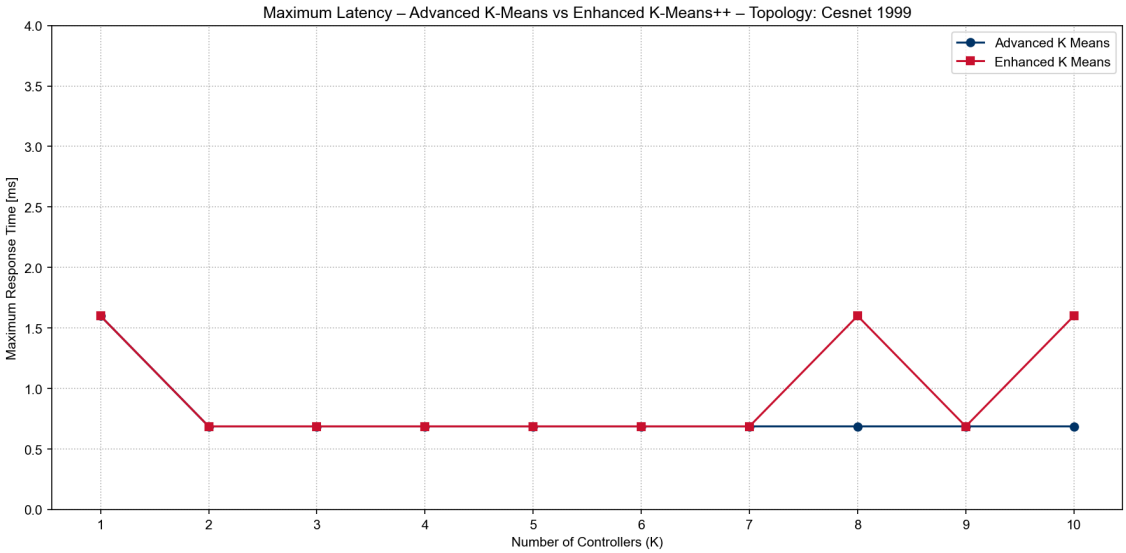


Fig. 7.38. Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Cesnet 1993 topology.

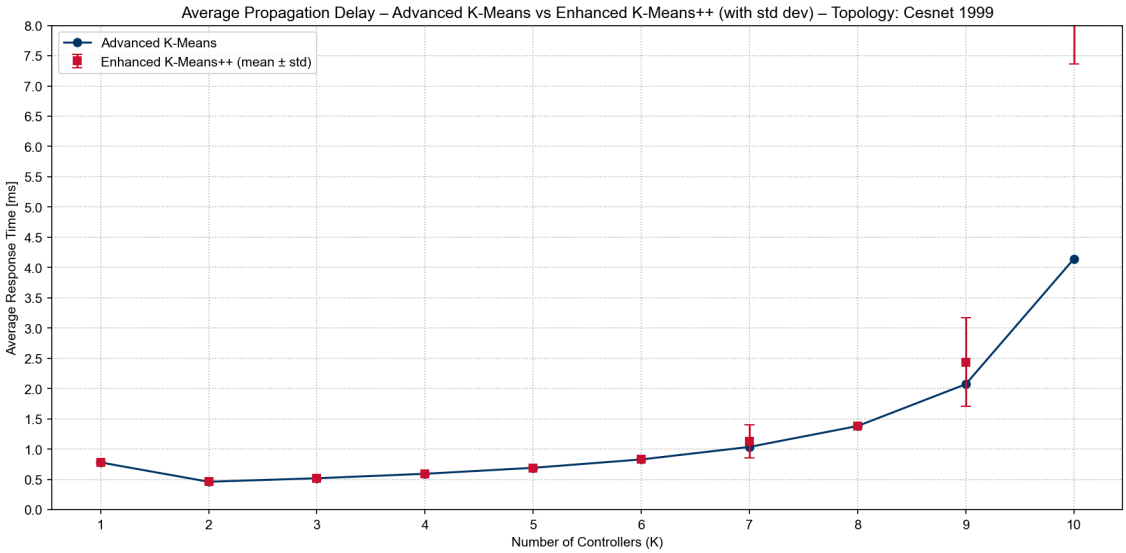


Fig. 7.39. Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Cesnet 1993 topology.

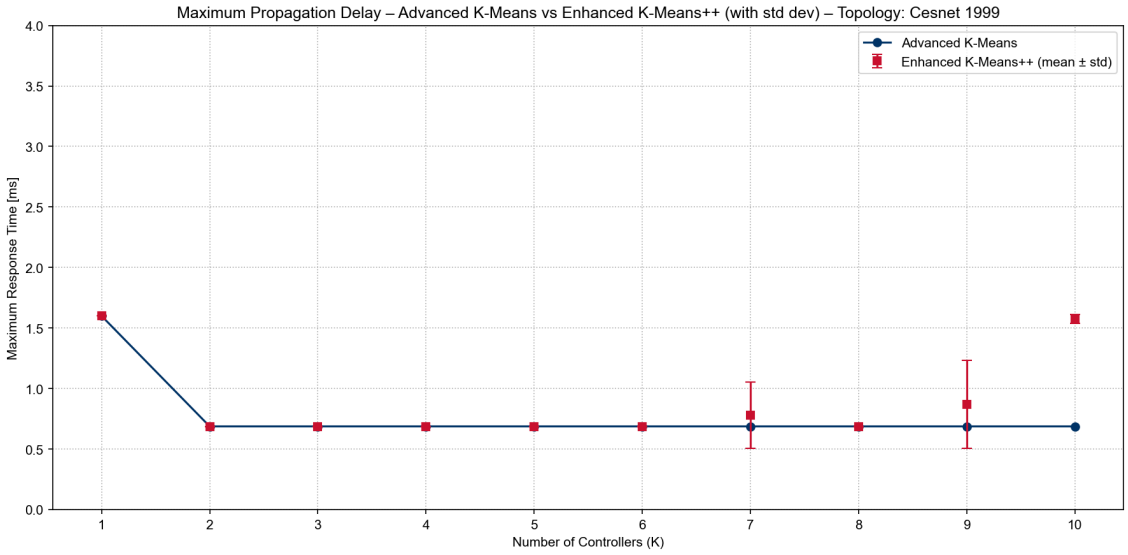


Fig. 7.40. Maximum Propagation Delay (mean $\pm$ std) – Advanced K-Means vs Enhanced K-Means$++$ for the Cesnet 1993 topology.

The Cesnet 1993 network, being the smallest and most regular topology in the test suite, reveals an important limitation of both algorithms when dealing with a low number of nodes.

- For $k = 1$ and $k = 2$, both Advanced K-Means and Enhanced K-Means$++$ provide almost identical and optimal results in terms of both average and maximum latency. However, as the number of controllers k increases beyond the natural clustering capacity of the network, the performance of Enhanced K-Means$++$ degrades significantly, particularly for $k = 8, 9$, and 10. This manifests as a sharp increase in average latency, with a corresponding rise in maximum latency and its variability.
- The underlying reason for this effect is that, in small networks, adding additional controllers beyond what the topology structurally requires leads to poorly balanced cluster assignments. The stochastic component of Enhanced K-Means$++$ causes it to place some controllers in suboptimal positions when the number of clusters exceeds the logical separation present in the network, resulting in clusters with very few nodes or even singleton assignments. This behavior is especially visible for $k = 10$, where the average latency increases abruptly, indicating the fragmentation of clusters and the loss of efficiency.
- Consequently, these results highlight that both algorithms—while robust for moderate values of k—can become highly sensitive to edge cases when applied to small networks with a high number of controllers. In such scenarios, the deterministic center selection of Advanced K-Means maintains more stable performance, while the stochastic selection in Enhanced K-Means$++$ may introduce instability, especially when the logical structure of the network no longer supports meaningful cluster formation for large k.

7.3.2. Comparative Analysis

The following section synthesizes the key findings arising from the systematic evaluation of both controller placement algorithms across a diverse set of network topologies. The comparative analysis aims to highlight the strengths and limitations of each method, interpret their behavior in various network scenarios, and extract actionable insights for SDN controller deployment. Particular attention is given to the impact of centrality weights, the stochastic versus deterministic nature of center selection, and the sensitivity of both approaches to network size and structure. This discussion provides a foundation for practical recommendations and further research directions in latency-aware SDN controller placement.

1. The Advanced K-Means algorithm demonstrates significantly higher stability due to its foundational assumptions. This method does not incorporate hybrid or modal solutions, but rather selects the initial center based on node degree and places subsequent centers at the farthest distance, resulting in consistent and predictable outcomes.
2. In contrast, the Enhanced K-Means++ method is more sensitive to edge cases and can converge to suboptimal solutions for high values of k. However, in some cases, it achieves better results by taking a notch—that is, instead of always searching for the most distant center, it distributes centers according to probabilistic opportunities, allowing for a more balanced, chance-driven deployment.
3. Both methods perform poorly in star-like topologies, where a single central node dominates the network and there is little room for meaningful clustering beyond the first center.
4. The influence of the weights for betweenness, closeness, and node degree is noticeable only for a single controller (or possibly for $k = 2$). As the number of centers increases and re-calculation through the k-means process occurs, the impact of initial conditions becomes negligible.
5. These weights proved significant only in very large and condensed networks (with some scattered nodes), where node degree had less influence than betweenness or closeness centrality.
6. For very small networks, clustering is not beneficial—furthermore, the stochastic method can dramatically increase latency. Such networks usually operate efficiently with one or two controllers, maintaining quick request handling.
7. Both algorithms stabilize in topologies featuring two dense clusters connected by a small number of long links, resulting in persistently high maximum latency, even as k increases. Request handling in these networks remains constrained by the propagation overhead of traversing the globe.
8. In highly irregular and larger networks, where some nodes lie far outside the main "cluster," both methods lose efficiency: the average latency is only marginally reduced, and the maximum latency cannot be adequately improved due to extreme node separation.
9. The methods typically converge well for $k = 3$ or $k = 4$, which for some topologies (e.g., Atmnet and Internet2 OS3E) may be the optimal number of controllers to meet QoS requirements. In such cases, the Enhanced K-Means++ method demonstrates its strengths.

8. ASSESSMENT AND RECOMMENDATIONS

This chapter presents a critical synthesis of the main findings from the experimental evaluation of controller placement strategies in Software-Defined Networks. Drawing on results obtained across diverse network scenarios, we provide a concise assessment of the advantages and limitations of different algorithmic approaches to latency minimization. Based on these insights, the chapter formulates practical recommendations to support the design and deployment of efficient and robust controller placement methods in latency-sensitive SDN environments. The goal is to offer clear guidance to both researchers and practitioners confronting the challenges of modern programmable networks.

8.1. Evaluation of the Results and Key Findings

The experimental study conducted in this thesis demonstrates that the choice of controller placement algorithm has a direct and significant effect on both average and maximum network latency. Deterministic methods such as Advanced K-Means consistently deliver stable performance across a range of topologies, while stochastic approaches like Enhanced K-Means$++$ can, in certain scenarios, achieve superior latency minimization—though with greater sensitivity to irregular network structures. These results highlight the importance of algorithm selection, parameterization, and a nuanced understanding of network topology in achieving optimal request-handling performance.

One of the most important findings is the robustness of Advanced K-Means, which consistently provides stable latency reduction by leveraging a degree-based initial center selection and deterministic allocation of subsequent centers as the farthest nodes. This ensures predictable improvements in delay metrics, particularly in regular or moderately heterogeneous topologies, making it a safe and reliable choice for many practical deployments.

On the other hand, Enhanced K-Means$++$, which introduces probabilistic seeding and multi-criteria centrality weighting, offers the potential to break free from local minima and achieve better results in certain cases—especially in large or clustered networks. However, its sensitivity to edge cases and propensity to fall into suboptimal solutions at higher values of k highlight the importance of careful parameterization and the need for additional safeguards when operating in networks with non-trivial structures.

A key lesson learned is that neither approach is universally optimal across all network types. Both algorithms struggle in star-like topologies where the addition of controllers beyond the primary hub provides little benefit, and both demonstrate diminishing returns or even performance degradation when an excessive number of controllers is introduced in small or highly centralized networks. Furthermore, the experiment has shown that the influence of centrality-based weights is significant mainly for initial center selection; as clusters are iteratively refined, their importance diminishes, especially as k increases.

Another essential observation is that, in networks characterized by geographically or logically

separated node clusters, even optimal controller placement can only partially mitigate high maximum delays, as inter-cluster propagation remains a limiting factor. This suggests that architectural changes or the addition of redundant paths may be required in such cases to achieve truly low-latency operation.

In summary, the results confirm that careful algorithm selection and parameter tuning are crucial for latency-sensitive SDN applications. Deterministic algorithms provide reliability and stability, while stochastic or hybrid approaches, when appropriately configured, can deliver improvements in more complex or less regular topologies. Ultimately, the best results are achieved through a nuanced understanding of the interplay between network topology, controller count, and the specific requirements for request latency in the target deployment.

8.2. Recommendations

The insights obtained from the comprehensive evaluation suggest several key recommendations and considerations for the design and implementation of future controller placement algorithms focused on minimizing request-handling latency in SDN environments. The following points summarize crucial factors, strategies, and potential pitfalls to be aware of during the development process:

1. **Topology-Awareness:** Algorithms should explicitly account for the structural characteristics of the network topology. Different topologies such as star, mesh, or clustered topologies present distinct challenges; therefore, a successful method must include preliminary analyses or adaptive mechanisms capable of dynamically recognizing and adjusting to such topological features.
2. **Balancing Determinism and Stochasticity:** Deterministic methods like Advanced K-Means provide consistent performance across various scenarios, whereas stochastic algorithms such as Enhanced K-Means++ can escape local minima and potentially yield better results. An effective algorithm could combine deterministic initialization and probabilistic adjustments or implement stochastic mechanisms under controlled conditions to balance reliability with performance gains.
3. **Sensitivity to Edge Cases:** Special care must be taken to handle edge cases and small-scale networks, which can expose vulnerabilities in certain algorithms. As demonstrated, overly aggressive clustering or unnecessary placement of additional controllers in small topologies can significantly degrade performance. Introducing conditional checks or thresholds to avoid unnecessary cluster formation is advisable.
4. **Centrality Metrics Selection and Weighting:** While metrics such as node degree, closeness, and betweenness centrality have proven valuable, their effectiveness is context-dependent. Algorithms should thus include mechanisms for dynamically adjusting centrality metric weights based on network characteristics or historical performance, rather than relying on fixed arbitrary parameters.
5. **Avoiding Local Minima:** Frequent reevaluation and recalculation of cluster centers are critical. Implementations must include explicit criteria to detect and break out of suboptimal local minima, possibly by employing restart strategies, hybrid selection heuristics, or simulated annealing

techniques.

6. **Adaptive Controller Placement Strategy:** An adaptive strategy that adjusts the number of controllers (k) according to real-time or near-real-time assessments of latency performance is highly beneficial. It would dynamically add or remove controllers, ensuring optimal resource allocation and responsiveness to changing network conditions.
7. **Propagation Delay and Geographical Constraints:** Algorithms should integrate geographical constraints explicitly. Real-world deployments frequently face latency limitations inherently linked to geographical distribution. Therefore, placement methods should explicitly optimize locations considering physical distance and propagation delays, rather than abstract topological metrics alone.
8. **Scalability and Complexity Management:** Efficiency is paramount, particularly in large-scale deployments. Algorithms should be carefully designed to maintain computational feasibility and scalability. A trade-off analysis between accuracy (optimal latency) and computational overhead should be regularly performed to ensure practicality and responsiveness.
9. **Resilience Requirements** Beyond minimizing latency, algorithms must also ensure sufficient redundancy and fault tolerance. Placement strategies should avoid overly centralized solutions, even if optimal from a latency perspective, as they may compromise network reliability. Strategies should explicitly balance latency minimization with network resilience requirements. As highlighted by Rak [42], the contemporary communication infrastructure is increasingly exposed to a wide spectrum of failures, including those caused by natural disasters, technological malfunctions, and intentional attacks, making it impossible to eliminate all sources of disruptions. Consequently, networked systems must be designed with resilience as a fundamental property, incorporating preventive and reactive mechanisms that ensure service continuity even in failure scenarios. The deployment of appropriate resilience strategies, such as redundancy, backup paths, and proactive failure detection, is essential to maintain acceptable levels of availability and reliability, thus fulfilling the expectations of modern ICT systems for uninterrupted service [42].
10. **Robust Validation and Testing:** Comprehensive validation across diverse network scenarios is essential. Extensive simulations or real-world validations on varied topologies, sizes, and latency requirements should be standard practice. Such rigorous testing ensures algorithmic robustness, generalizability, and the identification of potential hidden pitfalls.

In conclusion, designing effective controller placement algorithms requires a nuanced understanding of network topologies, intelligent balance between deterministic and stochastic methods, awareness of geographic constraints, and rigorous validation. Careful attention to these recommendations will help in developing robust algorithms capable of reliably minimizing request-handling latency in SDN deployments.

9. CONCLUSIONS

This thesis provided a comprehensive exploration and evaluation of algorithms designed to optimize the placement of controllers within Software-Defined Networks (SDNs), specifically aiming at reducing latency in handling network requests. Through an extensive theoretical and empirical investigation, the study addressed critical aspects of the Controller Placement Problem (CPP), offering insights and proposing enhancements that can significantly impact real-world SDN deployments.

The thesis began with an in-depth literature review and taxonomy of existing approaches to CPP, categorizing methods into optimal, heuristic, metaheuristic, clustering-based, game-theoretic, multi-objective, and reinforcement learning strategies. This foundational analysis set the stage for a detailed exploration of two central approaches: Advanced K-Means and High Degree with Independent Dominating Set (HDIDS). The study's analytical and experimental assessment revealed critical insights into these methods' strengths, limitations, and areas for improvement.

Empirical experimentation using real-world network topologies demonstrated the robustness and stability of the Advanced K-Means algorithm across various scenarios. It provided consistent performance benefits, particularly in terms of predictable and stable latency reductions. Conversely, the Enhanced K-Means++ variant, introduced in this thesis, exhibited greater flexibility and potential for latency improvement in certain scenarios through its probabilistic and multi-metric centrality-based enhancements. However, it also revealed sensitivities and instabilities in specific network configurations, particularly for higher numbers of controllers and edge cases.

One of the central findings of this work is that no single algorithm universally outperforms others across all network types. Each method has contexts in which it excels and others where it faces limitations. For example, deterministic methods consistently offer stable outcomes in regular network structures, while stochastic approaches may be beneficial in complex or clustered scenarios but require careful parameterization and risk management to avoid suboptimal placements.

The thesis also identified specific challenges in highly centralized networks, such as star topologies, and networks featuring significant geographical separation, which inherently limit the effectiveness of latency minimization through controller placement alone. These insights underline the necessity for algorithms to adapt dynamically to structural network properties and varying operational demands.

Based on the findings, practical recommendations were formulated, emphasizing topology-awareness, dynamic adaptability, careful management of determinism and stochasticity, multi-metric centrality weighting, and robust validation across diverse network scenarios. These recommendations provide clear guidance for future algorithm design, implementation, and deployment in latency-sensitive SDN environments.

In conclusion, this thesis contributes to the broader understanding of controller placement

strategies by offering rigorous comparative analyses, novel algorithmic enhancements, and clear practical recommendations. It sets a solid foundation for future research, particularly in developing adaptive and resilient algorithms capable of meeting the evolving demands of Software-Defined Networking.

BIBLIOGRAPHY

[1] M. Jammal, T. Singh, A. Shami, R. Asal, and Y. Li, "Software Defined Networking: State of the art and Research Challenges," *Computer Networks*, vol. 72, pp. 74–98, 2014.

[2] T. Das, V. Sridharan, and M. Gurusamy, "A Survey on Controller Placement in SDN," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 472–503, 2020.

[3] B. Heller, R. Sherwood, and N. McKeown, "The Controller Placement Problem," *ACM SIGCOMM Computer Communication Review*, vol. 42, no. 4, pp. 473–478, 2012.

[4] F. Zobary, "Optimizing SDN Controller to Switch Latency for Controller Placement Problem," *Informatica*, vol. 48, no. 8, pp. 165–176, 2024.

[5] S. Lange, S. Gebert, T. Zinner, and P. Tran-Gia, "Heuristic Approaches to the Controller Placement Problem in Large Scale SDN Networks," in *IEEE Transactions on Network and Service Management*, vol. 12, pp. 4–17, 2015.

[6] C. Liao, J. Chen, K. Guo, S. Liu, and D. Gao, "MODECP: A Multi-Objective Based Approach for Solving Distributed Controller Placement Problem in Software Defined Network," *Sensors*, vol. 22, no. 15, p. 5475, 2022.

[7] Y. Qi, D. Wang, W. Yao, H. Li, and Y. Cao, "Towards Multi-Controller Placement for SDN Based on Density Peaks Clustering," *IEEE Transactions*, 2019.

[8] N. Firouz, M. Masdari, A. Babazadeh Sangar, and K. Majidzadeh, "A Hybrid Multi-objective Algorithm for Imbalanced Controller Placement in Software-Defined Networks," *Journal of Network and Systems Management*, vol. 30, no. 51, 2022.

[9] G. D. Singh, V. Tripathi, A. Dumka, R. S. Rathore, M. Bajaj, J. Escorcia-Gutierrez, N. O. Aljehane, V. Blazek, and L. Prokop, "A novel framework for capacitated SDN controller placement: Balancing latency and reliability with PSO algorithm," *Alexandria Engineering Journal*, vol. 87, pp. 77–92, 2024.

[10] K. Killi and S. Somayaji, "Controller placement with planning for failures in SDN," *Computer Networks*, vol. 176, p. 107282, 2020.

[11] A. Alowa and T. Fevens, "Combined Degree-Based with Independent Dominating Set Approach for Controller Placement Problem in Software Defined Networks," in *2019 22nd Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)*, pp. 269–276, IEEE, 2019.

[12] J. Chen, Y. Xiong, and D. He, "A Density-Based Controller Placement Algorithm for Software Defined Networks," in *2022 IEEE International Conferences on Internet of Things (iThings), Green Computing and Communications (GreenCom), Cyber, Physical and Social Computing (CPSCom), Smart Data (SmartData), and IEEE Congress on Cybermatics*, pp. 287–294, IEEE, 2022.

[13] N. H, M. Kalmat, F. Khan, S. Shinde, and N. D.G., "Dynamic Controller Placement in Software Defined Networks using Spectral Clustering," in *Proceedings of the 14th IEEE International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, IEEE, IEEE, 2023. Paper ID: IEEE - 56998.

[14] K. Venkatesan, K. Meena, V. Priyadharshini, and V. G. Kumar, "Optimal Controller Placement in Software Defined Networks (SDN) Using Non-Zero Sum Game Approach," in *Proceedings of the 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS)*, pp. 1–6, IEEE, 2023.

[15] A. Ksentini, M. Bagaa, T. Taleb, and I. Balasingham, "On Using Bargaining Game for Optimal Placement of SDN Controllers," in *Proceedings of the IEEE International Conference on Communications (ICC)*, pp. 1–6, IEEE, 2016.

[16] H. Xu, X. Chai, and H. Chen, "A Collaborative Approach based on Competitive Game for Multi-Controller Placement in SDN," in *Proceedings of the 2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, (Wuhan, China), IEEE, 2022.

[17] J. Hemagowri and P. T. Selvan, "A Hybrid Evolutionary Algorithm of Optimized Controller Placement in SDN Environment," *Computer Assisted Methods in Engineering and Science*, vol. 30, no. 4, pp. 539–556, 2023.

[18] Y. Wu, S. Zhou, Y. Wei, and S. Leng, "Deep Reinforcement Learning for Controller Placement in Software Defined Network," in *IEEE INFOCOM WKSHPS*, 2020.

[19] O. A. Aboelela and R. A. Sadek, "An Efficient Reinforcement Learning Based Approach For SDN Controller Placement Optimization," in *2024 41st National Radio Science Conference (NRSC)*, 2024.

[20] L. Mamushiane, J. Mwangama, and A. A. Lysko, "Given a SDN Topology, How Many Controllers are Needed and Where Should They Go?," in *2018 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pp. 1–6, IEEE, 2018.

[21] C. C. López *et al.*, "Parallelization of the Algorithm K-means Applied in Image Segmentation," *International Journal of Computer Applications*, vol. 88, no. 17, 2014.

[22] G. Wang, Y. Zhao, J. Huang, Q. Duan, and J. Li, "A K-means-based network partition algorithm for controller placement in software defined network," in *2016 IEEE International Conference on Communications (ICC)*, pp. 1–6, IEEE, 2016.

[23] I. Katsavounidis, C. C. J. Kuo, and Z. Zhang, "A New Initialization Technique for Generalized Lloyd Iteration," *IEEE Signal Processing Letters*, vol. 1, no. 10, pp. 144–146, 1994.

[24] "Internet Topology Zoo." `https://www.topohub.org/`. Accessed: June 2025.

[25] J. Liao, H. Sun, J. Wang, Q. Qi, K. Li, and T. Li, "Density Cluster Based Approach for Controller Placement Problem in Large-Scale Software Defined Networkings," *Computer Networks*, vol. 112, pp. 24–35, 2017.

[26] G. Wang, Y. Zhao, J. Huang, and Y. Wu, "An Effective Approach to Controller Placement in Software Defined Wide Area Networks," *IEEE Transactions on Network and Service Management*, vol. 15, no. 1, pp. 344–355, 2017.

[27] M. E. J. Newman, *Networks: An Introduction*. Oxford University Press, 2010.

[28] L. C. Freeman, "A Set of Measures of Centrality Based on Betweenness," *Sociometry*, vol. 40, no. 1, pp. 35–41, 1977.

[29] T. Opsahl, F. Agneessens, and J. Skvoretz, "Node centrality in weighted networks: Generalizing degree and shortest paths," *Social Networks*, vol. 32, no. 3, pp. 245–251, 2010.

[30] S. P. Borgatti, "Centrality and network flow," *Social Networks*, vol. 27, no. 1, pp. 55–71, 2005.

[31] D. Arthur and S. Vassilvitskii, "k-means++: The Advantages of Careful Seeding," in *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1027–1035, 2007.

[32] J. Jacob, S. Shinde, and D. Narayan, "An Efficient Controller Placement Algorithm using Clustering in Software Defined Networks," *Journal of Telecommunications and Information Technology*, vol. 2023, no. 4, pp. 9–16, 2023.

[33] S. Mastorakis, "Internet2:OS3E Topology Data." `https://github.com/spirosmastorakis/ndnSIM-1.0-naive_scenarios/blob/master/Internet2:OS3E/OS3E.txt`, 2015. Accessed: June 2025.

[34] P. Jurkiewicz, "TopoHub: A repository of reference Gabriel graph and real-world topologies for networking research," *SoftwareX*, vol. 24, p. 101540, Dec. 2023.

[35] "Internet2 Infrastructure Map." `https://internet2.edu/infrastructure/i2-network-map/`. Accessed: June 2025.

[36] "Python 3.13.0 documentation." `https://docs.python.org/3.13/`. Accessed: June 2025.

[37] "NetworkX Documentation." `https://networkx.org/documentation/stable/`. Accessed: June 2025.

[38] "Graph Modelling Language (GML) Documentation." `https://networkx.org/documentation/stable/reference/readwrite/gml.html`. Accessed: June 2025.

[39] "random — Generate pseudo-random numbers." `https://docs.python.org/3/library/random.html`. Accessed: June 2025.

[40] "NumPy Reference Documentation." `https://numpy.org/doc/stable/`. Accessed: June 2025.

[41] "Matplotlib: Visualization with Python." `https://matplotlib.org/stable/contents.html`. Accessed: June 2025.

[42] J. Rak, *Resilient Routing in Communication Networks: A Systems Perspective*. Computer Communications and Networks, Springer, 2nd ed., 2024.

LIST OF FIGURES

1.1 SDN architecture 8

5.1 Example of an Independent Dominating Set (IDS) in a simple network graph. Red nodes belong to the IDS. 30

5.2 Step 1: Selection of node 5 as the first controller (degree = 5) 34

5.3 Step 2: Selection of node 8 and other intermediate controllers 34

5.4 Step 3: Final partitioning of the network after four controller selections 35

5.5 Average Response Time on Internet2 OS3E 36

5.6 Average Response Time on Bell Canada 36

5.7 Average Response Time on ATT North America 37

5.8 Average Response Time on TataNID 37

5.9 Maximum Response Time on Internet2 OS3E 38

5.10 Maximum Response Time on Bell Canada 38

5.11 Maximum Response Time on ATT North America 38

5.12 Maximum Response Time on TataNID 39

5.13 Comparison of average propagation latency (K = 5) across methods. 47

5.14 Comparison of maximum controller load for different approaches as a function of the number of controllers in OS3E. 48

7.1 Atmnet Network Topology (USA) 60

7.2 Internet2 Network Topology (USA) 61

7.3 Geant 2012 (Europe) 61

7.4 IIJ (USA, Japan) 62

7.5 Pionier L3 (Poland) 63

7.6 Belnet 2003 (Belgium) 63

7.7 Bellsouth (USA, Southeast) 64

7.8 Cesnet 1993 (Czech Republic) 65

7.9 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology. 68

7.10 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology. 69

7.11 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology. 69

7.12 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Atmnet topology. 70
7.13 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology. 71
7.14 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology. 71
7.15 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology. 72
7.16 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Internet2 OS3E topology. 72
7.17 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology. 73
7.18 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology. 73
7.19 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology. 74
7.20 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Geant 2012 topology. 74
7.21 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology. 75
7.22 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology. 76
7.23 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology. 76
7.24 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the IIJ topology. 77
7.25 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology. 78
7.26 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology. 78
7.27 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology. 79
7.28 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Pionier L3 topology. 79
7.29 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Belnet 2003 topology. 80

7.30 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Belnet 2003 topology. 81
7.31 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Belnet 2003 topology. 81
7.32 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Belnet 2003 topology. 82
7.33 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology. 83
7.34 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology. 83
7.35 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology. 84
7.36 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Bellsouth topology. 84
7.37 Average Latency – Advanced K-Means vs Enhanced K-Means++ for the Cesnet 1993 topology. 85
7.38 Maximum Latency – Advanced K-Means vs Enhanced K-Means++ for the Cesnet 1993 topology. 86
7.39 Average Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Cesnet 1993 topology. 86
7.40 Maximum Propagation Delay (mean ± std) – Advanced K-Means vs Enhanced K-Means++ for the Cesnet 1993 topology. 87

LIST OF TABLES

3.1 Comparison of Controller Placement Problem (CPP) solution categories in SDN 22

7.1 Summary of Test Topologies Used in Experiments 66

LISTINGS

7.1 Advanced K-Means: Initial Center Based on High Degree 58
7.2 Enhanced K-Means++: Weighted Initial Center 58
7.3 Advanced K-Means: Farthest Subsequent Node Selection 59
7.4 Enhanced K-Means++: Probabilistic Center Selection 59

A. SOURCE CODE

Appendix A contains the complete source code developed for this thesis. The repository encompasses the implementation of all discussed algorithms, along with proposed enhancements, as well as scripts for running experiments, generating plots, and collecting results.

Location:

`/Appendix/Appendix A - Source code/`

B. PLOTS

Appendix B includes high-quality plots illustrating the experimental results analyzed in the thesis. The appendix provides all the figures in full resolution for both the baseline and the extended algorithms, for each considered network topology.

Location:

`/Appendix/Appendix B - Plots/`

C. RESULTS OF THE EXPERIMENTS

Appendix C presents the raw experimental results in the JSON format. The datasets cover all scenarios described in the thesis, for both the baseline and the extended algorithms, and for every analyzed topology.

Location:

`/Appendix/Appendix C - Results of the experiments/`

D. LOADS OF THE CONTROLLERS

Appendix D provides the results of controller load measurements in text format. The appendix contains detailed statistics for the baseline and extended algorithms, across all topologies.

Location:

`/Appendix/Appendix D - Load values/`